INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

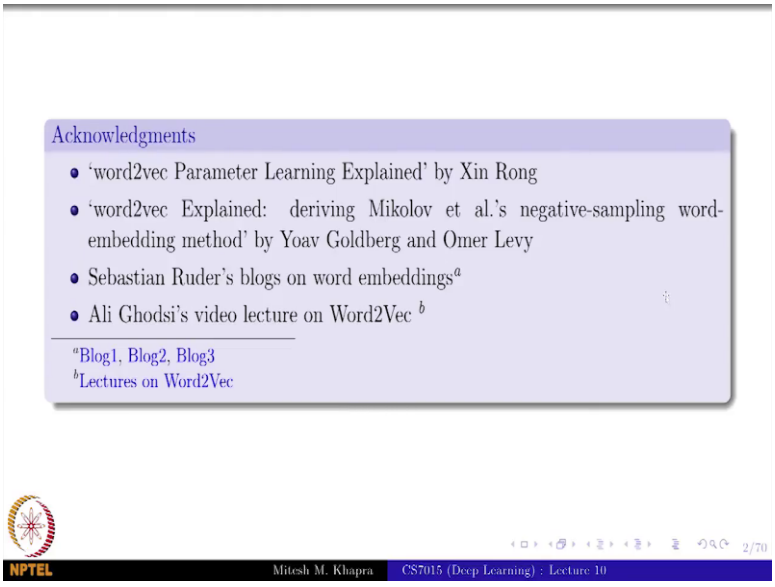
112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Lecture – 10
Learning Vectorial Representations of Words

So, today we are going to talk about learning vectorial representations for words.

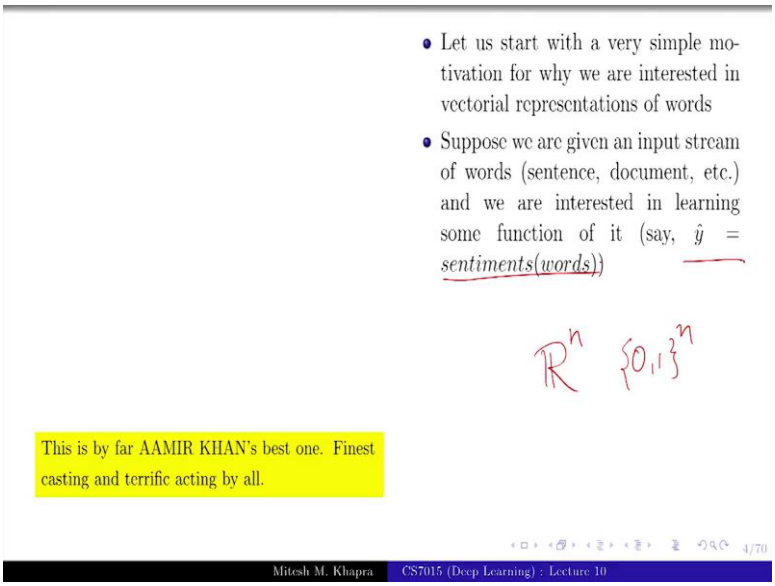
(Refer Slide Time: 00:18)



So, these are the acknowledgement slash references for where are the things that I have referred to by preparing for this lecture. So, you can just go this some of these are also available as video lectures on YouTube can take a look at them also.

In the first module, we are going to look at one hot representations of words.

(Refer Slide Time: 00:39)



So, as usual we will start with this motivation or motivation, motivating question why do we need to learn representations for words. Vectorial representations for words, when words are there right, you can write them using alphabets and characters and so on. So, why do we need vectorial representations? Mention whatever you have seen so far in the course, I have seen anything like this let us see.

So, suppose you are given an input stream of words, and it could be a sentence or documented. If I say documented pretty much covers almost all the text that you see right, you can always abstract everything as a document and email is also a document, a manuals are also documents and so on.

And we are interested in learning some function of it. And so, I am given a document and I am interested in the function y hat. Say y hat is equal to sentiments of the words in the document or the sentiment of the document itself.

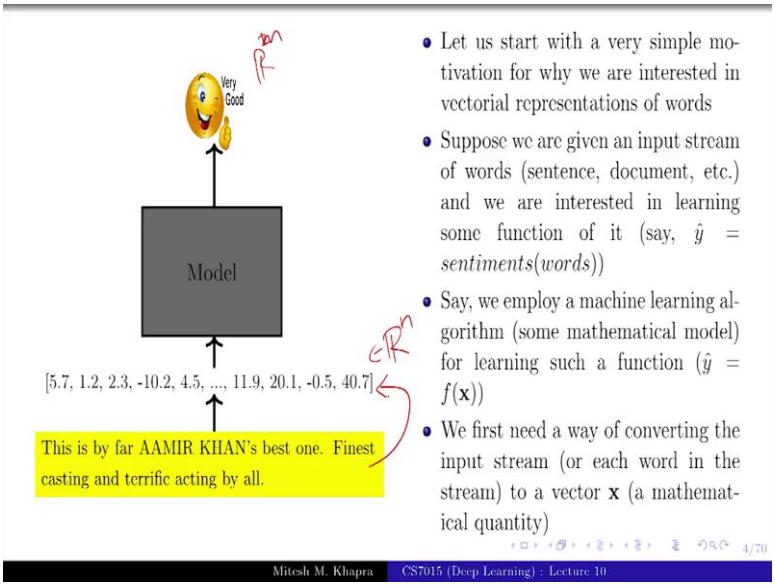
This is imaginable this is not something that I am cooking up this is something that, you would want to do you would log on to for example, if you are a movie maker you would want to know once the movie is released people have written reviews about it what is a sentiment is positive or negative. Similarly, if Apple has released a new product or a new feature you would want to know what are the reviews written about this product and what is the feature what is the sentiment coming out of it is positive or negative.

Now, sentiment is a binary thing, or it could be rated also right it could be on a scale of one to 10 also, but let us consider it is binary that either people liked it or did not like it.

So now I am trying to learn this function, which gives me which takes as input words, but as output gives me real numbers, either 0 to 1 or on a scale or 1 to 10 or whatever. right And this is not something that we have dealt within the course so far, let how do we take as input word. So, all inputs have already always been numbers, right? They were either coming from Rn or they are coming from 0 to 1 raise to n or something of that sort.

We never had the situation when we have words as written. So, right so now how do we deal with the situation and also I have made a case for that learning this function is a valid thing to do, you have several news cases where you will need this.

(Refer Slide Time: 02:41)



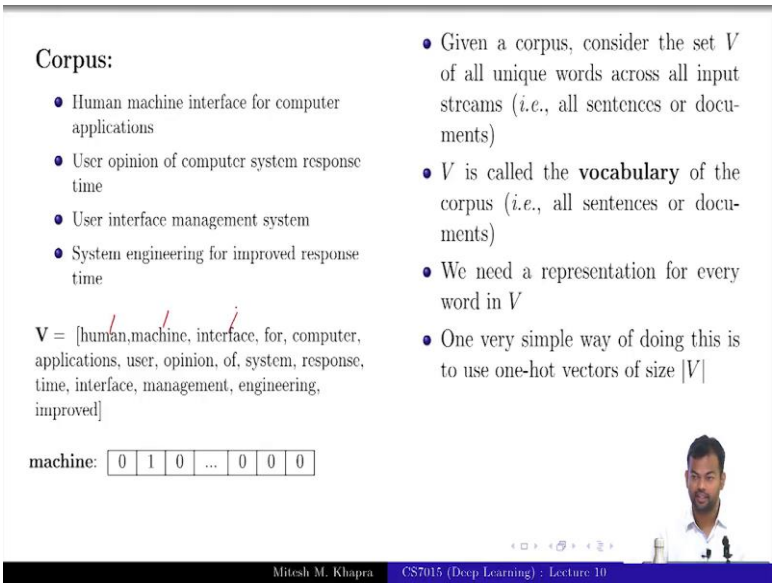
So now, if we employ a machine learning algorithm, that some mathematical model; so we saw that we could have several such models logistic regression, SVM and neural network and feed forward neural networks and so on, right? And at the end we are trying to learn this function y hat is equal to f of x, but in our case, the x instead of have be instead of x being numbers it turns out that x is actually a collection of words.

So now how do we reconcile with the situation where we have suddenly have words instead of numbers. So, the way to do that would be we need a way of converting these

words or documents into some number, into some vectorial representation and once we have this vectorial representation, right? So now, we have R, R raised to n and we know how to deal with R raise to n given R raise to n, how to predict R or even R square or Rm in general, that we know right? We can design neural networks or any other machine learning algorithm should do that. But how do we go from here to here that is the question, right? And that is why we need to learn vectorial representations of words.

This is a motivation clear to everyone? Okay. Now let us start getting with a (Refer Time: 03:52) how to do that right?

(Refer Slide Time: 03:54)



So, now we will start hearing this word corpus. Have you heard this word before? That is exactly what you are collecting for the word to like assignment right you are collecting a corpus in specific languages. And you have taken a very Toyish corpus for the purpose of illustration. So, here is a corpus it just contains 4 sentences, right? So, think of it that I have a very restricted domain, I have very small set of documents and I just have these 4 sentences with me, this is the valid corpus.

The corpus that you have constructing is probably much larger scale you are trying to collect 100 thousand sentences or 50 thousand sentences or something of that order right? Ah, But we will take this toy example.

Now, consider set V of all unique words across all these input streams. So, I just call them input streams by input streams I mean; sentences or documents or whatever right. You could take it as any sequence of words. And V is set of all unique words across all this input sentences that you have.

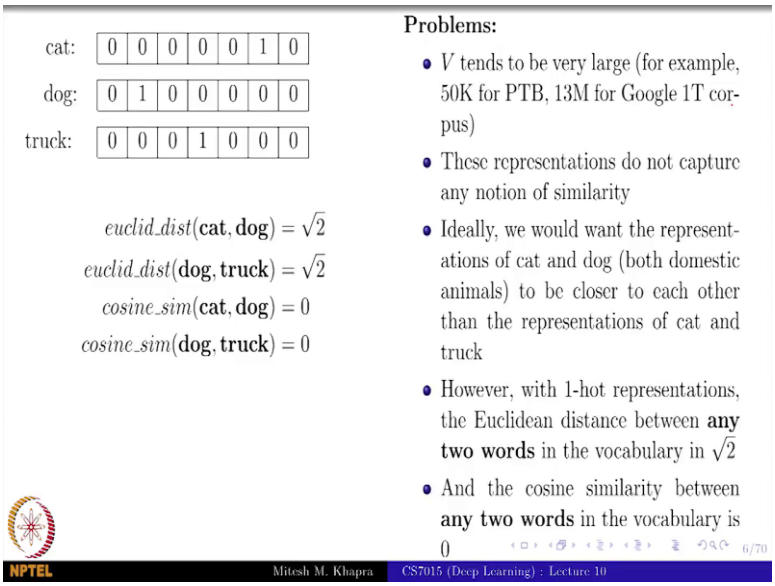
So, can you tell me what V is here? What would can you tell me some elements of V of the set V? Human machine interface and so on. So, that is why, in fact this is the entire set V which is written on the left hand side right. And V is called the vocabulary of the corpus so; that means, everything in the corpus comes from this vocabulary, all sentences are constructed by arranging words from this vocabulary.

Some of you might always I mean find this very trivial, but I am just going over the basics. So, that at least the terminology is clear to everyone. And what we want is a representation for every word in V. So, that is the title of the lecture learning vectorial representations of words. So, for every word in our vocabulary whatever corpus we are dealing with the vocabulary would change, and for every word there you want to learn a representation for that word. So, that is what our quest is today, ok?

And now one very simple way of doing this is right, you tell me you want a vector and that is all you care about. Here is a vector I will give you one hot representations. So, if a total number of words in my vocabulary is V. I just construct a vector of size V ok. And I have assigned a number to every word in my vocabulary, right? So, I will say human is equal to 0 machine is equal to 1 interface is equal to 2 and so on.

And if you ask me for a vectorial representation of that word, I will just say take this the or vector of size V, and switch on the corresponding bit and anything else would be 0. Hence 1, hot right at any given point of time only one of the elements in the vector would be on. So, that is a simple one hot representation as this is a very simple recipe to get a vectorial representation of words, and for every word in your vocabulary.

(Refer Slide Time: 06:31)



Now, what is the drawback of this? V tends to be very larger, right? So, for example, there is a standard corpus known as the Penn Treebank corpus, which is used in various NLP applications for various reasons. And that corpus has a vocabulary of 50 k.

Google of course, operates at it is own scale. So, they have a word 1 T corpus which has 13 million words. So, this is like all the most of the web pages that they have drawn constructed a vocabulary from that.

So now I am talking about, for every word representing it by a vector of size 13 million, theory does not good work right there is too much of storage required for that. And if you look at that information in it is so redundant that is all 0 except for that one bit which is on.

And the other important problem is that these representations do not capture any notion of similarity. Other 3 words that I have shown you which do you want to have similar representations? Cat and dog why, because both of them are domestic animals right? Both of them are mammals. So, there are some things that you would want at least, at the minimum that the similarity between a cat and dog is more than the similarly between cat and truck.

Or alternately the distance between cat and dog is less than the distance between cat and truck. So now, once I start talking about vectors, I can talk about similarities like cosine

similarities or I can start talking about Euclidian distance. So, once anything I convert it to a vector I can start asking these questions other 2 questions. Which I am asking are valid right what would you expect to be the Euclidian distance between cat and dog as compared to cat and truck.

Now, what happens with the one hot representations? Take any 2 words in your corpus, any 2 what will be the Euclidian distance? What will it be? Square root of 2, right? For all the words. Take any 2 words in your corpus what will that cosine similarity, mean 0 because all these vectors are orthogonal, right? So, the cosine similarity is going to be 0, but this is; that means, these vectors are not really capturing any information about the essence of the word, right?

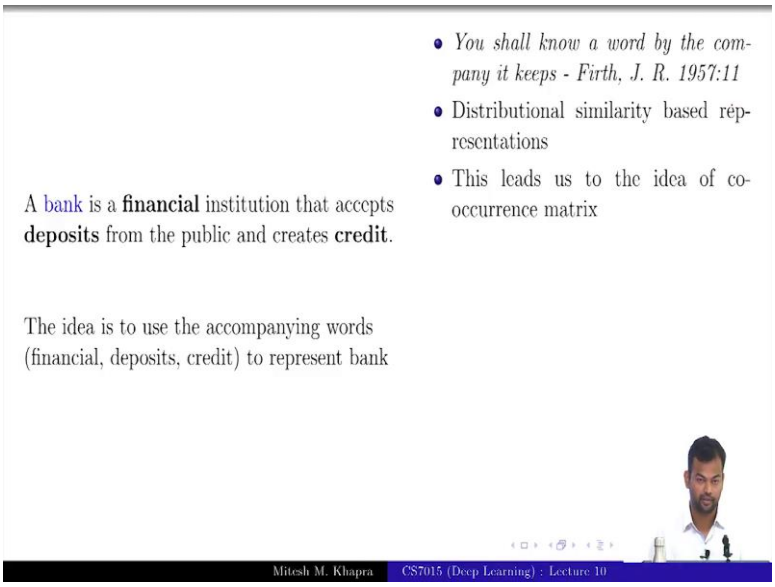
So, remember always we are interested even like that has been our philosophy right from auto encodes. And so, right or even principle component analysis, they are always interested in learning meaningful representations which capture something fundamental about the entity that we are trying to represent, right? But here something like that is clearly not happened, ok? So, that is not acceptable.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Module – 10.2
Lecture – 10
Distributed Representations of Words

So, what we saw now was sparse representations; or one hot representation sparse. Because, only one bit is on from there, we will move on to something known as Distributed Representations of Words. And you have already seen that sparse is in theory, but it gives a very simple recipe of converting words to vectors, but it does not serve much purpose.

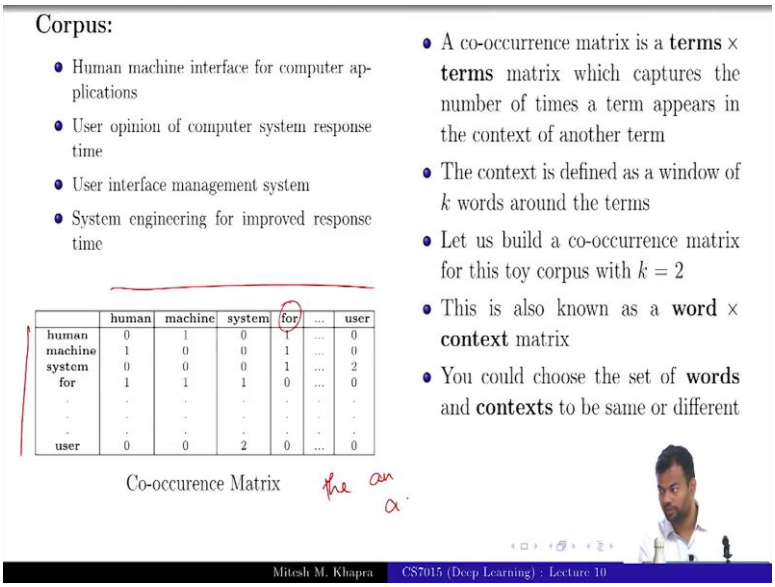
(Refer Slide Time: 00:36)



So, let us see what distributed representations of words are? So, this around 1957 J R Firth made this very profound statement that, you shall know a word by the company it keeps and this of course, a play on some other similar code, but what does this actually mean; so it means that, you want to know what does the word bank convey; or what is the essence of the word bank, right? What this code says is that, if you want to know about bank, you should say, you should see the company that it keeps; that means, what are the other words which appear typically in it is neighborhood? And of course, when you have a large amount of corpus given say the entire Wikipedia.

Of course, at that time Wikipedia did not exist, but any large corpus. And does this led to something known as distributional similarity based representations. So to understand this, we first have to understand the idea of a co-occurrence matrix.

(Refer Slide Time: 01:38)



So, the basic idea is to use the accompanying words, which in this example happen to be financial; financial deposits credit etcetera to represent bank and to do that we will construct something known as a co-occurrence matrix which looks like this, right? So, a co-occurrence matrix is a terms cross terms matrix; that means, every row in the matrix corresponds to a term or a word and every column in the matrix also corresponds to a term or a word.

Can you guess, what how many rows would there be? Size of the vocabulary how many columns would there be size of the vocabulary, right? Okay, so here is how we construct a co-occurrence matrix. So, we take a word we are interest interested in constructing the row for that word. The number of columns is the same as the size of the vocabulary.

Now for every column we will make an entry, which tells us whether or how many times this this word appeared in the context of the target word, right? For example, if I look at machine, I am looking at the row for machine; I am trying to construct the entries in that row. I know that the number of columns is equal to the all the unique words in my vocabulary.

So, I look at the first word which is human and in that cell I enter the value, which is the number of times human appeared in a window of k words around machine; is that fine is that straightforward?. And that is how, I will construct this co-occurrence matrix, where I have taken the window size as 2; that means, in any given cell my entry would be the number of times human appeared within a 2-word window of machine is, right the ig'th cell clear, the definition of the ig'th cell clear. So, this tells me that user actually appeared 2 times around the word system in a window of 2 words around it is that clear?

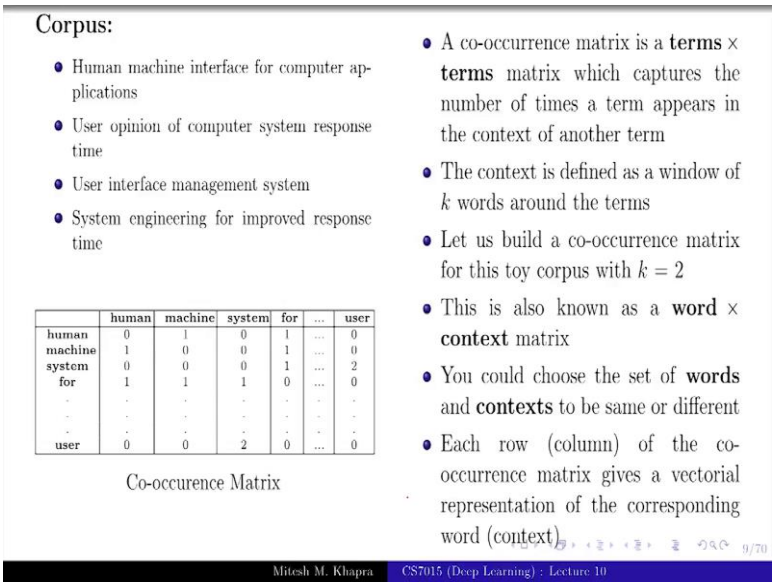
Everyone gets this how I construct the co-occurrence matrix, ok. Now, you could use the same, so this is known as words and this is known as context; that means, the rows we refer to them as words and the columns they refer to them as context. Now as you said that, the number of rows and the number of columns can be same that we can consider the same words in the context, as well as the same word as the target words, right?

But you could also do something different you could say, that I do not want to consider all words that is context words. Because, for example the word for appearing with any other word does not really give me much information because, it is just a stop word right or the word the or and or a these are known as stop words in the language, these do not really give me much information.

So, if you go back to the bank example, financial credit deposit are the words which I really care about and these are the financial bank or with deposits, and also these words do not really matter a lot ok, do you get the intuition?

So, you could choose to have fewer columns, which are only the important words that you consider and you ignore the stop words ah. In this discussion, I will alternately switch between considering the columns as the same as the rows and sometimes are restricting the columns to fewer number of entries.

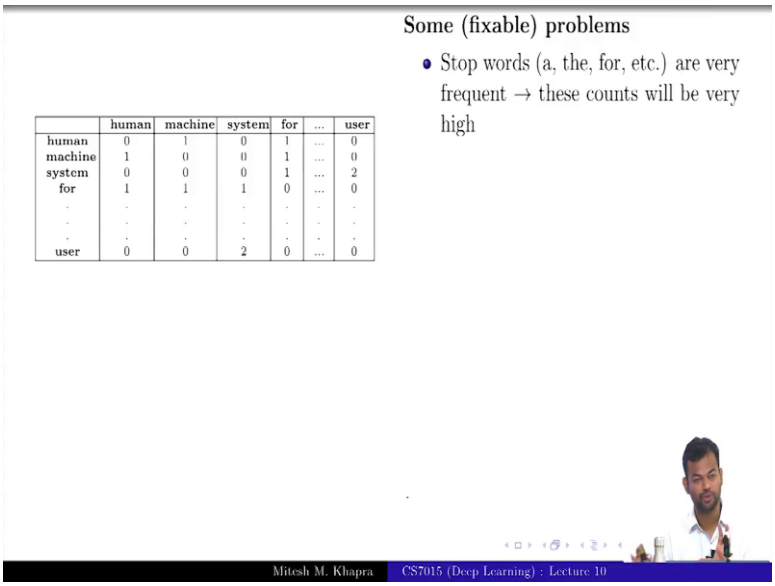
(Refer Slide Time: 04:38)



So now, each row gives us the vectorial representation of the word. So, we have seen how we have moved from sparse representations to distributed representations. So now take a guess now would this vector be sparse.

So, we saw the extreme sparse right which was one hot. Now, the vectors which you get here are they going to be dense or still sparse, sparse, right? Because every word does not appear with every other word right; you still have these V dimensional vector and there are some words which will appear with very few words, right? So, you expect to have non 0 entries in very few columns right; so, these representations are also sparse.

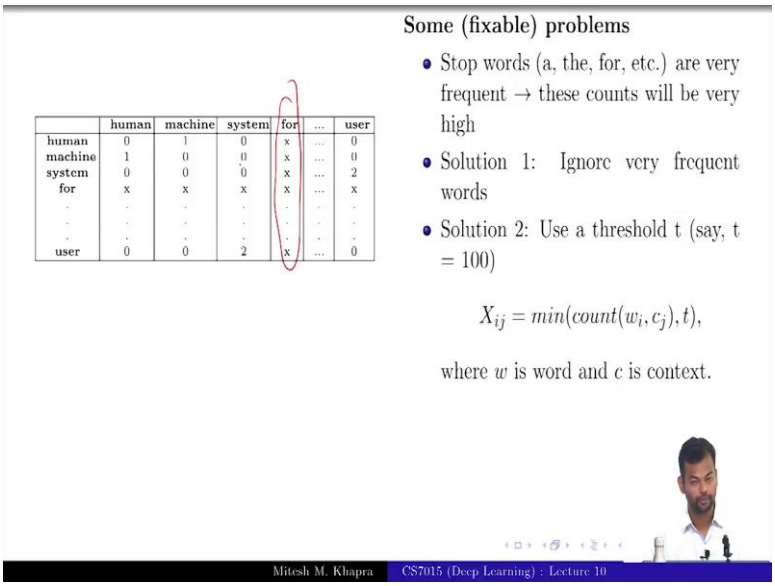
(Refer Slide Time: 05:19)



So, there are some problems some of which are fixable. So, we look at the fixable problems first. The first thing as the stop words are very frequent so, these counts should be very large.

So, if you take the entire Wikipedia corpus and you take the word machine or system. Then, the words the, and, for and so on would have appeared like more than 1000 times in the context of the word machine, right? And as compared to the other words like system or user they would have appeared much fewer times. So, this kind of skews your counts right, it is like highly biased in the favor of stop words. So, how do you deal with this?

(Refer Slide Time: 05:55)

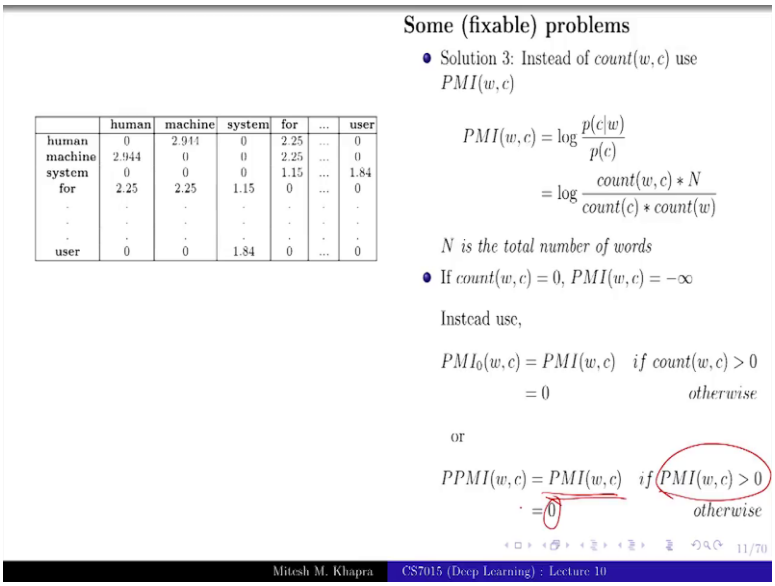


So there are 2 ways, one is ignore frequent words right. So, that is the solution which I suggested earlier; that your number of columns would be less than the number of words is that fine? So, you do not actually consider frequent words at all.

The other is user threshold t so; that means, in these columns like for and with and so on, whatever be the entry, if that crosses a certain threshold then, I will just replace it by that threshold is that clear?

So, I am just saying that, this means that the word has appeared more than 100 times. And I am not interested in keeping the actual count which was more than 100. I just saying that, more than 100 is enough for me right. Because, I know that all the other entries are going to be much less than this. So, just like replacing it by a very large number instead of actually counting that number.

(Refer Slide Time: 06:43)



The other solution is instead of count you can use something known as PMI. So, this is how you compute PMI. Even if you do not know, it does not made a lot of difference because, you know that it will always be there on the slide that is why you guys do not read anything. So, PMI is computed like this. So, intuitively tell me what does PMI capture? Look I would say focus on this formula rather than the upper one; when would it be high? The easier question to answer is when would it be low; remember you are dealing with a fraction.

So, if independently the 2 words appear a lot of times. But together, they appear very rarely then the PMI is going to be low is that clear? Now if both the words appear 100 times and together also they appear 100 times, that is the best case scenario; that means these are very tightly tight words right, they always tend to appear together.

So, the PMI would be high for words which are very frequently co-occurring. Now, so this is what would happen, if you replace the counts; the co-occurrence counts by the corresponding PMI, ok. Now, if the count of 2 words is 0 we have a problem because, then the PMI tends to be minus infinity, right? So, how do you deal with this situation? Epsilon or some we will use some hack right as usual.

So, instead of PMI use something known as PMI 0, which works like this. If the count is greater than 0, then you use PMI; if the count is equal to 0, then you just put the entry 0

in the set, make sense. There is also something known as positive PMI, which is slightly more extreme; it says that use the PMI only if the PMI is greater than 0 otherwise use 0.

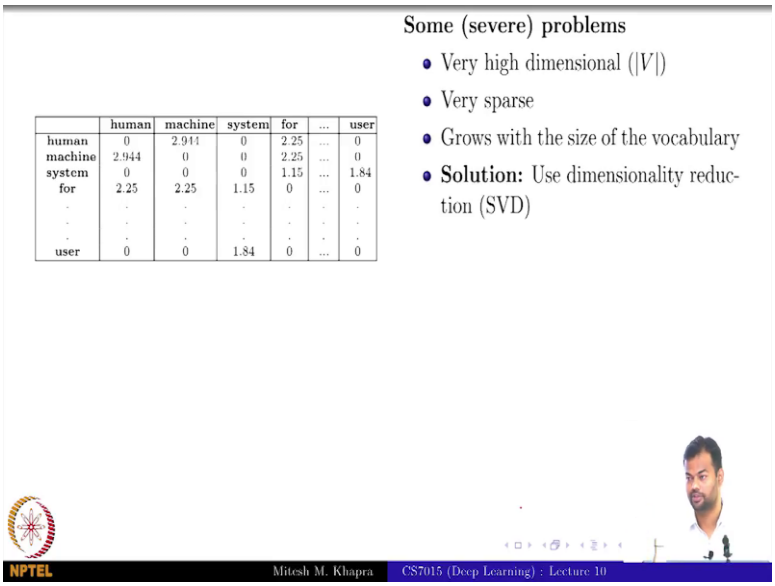
Can anyone tell me the rationale for doing this? You see the subtle difference between the 3 things right. One is of course doomed because, you cannot handle 0 counts. The other one is saying that, if the count is 0 then, I will just substitute 0, the last one is saying that, if the PMI is negative right. Then I will replace it by 0; that means, in the last case all the cells in your or in your PMI matrix would be positive right, non negative rather.

So, can you tell me the rough intuition for using this and there is only a rough intuition, but can you tell me, so the very rough intuition for this is, that what does it even mean to say that 2 words are negatively correlated. I mean, either they occur together right; which means, there is some relation between them, but a negative relation between words does not make sense that is the intuition behind this. Now of course, I could argue that what about antonyms and things like that, but that is also not the same right, because you could have good and bad in the same sentence, right?

But that is the roughly the intuition that negative values do not mean much. So, just replace them by 0s.ok There is no again a formal reasoning behind this, but just the intuition. So, we have looked at the co-occurrence matrix where, we started with counts. These counts were very sparse and there are also some other problems with counts in the terms of some frequent words taking a lot of limelight and so on.

So, we have fixed although then, we have done some very minor and simple fixes. And I just very rush quickly rush through them because they are very simple. But these were all fixable problems, what a non-fixable severe problem with this? What is the problem with the one hot representations? Large.

(Refer Slide Time: 10:27)



What about these representations, still large it is still of size V? It is still very high dimensional, still very sparse not as parse as the one hot encoding, but still sparse, and it grows with the size of the vocabulary. So now, remember that Penn treebank at 50 k words, Google 1T corpus at tha 13 million yeah. So, it keeps going with the size of the corpus.

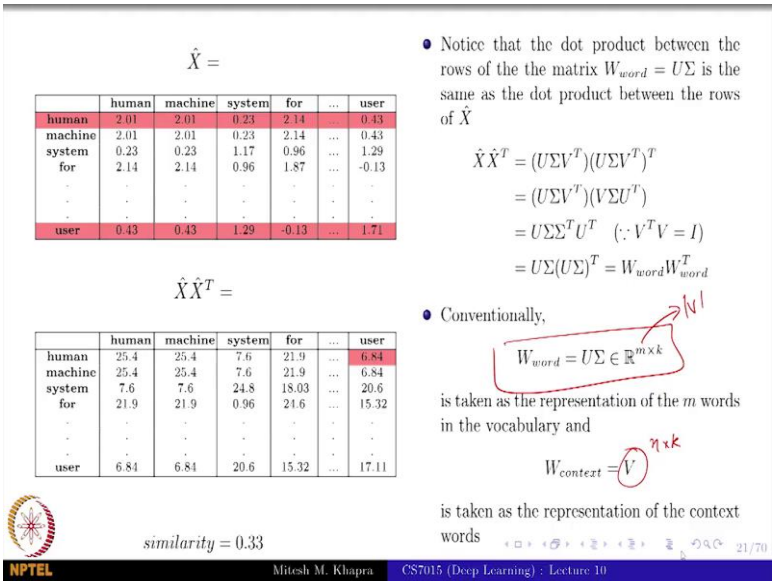
So now how do you know? How do you fix this? I wish I had that Harry Potter thing. Anyone remembers that spell to wipe out your memory? How would you deal with it? So you now see how it connects? So now, again you have ended up in a situation where, you have a very high dimensional matrix, right? And you are looking for ways to reduce the dimensions; so we will go back and rely on things that you have learned and one of those was SVD right? So, you can use singular value decomposition.

Why did I say SVD and not PCA? Because, this is not necessarily a square matrix A; this could be a rectangular matrix and for all practical purposes SVD is just a generalization of PCA.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Mod-03-Part 2
Lecture – 10
SVD for learning word representations

(Refer Slide Time: 00:12)



So, we will start from where we left yesterday ah. So, we did this whole story on starting from co occurrence matrices, we learnt how to get better word representations and the key thing there was we used SVD as a dimensionality comp reduction tool. And we came up with this neat result that you could use W word as the representation of the mate of the words, it has m rows and k columns where k is very less than the size of the vocabulary.

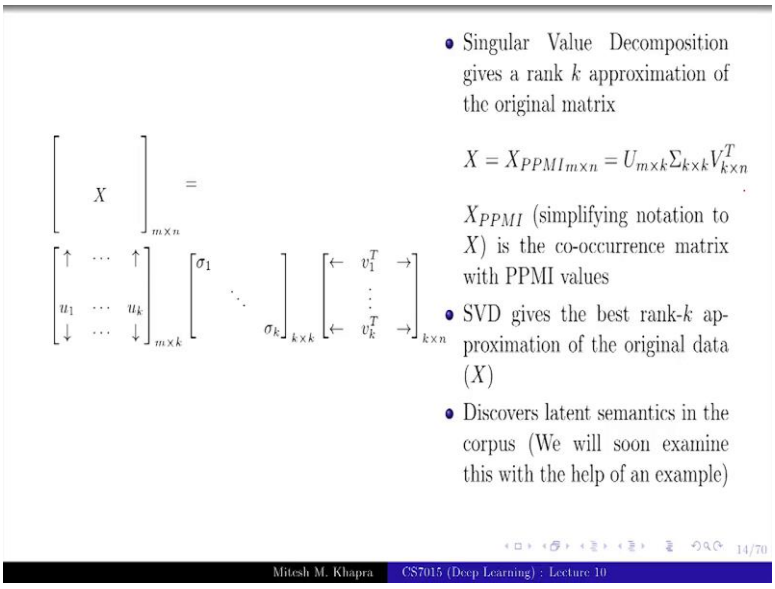
So, you have achieved lot of compression and you are still able to learn very meaningful representations, which you could use for several downstream tasks. What to use these for and how to use these for you will see that later maybe four lectures from now, I mean I say 4 lectures I mean 4 2 hour lectures right? So, it might be more in terms of actual lectures. So, we will get to that, but for now we have a way of learning representations for words.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Module – 10.3
Lecture – 10
SVD for learning word representations

So, in this module we will talk about using SVD for learning word representations.

(Refer Slide Time: 00:18)

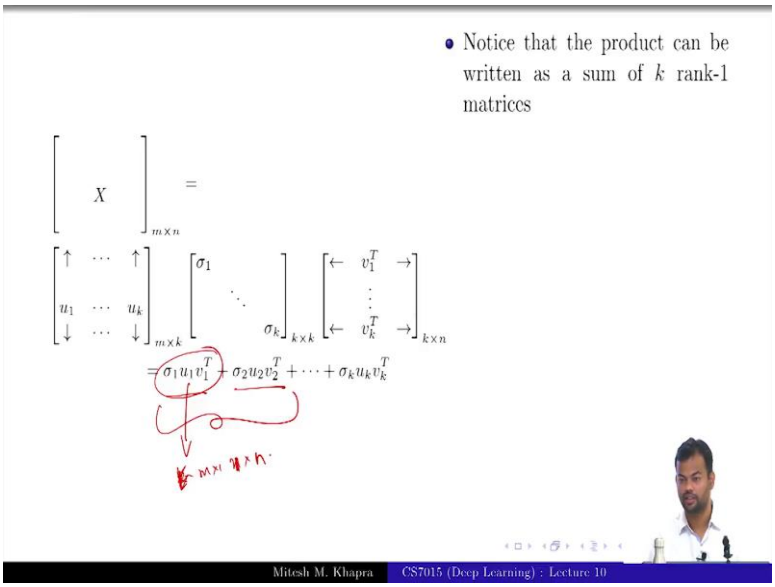


So, what does singular value decomposition do? yeah these are all possible variants. So, people have tried various things and one of the PPMI one is the is the most reliable thing that is what is given. But you can think of, I mean you said one there are 10 different things which we can do for the co-occurrence matrix, right? But this is the most popular and most stable thing to do.

Yeah what is the single value decomposition do? can you read it from the slide please. It gives the rank k approximation of the matrix. So, let me start defining a few things. So, from now on when I refer to the co-occurrence matrix I would mean the XPPMI matrix right which was the positive PMI which was replacing all negative PMIs by 0, and just do not have this nasty variable I will just call it as X.

So, from now on whenever I say X, I mean the positive PMI co-occurrence matrix ok. So, that is what this matrix is ok and we know that SVD gives us this reconstruction of the original matrix. And fine it gives us the best rank k approximation of the original matrix, and it discovers the latent semantics in the corpus. Everyone remembers this like that is what we were by we were using PC and SVD and auto encoders it was able to discover some latent semantics. And we will concretize this intuition with the help of our current example, but for now I just want you to recall that it helps in discovering the latent semantics.

(Refer Slide Time: 01:49)



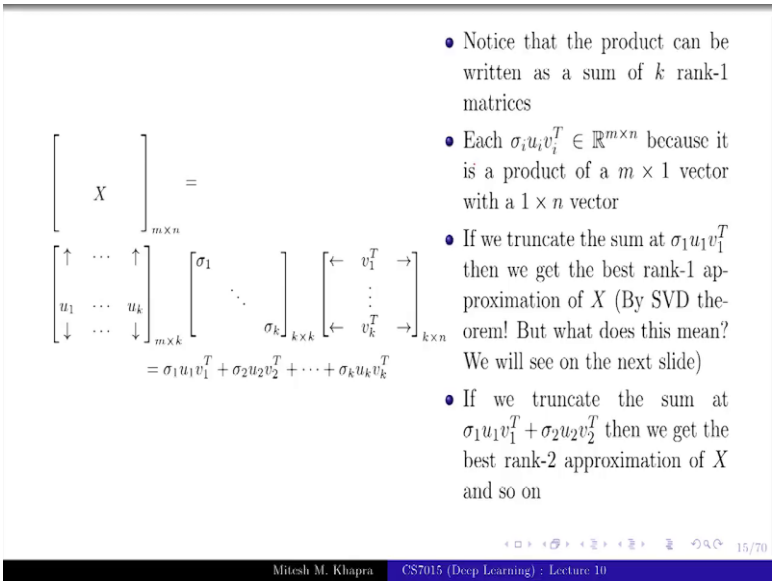
Now, notice that this product and I think I have done this in one of the assignments or something can be written as a sum of the following products. So, I can write it as sigma 1 u 1 v 1 transpose, sigma 2 u 2 v 2 transpose and so on. Can you tell me what this sum is this is the rank 2 approximation of the original matrix and I keep taking more terms I get more and more rank approximations of the original matrix ok Now, and we all know that, ok. We all hopefully know that what is the dimension of this? It is a scalar vector matrix, scalar vector matrix.

Student:Matrix

Ok. Now of course, you will say matrix, but what is the dimension of the matrix; why is it a matrix? It is an outer product of 2 vectors right this, what is the size of this? n cross 1 into n cross 1 so that; sorry 1 cross n that gives you n cross n matrix. Everyone gets this

otherwise how is it a rank 1 approximation you have to get the original dimensions right. Everyone is clear with this is an outer product.

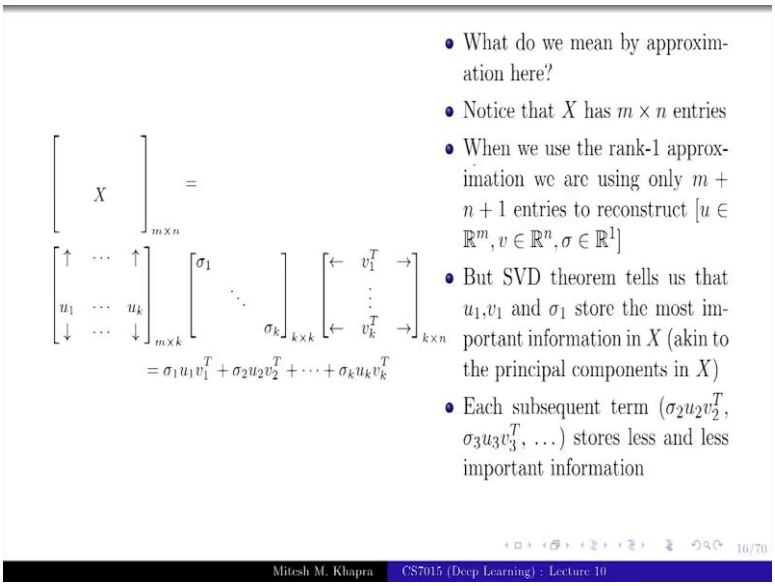
(Refer Slide Time: 02:49)



And it belongs to R m cross n. Ok and if we truncate the sum at the first term we get the rank 1 approximation and by SVD theorem we know that this is the best rank 1 approximation.

Now, what does this actually mean? That this is an approximation, what do we mean by that? So, we will see that on the next slide and similarly in the same way if we truncate it in the second term you get the same best rank 2 approximation.

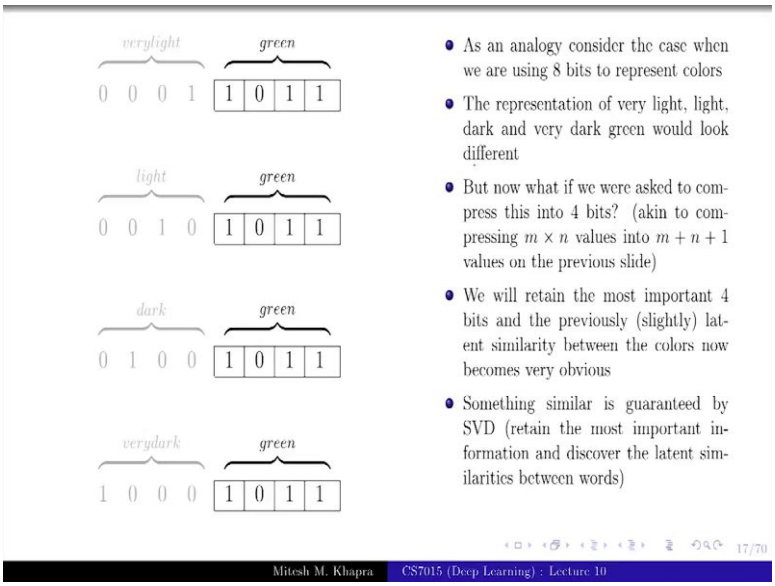
(Refer Slide Time: 03:14)



Now, what do we mean by approximation here? Actually and I mean to say approximation always in this course at least try to think in terms of compression. How many elements are there in the original matrix? m cross n, that is how many elements you need to describe the matrix completely. If you do a rank one approximation how many elements are you using? m plus n plus 1 right? So, the original matrix has m cross n entries, entries and when you do a rank 1 approximation you have m plus n plus 1 entry. So, that that is the approximation, right? So, you are trying to really compress the original data using only these many variables you get that. ok

And if we do a rank 2 twice this right; So, as many rank I mean as deeper as you go in the sum you will have that many elements to do the approximation, ok? But what is important is that the SVD theorem tells us that this is not just any random approximation. But this is the best approximation that you could have done; that means, if you wanted to use only these many elements, these are the best elements to use, right? Everyone gets that.

(Refer Slide Time: 04:16)

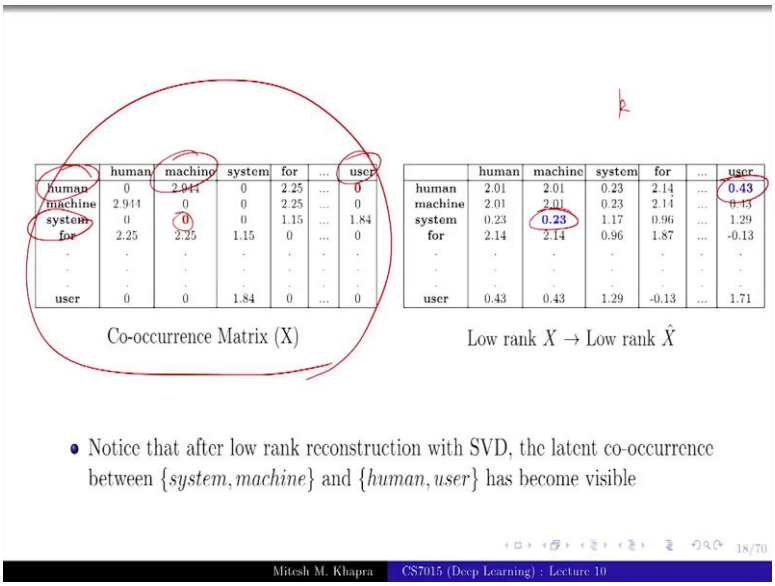


So, as an analogy consider this right suppose you are given 8 bits to represent colors, ok. And this is how you represent very light green, light green, dark green and very dark green this is what your representation is.

In this original 8-bit representation, there is some similarity between the colors, but it is still a bit latent, but now if I were to ask you to use only 4 bits to represent these colors, what would you do? The lowest significant bits; if you use the first 4 no then use only get very light that is not the essence of that color right, you need the color to be there. So, if you compress what would happen is. So, that is what happens in when you go from 256 bit colors to higher or lower, right? The distinctions between the colors go off.

So, all of them would be compressed to green well that is the most important, important information in terms of the color right, because you need to be able to distinguish between green and red, as suppose to very dark and very light that is the more important information that is there, right? So, when you compress it the most important information in that entity should be retained. And that is exactly what SVD does when it does a compression, it retains the most important information in the corresponding entries is that clear is the intuition clear fine.

(Refer Slide Time: 05:32)



So, let us actually do this. So, this is my original co-occurrence matrix X, and I just repeat when I say, X I mean X PPMI. And now I have done SVD and I have done a low rank approximation of it. I do not know what was the value of k I selected, but some value of k it was definitely greater than 1 or 2. So now, you see a low rank approximation of X, what is the first obvious thing that you notice? It is dense now it is the longest sparse.

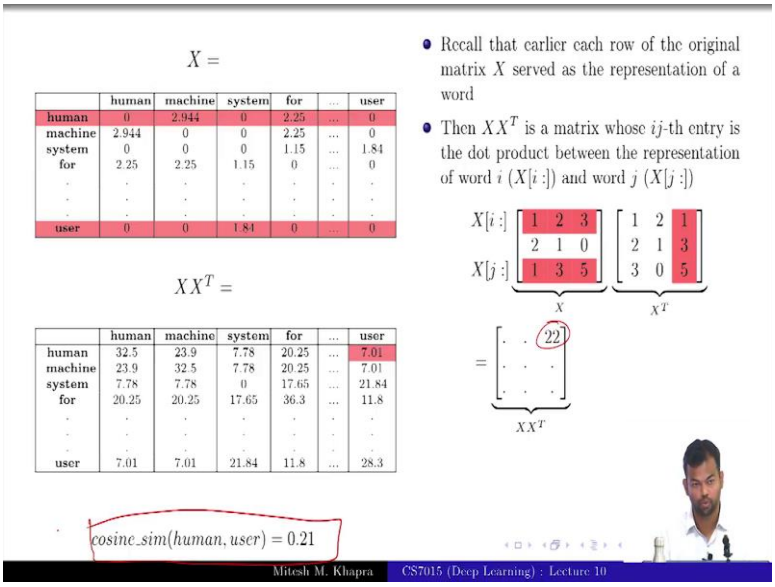
Now, can you tell me something about the colored entries, what was happening in the original matrix X? The word system and machine was never co-occurring because of which their value was 0. Same for human and user, but remember there is some important information in this matrix, which also tells you what are the words with user appears with and what are the words with human appears with, and that actually gives you intuition that these two words are actually related right same for system and machine

System and machine both would appear in the context of words like interface, install, run and so on. So, you know they are similar it just happens that these two words never appeared together. So, this similarity between them was latent or hidden in the original co-occurrence matrix. Now once I have done the SVD what has happened because I have forced it to compress the data, it has retained the most important in information and under that information these two words have actually come closer to each other, right? So, you see that now you have a non-zero entry for the similarity between those two

word pairs do you get the intuition and can you imagine that this would happen with SVD

And what is wrong in imagining you can, but I guess right that is what is happening with this. So, you think about PCA you think about SVD you think about auto encoders all the intuitions that we had build there the same is being applied here, right? All if you get this ok fine. yeah. After SVD you could have right that is not necessary that it should be positive in the original matrix you do not have negative entries.

(Refer Slide Time: 07:31)

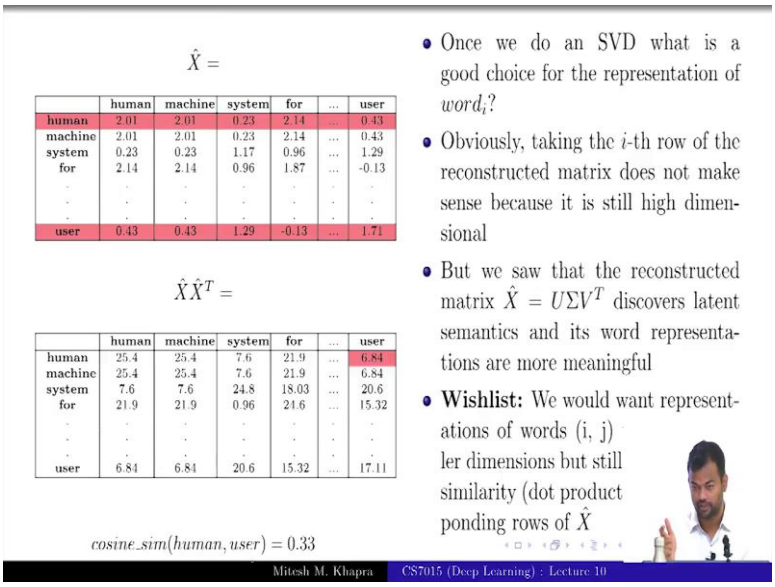


Now, here is a question right recall that earlier each row of the original matrix X served as the representation of a word, ok? This was my original X PPMI not the rank approximation now in that case what would X X transpose give me? What would the ij th entry of X X transpose be. So, let us look at this toy example you have this X matrix you have xi and xj now I take X transpose, ok?

Now, this is Xi this is Xj just standing. Now what would be the ij th entry of X X transpose, it will just be the dot product between these two right; is that fine? So, this is just the dot product between them and we know that dot product is more or less the same as cosine similarity module over the normalization right, you just need to normalize it by the norms of X and Xi and Xj in this case, right?

So, I will just assume that this is a substitute for the cosine similarity, ok? So, every entry at every ij'th cell in X X transpose is the cosine similarity between the representations of the i'th word and the g'th word, is that clear to everyone? Ok fine. And in the original case which was the XPPMI the cosine similarity between human and user was 0.21.

(Refer Slide Time: 08:51)



Now, once we do in SVD what is a good choice for the representation of the word i. After SVD what is the dimension of X hat? It is again n cross m because it is a sum of m cross n matrix. So, that the dimension of X hat is m cross n. Although it has been constructed using fewer information, but the dimension is m cross n right; that means, what is the size of the representation of every word? Still high dimensional still the same n or v, whatever everyone gets that is there any confusion with that, ok?

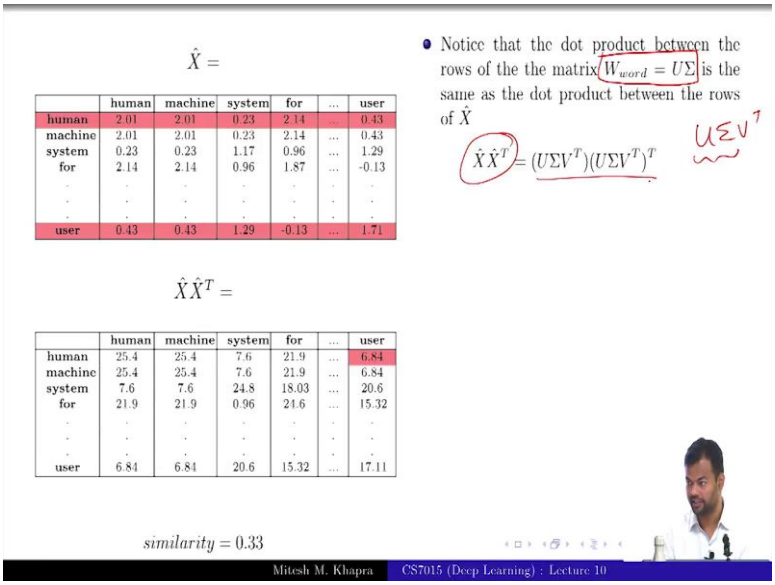
Now you could say that ok, I will just take the i'th row of the reconstructed matrix and use that as the representation. Because I know that now this representation is better some of those 0 entries have changed they have captured the latent semantics between the words. So, this is definitely better none is denying that that this compression has given us better representation because we are only keeping the most important information.

Now, if I do X hat X hat transpose, remember X hat is the reconstructed matrix. Then again by the same argument the ij'th cell actually gives me the cosine similarity between the I th word and the j th word. And you can see that now the cosine similarity between human and user has actually increased. So, this is just for me to convince you that we

have learned more meaningful representations. So now, what do we choose as the representation? I have still while computing this cosine similarity I have still used Xi which is high dimensional which has the entire vocabulary as the number of columns as a representation, right?

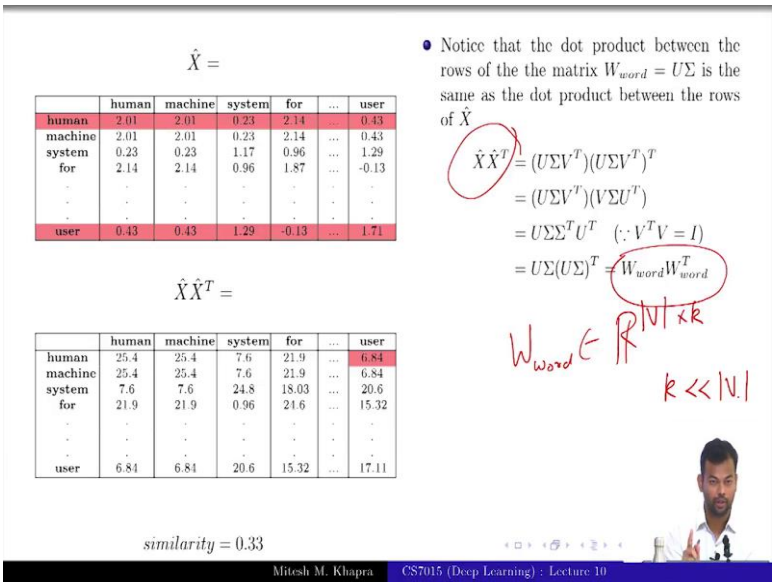
So, there are 2 things coming out of here one is I really like this cosine similarity I see that it has improved; that means, the representations were computing something meaningful, but on the flip side I am still not happy because the representations are still high dimensional. So, can you construct a wish list for me based on this. I would want the same cosine similarity to be present as given by X hat, X hat transpose right, but I would like to represent it by fewer dimensions, that is exactly what my wish list is, ok?

(Refer Slide Time: 11:01)



So, let us see how do we do that now for no reason I am going to construct a matrix W word equal to U sigma what is u sigma? it is the part of the SVD right the SVD told us it was U sigma V transpose. So, I am just considering this matrix I am going to call it W for no particular reason, ok? Now, let me take X hat X hat transpose, I can write it as this is that fine? Now what is the next step?

(Refer Slide Time: 11:32)

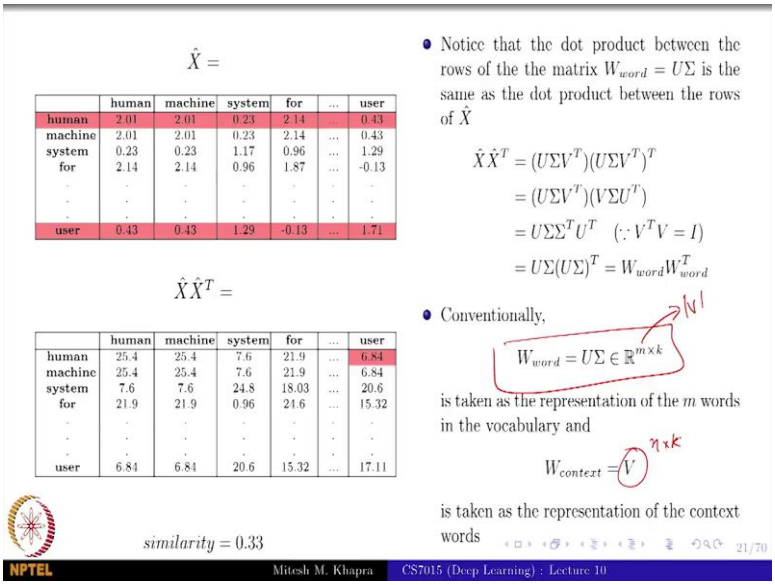


What does this mean? I want an answer right, this is that aha moment should be there or otherwise there is no point. What is how many rows are there in W? The same as the number of words in our vocabulary; what is the dimension of each row? K. So now, W word has low dimensional representations for the words in the vocabulary, but while doing this what have we not sacrificed the cosine similarity. The cosine similarity obtained by this is actually the same as this, do you get that; how many if you see this is very, very important? That if you have not understood, this everything is meaningless.

So, you see how from SVD we got a low rank or a low dimensional representation for the words right, W word is just to be clear k and k is very, very less than V, right? So, now, we have representations for words which are much smaller they are no longer V dimensional remember in practice this k would be of the order 100, 200, 300 and remember your vocabulary was of the order 50 k 1000 k and so on, right?

So, the huge reduction that you have got, and you have still been able to learn meaningful representations which give you better similarity between related words, right ?

(Refer Slide Time: 12:49)



So, conventionally W word which is U sigma and belongs to m cross k. So, I am sorry for messing this up, but I have used m n and V are interchangeably. So, you would understand it from context that m is V. And the other matrix which is V is known as the W context matrix right what is the size of W context n cross k or k cross n, right?

That means it has the representations for all the context words and W word has a representation for all the target words right. So, we had these words on the rows and the context words on the column. So, W word has the representations for the rows and W context has the representation for the corpus, ok?

So, this what we have seen so far, and this is where we learn today; is what a NLP was 6 years back right before the advent of deep learning. If you wanted to use word representations this is what you would do, you would do con construct a co-occurrence matrix try these tricks of PMI PPMI positive negative 0 and all those things those heuristics. Then do a simple SVD retain the most important 100 200 dimensions and treat that as word representations and use it for whatever you want to do.

Now, what needs to be seen is, what happened with deep learning and how have this way of computing word representations changed over the past few years, right? So, that is what we are going to see in the next lecture, right?

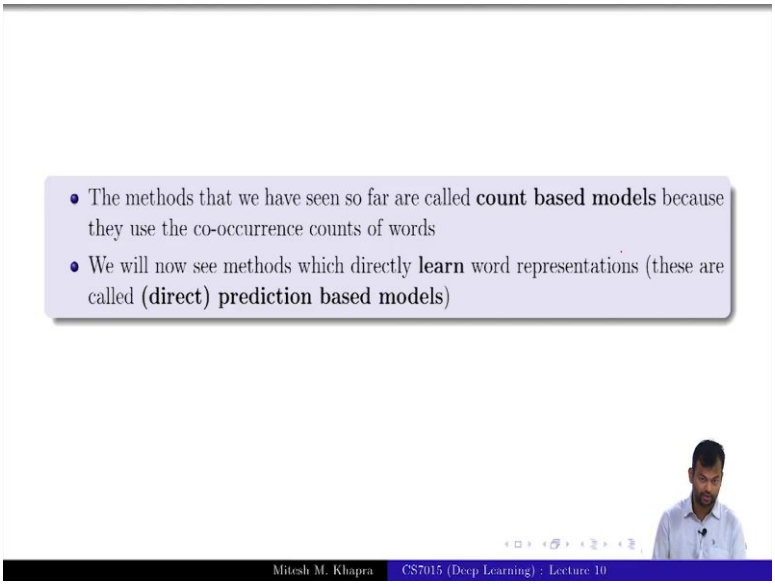
Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Module – 10.4
Lecture – 10
Continuous bag of words model

So, from here on so, none of this that we covered had anything to do with neural networks say, but it was important to understand the context and I will tell you why it was important to go over the traditional way of learning word representations and then we will see how it ties to the modern way or the neural network way of learning representations.

So, we will start with the first neural network based model for learning word representation, which is known as the Continuous bag of words model.

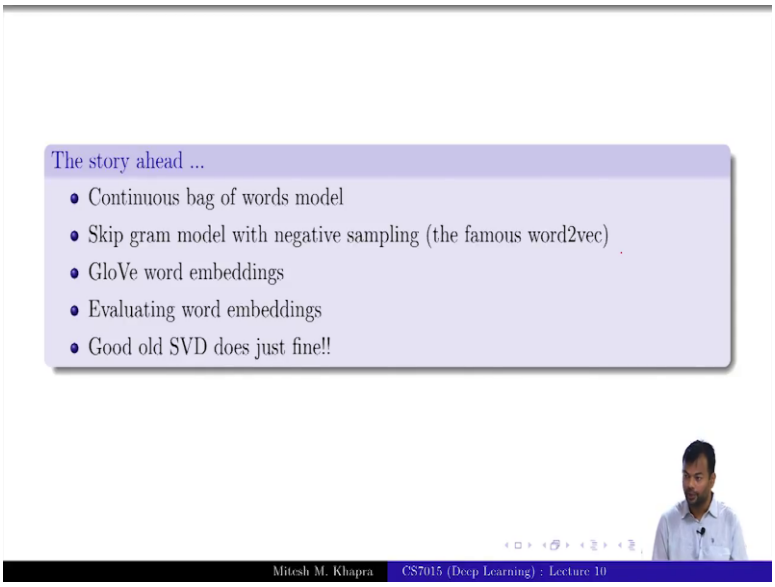
(Refer Slide Time: 00:37)



So, just to set the context the methods that we have seen so far are known as count based models because they rely on these co occurrence counts for learning representations of words, and the methods that we are going to see now are called prediction based models and it will become clear shortly why the term prediction, and how they learn the word representations.

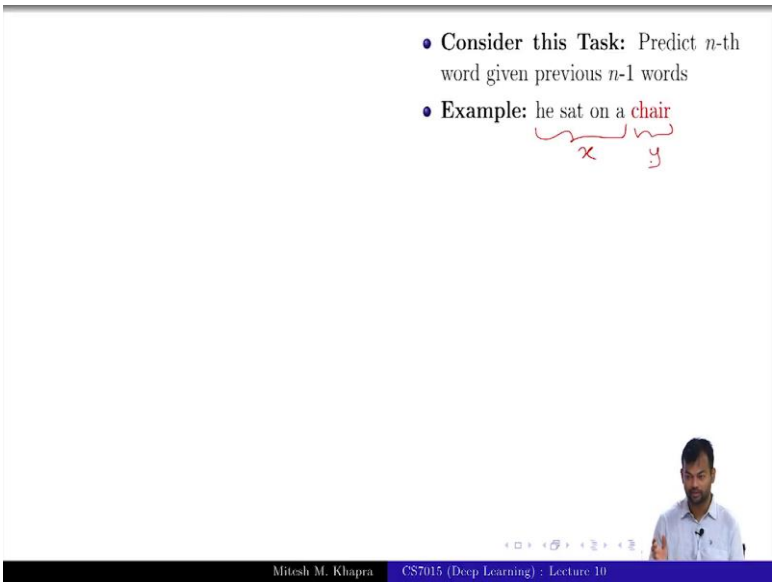
So, in a way in the original thing there was no learning involved of course, you can say that you were trying to learn these eigenvectors and eigenvalues and so on, but it was not in the same way as it be as we have been learning parameters of a neural network and so on right it was not in the same spirit. But now once I do these second type of models, this distinction would become very clear one why is there a learning involved and why they are prediction based models.

(Refer Slide Time: 01:24)



So, the story is we are going to look at continuous bag of words model, then something known as skip gram. So, this is the famous word2Vec model which you guys have already started looking at then we look at GloVe word embeddings which is some kind of a hybrid between the count based models and the prediction based models. And then we see how to evaluate word embeddings and then end with this depressing note that good old SVD is just fine, right? So, all the progress that has happened in the past 5-6 years, you could just use SVD and still go by, but if you do that you will probably not get a job right you have to learn these things.

(Refer Slide Time: 01:59)



So, now let us start with the continuous bag of words model. So, consider this task and just bear me for a few slides that when why this is connected to our problem and all that. So, I am going to consider ka task, we are here to predict the n'th word given the previous n minus 1 words, right? As this is something that you do regularly sometimes even in the class when you are whatsapping or smsing. So, this is what you do right you start typing he sat on a and you get this prompt that the next word should be chair or something like that, right?

Now, you can think of this as a classification problem; tell me why you can think of this as a classification problem, can you tell me what is. So, remember that we have always thought of this that there is always a y there is always an x, and then we are trying to learn this relation from x to y. So, given this example can you tell me what is x here and what is y here?

Student: (Refer Time: 02:51).

Everyone is clear about that, right? So, this is x and this is y now; I made a statement that I could think of this as a classification problem, right? So, the minute I say classification what is the y that comes to your mind or dash hot by.

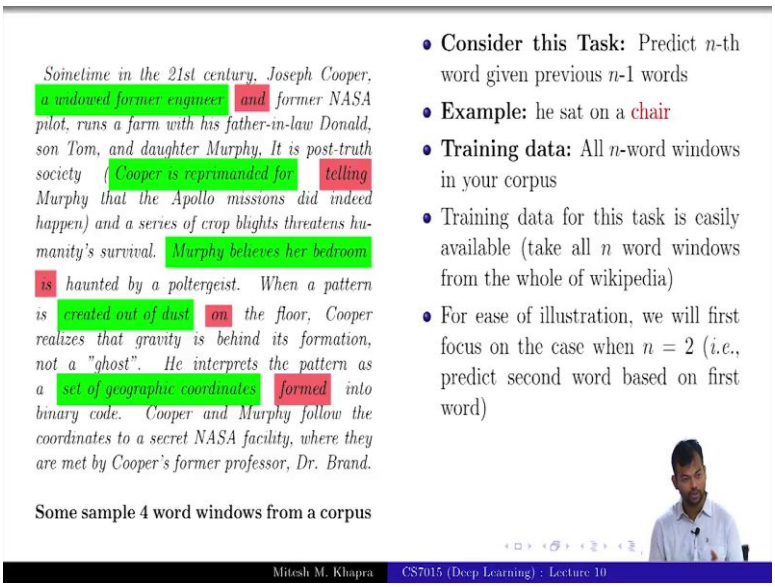
Student: (Refer Time: 03:07).

Y not I, anything else would have been inappropriate, but. So, one hot y is what you would expect there right and now what is the size of this one hot vector?

Student: Size of (Refer time: 03:19).

Size of the vocabulary. So, we are trying to predict one of the words in the vocabulary. So, you see why this is a multi class classification problem you see that there are many classes and you want to select one of these class. Now the moment I say classification I give you an x and y, I will start asking me who will give me the training data for this. So, can you think of training data for this any corpus similar to the one that you are creating, right?

(Refer Slide Time: 03:46)



In specific what you will do is, suppose you have framed it as the following problem that you are given for words and you want to predict the fifth word. So, in general I have call it as that you are given n minus 1 words and you want to predict the n'th word the n that I am considering here is 4.

Now, what is going to be the training data for this? If you take any corpus that you have built anything right consider all five word windows from there do not get too engrossed in the story. So, there are four the first four words you can treat as x and the fifth word would be your y. So, you can construct many such x comma y pairs from the raw corpus that you are creating, any 5 word window and you can keep sliding this window right

what I mean by that? This is your first training instance x comma y, this could be your second training instance. So, this could be these overlapping training instances you keep sliding this window and you will get many training instances ok you see that.

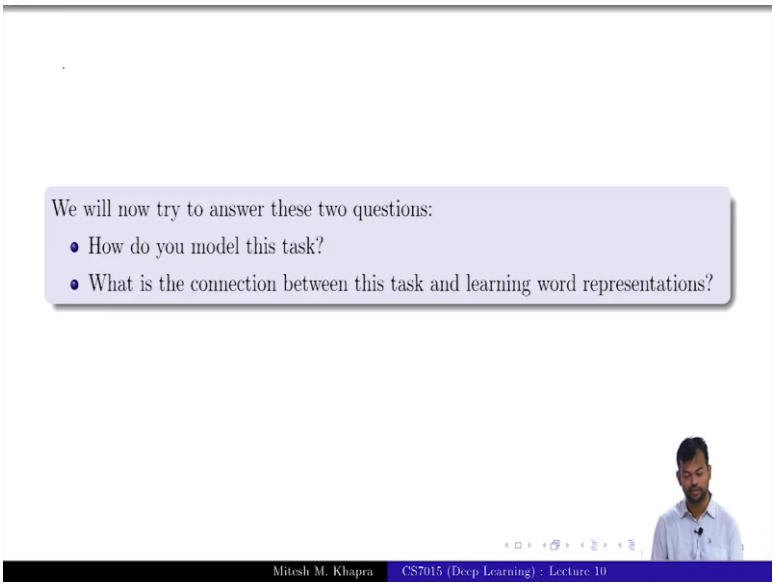
So, this task the advantage is that, given the size of the web and so on at least for popular languages the training data almost comes for free, right? Compare this to MNIST or any other task where you have to actually acquire these labels that this is an apple, this is a banana and so on here you get the training data for free just need to scrape it from the web know. No, window size is something that you will set right whether you want to learn four word windows or what do you mean we do not know the window size no.

So, this again there is a lot of existing literature in the traditional NLP, where various and lot of work has been done to figure out what is the right n. So, in most NLP task right if you want to predict the next word, a three word window is enough actually. If you know the last three words and you can try this as a mental exercise right; if you know three words you do not really need to know the words before that. So, this is the mark of assumption with where this is a trigram dependency in the words, right?

So, this n is not really so difficult, and in the default tool that you guys would try probably they take the value of n is 7 that is an overkill, but that is again it comes from a lot of existing literature in NLP right this is not a this task is not deep learning broad right this task is a simple language modeling task, which has existed for many years right from probably 1950s or 60s or something.

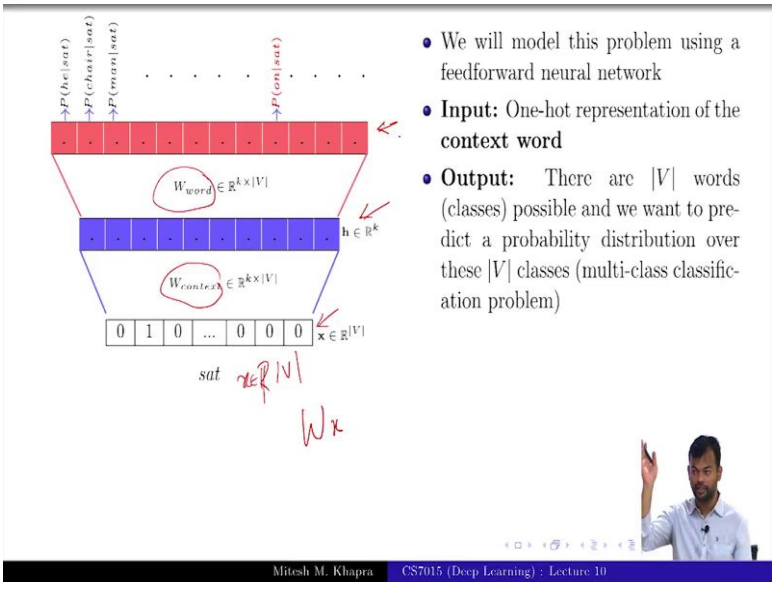
So, this is all n word windows in your corpus as I said training data comes for free and for ease of illustration we will now focus on the case when n is equal to 2; that means, I am given one word and I want to predict the next word.

(Refer Slide Time: 06:18)



And we will see how to model this using a neural network. So, these are the 2 questions which I need to tell you how to model this task and what is the connection between this task and our original task of learning word representations these are the 2 things that I am going to answer.

(Refer Slide Time: 06:28)



So, we will model this problem using a feed forward neural network, what is the input? One word,ok so say the word is sat I am going to represent it using a one hot vector ok and what is the output? I want to predict a distribution over all the words in the

vocabulary and I want to predict I want to pick the word which has the maximum probability that is how you did. So, for example, in the case when you had this classification problem of banana, apple, orange, mango, you predicted a distribution over these 4 classes and then picked the one which had the highest probability exact same idea here it just that instead of 4 classes now you have V classes and your V is very large ok. It is trying to learn a distribution over there.

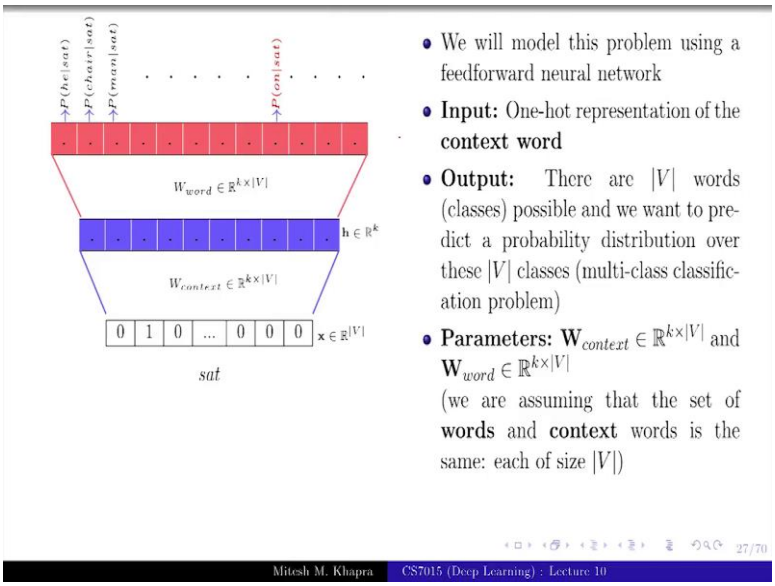
And you know that in this case or the example that you are considering on is the actual next word. So, you type sat and the next word is on, and probably leading to sat on the chair or something like that. So, this is what you would want to maximize, ok. I have given you the input, I have given you the output give me a neural network to model this, there are lot of hints in the diagram itself right you see some space between the input and the output.

So, what will you put in the middle layer we will put a middle layer there, right? Is this an ok way of modeling this task. I have an input I want to predict an output, so, I just use a regular feed forward neural network and let us analyze these parameters a bit more carefully, right? So, I am something known as W context, I have something known as W word I am already using some notations from the SVD lecture. There at the end we ended with W word and W context right it is not clear why I am using the same notations, but it will become clear in some time, but let us look at their dimensions.

So, we have this one hot vector I have a parameter W context which I am going to learn right and its size is k cross V. So, what does that mean? This matrix is going to multiply by the vector and give me a k dimensional output right is that clear. So, I have this is of size V because always keep surprising me I do not know why you cannot do this R this is a V dimensional vector, you multiply it by a k cross V vector. So, you do W into x. So, you will get a k dimensional vector. So, this is k dimensional you have a k dimensional hidden representation.

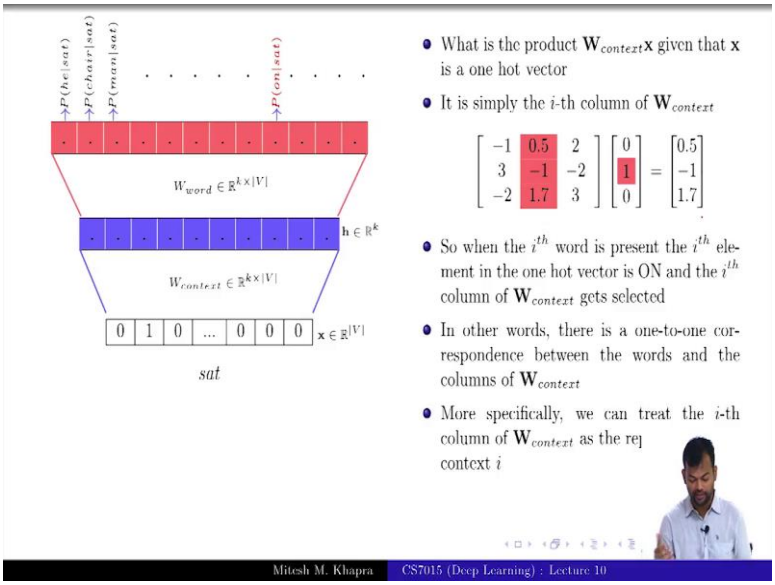
And from there now having captured this hidden representation, you are trying to predict which is the next possible class. This is the same as any other thing right if you had done the image classification or the MNIST digit classification, you had this 784 dimensional input vector, you pass it through a hidden layer and then you predicted one of the 10 classes, there is nothing magic here it is the same thing that you have done seen before.

(Refer Slide Time: 09:17)



How many if you get this and what are the parameters? W context and W word, ok. And we are going to focus on these parameters and understand what they actually mean.

(Refer Slide Time: 09:26)



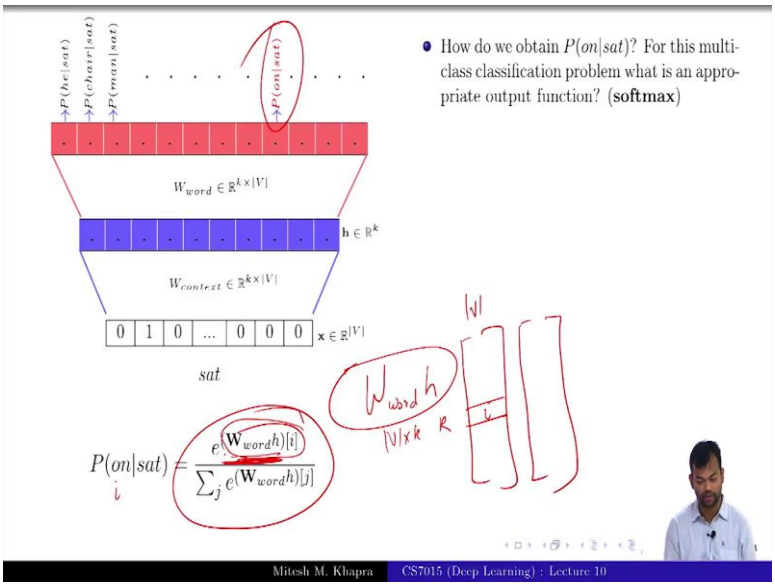
So, what is the product W context into x given that x is a one hot vector. So, I will tell you this suppose the i'th entry is hot here, how many if you say it is the i'th column of W context. So, it is simply the i'th column of W context why? Because you have this W context matrix you take a one hot vector which has the second entry as hot, if you do this

multiplication you basically get the second column of W and you can just see it everyone gets this now how many if you get this now?

So, if you have a one hot vector if its i'th entry is on u multiply it by a matrix, you will get the i'th column of the matrix, ok. So, if the i'th word is present in the input then the i'th element of the one hot vector is on and the i'th column of W context can be would be selected. So, then can what can you tell me about the i'th column of W context? You see there is this one to one correspondence between words in your vocabulary and columns of the W context matrix; how many columns has W context have? V columns how many words are there in your vocabulary? V. Any one word is on only one column will get selected and that is a unique column it is not going to change, right? So, there is a one to one mapping between the columns of W context and the words in your vocabulary; that means, the columns of W context are the are the vector representations.

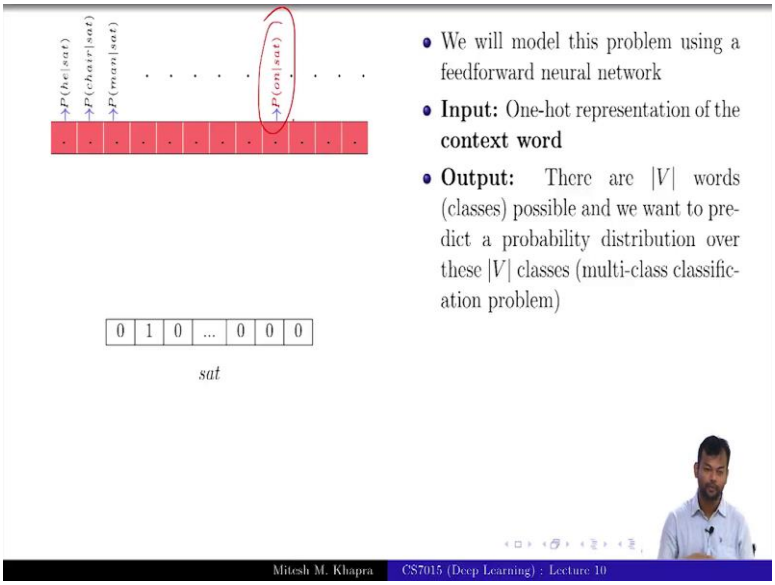
Do you know these vector representations? No these are parameters of your network. So, they will they will be learned how we will see, ok. So, you see the intuition for W context setting it up this way. So, now, I have set up the problem in a way that by parameter matrix directly gives me the word representations, ok. But any kind of learning has to be driven by some objective. So, what is that objective it is already clear to a lot of you, but we will just do that in a bit more detail. So, this is exactly what I have just said.

(Refer Slide Time: 11:18)



Now, how do you obtain P on given sat no no. So, for a given training instance so, when you, so you could so, I will. So, for a given training instance you said that your corpus has been divided into those training windows, right? So, it is possible that engineer sometimes the word does not and is not the next word, but for this training instance what is it. So, that is what you have to predict right is that fine.

(Refer Slide Time: 11:48)



And at test time so you are saying that what you are saying is more practical that when I have typed sat in the whatsapp message, I do not want on as the always the answer. So, we get these 5 options right 3 to 5 options. So, what could be that you have this probability distribution pick the top 5 from there and show it as options so, is that fine? So, we are done with this now how do you compute P on given sat, what is the actual operation happening there, what is the appropriate output function? This is a multi class classification problem Softmax.

This is what Softmax looks like. So, the property if suppose on is the i th word in your vocabulary, then I am saying that the probability of on given sat is going to be this quantity; how many of you agree with that? I mean those who agree is fine I am asking why the others do not agree what is not clear about this? I do not know how to explain this I mean it is just so plain obvious, what is the Softmax function? First of all you will do this aggregation so, you will do this W word into h that is fine, right? So, for you will compute this vector consisting of W word into h fine what is the dimension of that, what

is the dimension of that mod this is k dimensional this is V cross k or k cross V depending on how you multiply it. So, what is the output going to be V. So, you have V entries.

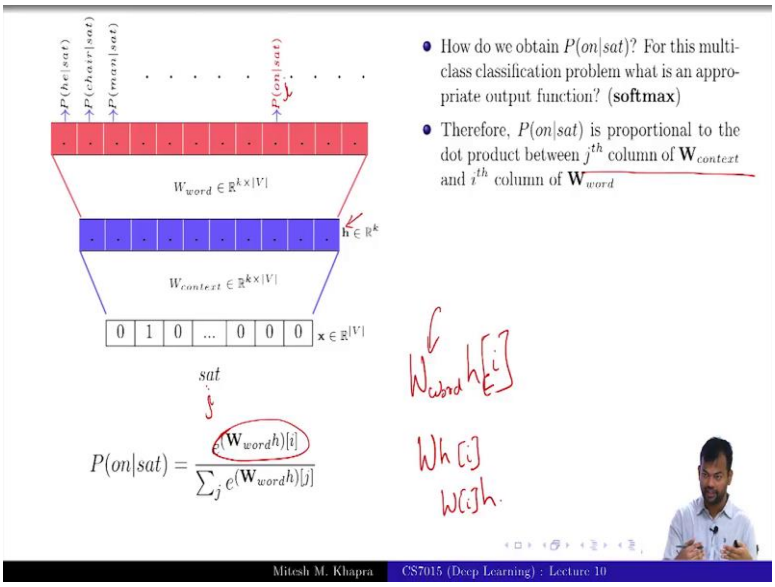
These are dash entries the options are normalized unnormalized, unnormalized now what does Softmax do?

Student: Normalization.

Normalization that is exactly what this formula is doing, right? You want for the i th word you see what was the end this product right this gave you a V dimensional vector, you look at the i'th entry there right that is what you are doing here raise it to an exponent and divided by the summation of all these entries. Come on guys this is highly disappointing I cannot teach Softmax, I had in the tenth lecture eleventh lecture of the course right what is wrong? How many if you get this now just have to ask of it tangle you.

So, you see this right this is what is happening here. So, you get this V dimensional vector and you just con converting into a probability distribution using the Softmax function. So, now, this value how did he compute this value actually? You computed this product which is W word into h and then you took the i'th entry of that and then this some transformation on that, the Softmax transformation you see that.

(Refer Slide Time: 14:21)

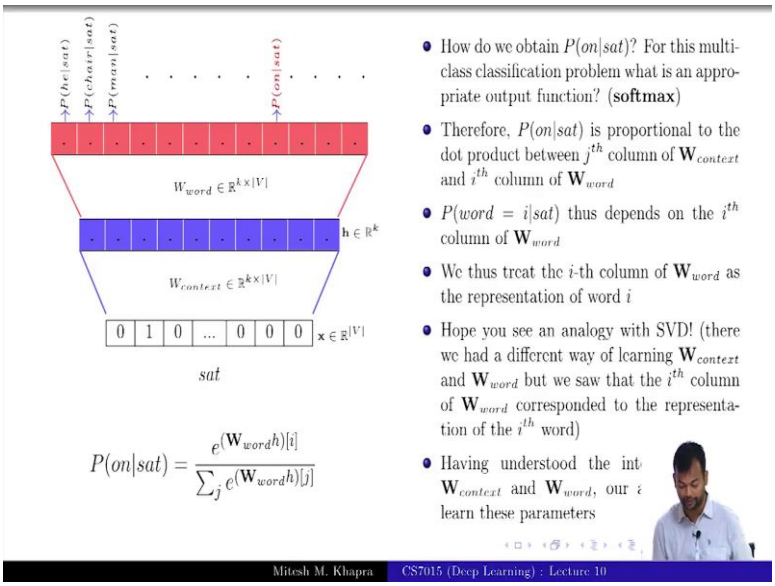


So, now I can say that P on given sat is actually proportional to the dot product between the j'th column of W context and i'th column of W word why am I saying that?

So, remember that this was the i'th word in your vocabulary and on was the j'th word in your vocabulary. So, can you explain the meaning of this sentence to me? First let us look at the first part what is h? It is a j'th column of W context oh sorry this should be i this should be j. So, this you already saw that h is the jet column of W context because I am multiplying a one hot vector with the matrix is that fine and what is the i'th column of W word? So, why what is this product actually equal to if I say W word into h W word into h that is a vector, and then I am taking the i'th entry of that. So, I am saying that is the same as taking the i'th column of W word and multiplying it by h; how many if you get this is basically in your algebra, right?

Now, these 4 different ways of multiplying matrices I am just using one of those, right? So, if I multiply a matrix with a vector and then take the i'th entry of that, that is the same as multiplying the i'th column of the matrix with the vector, ok. Just go back and verify this just take my word for it for now. So, now, what is happening is that it is proportional to the product between the j'th column of W context and the i'th column of W word is that clear now, everyone gets this.

(Refer Slide Time: 15:56)



So, P word equal to I given sat does depends on the i'th column of W word, ok.

So, now what can you say. So, earlier we saw that the i'th column of W context corresponds to a particular word now what can you say about the i'th column of W word? It also corresponds to a particular word. So, now, why these 2 correspondences I already had a correspondence between W context and every word in my vocabulary, now I am saying that there is also correspondence between W word and every word in my vocabulary; how many of you first of all are comfortable with the sentence? That, every column of W word has a correspondence with some word in the vocabulary.

The second sentences every column of W context has a correspondence with some word in the vocabulary do you all of you agree with both these statements? Okay that is what we have try to prove so far. So, now, for every word; that means, I have 2 columns waiting for it, how do I deal with this situation, have you ever dealt with it before? The same thing happened in SVD also, right? SVD also gave you this u sigma which was W word and then V which was W context. So, you can always learn 2 different representations for the words one is when the word appears as a context word and the other is when the word appears as the target would you get that, you see why we have 2 different representations fine.

And as I said hope you see the analogy with SVD right you already saw there that there were these two representation. Now given all this set up and please do not disappoint me can you learn these parameters with some tweaks to the code that you have written for MNIST, can you use the same code to learn these parameters? How many if you say yes. So, what is the tweaks, what are the tweaks? The input changes instead of the image input you have this V dimensional input, what else changes?

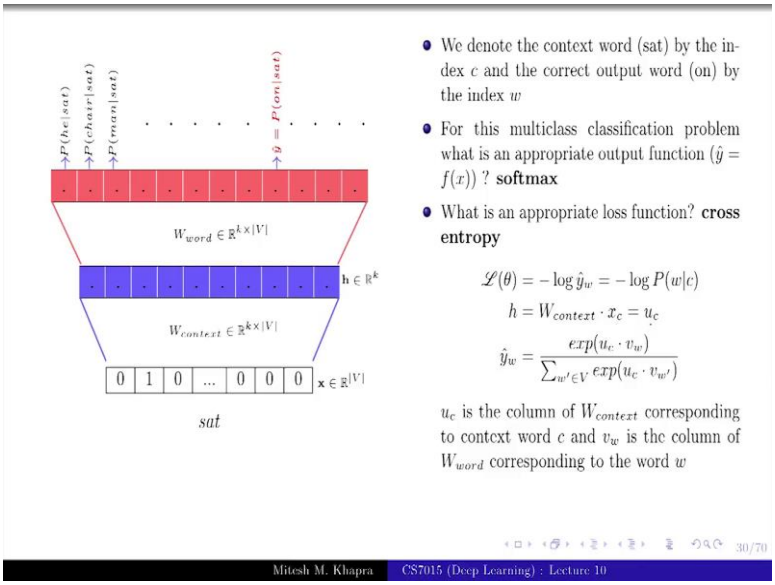
Student: Output.

The output changes instead of a 10 dimensional output you have a V dimensional output all of you are absolutely clear about this and what is the training algorithm?

Student: (Refer Time: 17:59).

Back propagation what is the loss function? Cross entropy good, fine.

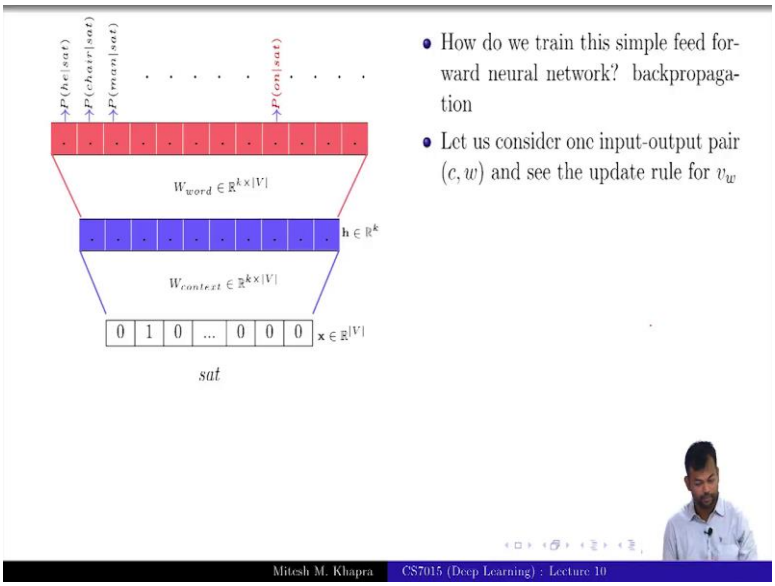
(Refer Slide Time: 18:05)



So, for some I will do some more stuff on this because there is some in interesting interpretations of the gradient descent update rule here. So, I will refer to the word sat by the index c, and the word on by the index w. Ok. And you already saw that the appropriate loss output function is Softmax, the appropriate loss function is cross entropy. So, let me just look at this, right? So, w was the index of the output word. So, my cross entropy formula would just boil down to this everyone is fine with this? Ok. I will just try to maximize the w th entry in my y hat, how many of you are fine with this? Okay and that is nothing, but the probability of the word given the context.

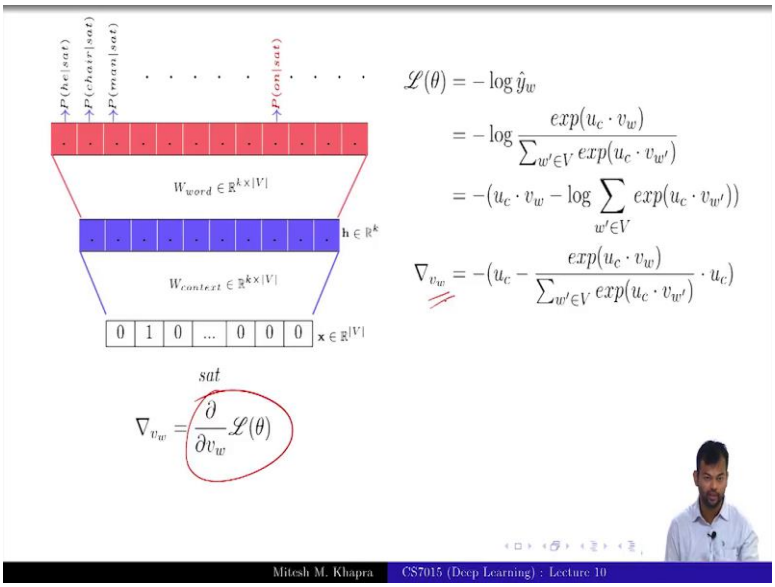
Now, remember that h is equal to W context into x c, I am going to call that as u c. So, u c is the dash of the word sat title of the lecture? It is a vectorial representation of the word sat everyone is fine with that? Because that is exactly what this product is going to do and now my y hat w is equal to this because I already said it is the product of the c'th column of W context and the w'th column of W word.

(Refer Slide Time: 19:32)



So, now I have a formula for y hat w what is the training algorithm that you will use? Gradient descent with back propagation. Now let us consider one such input output pair and see the update rule for V w.

(Refer Slide Time: 19:44)

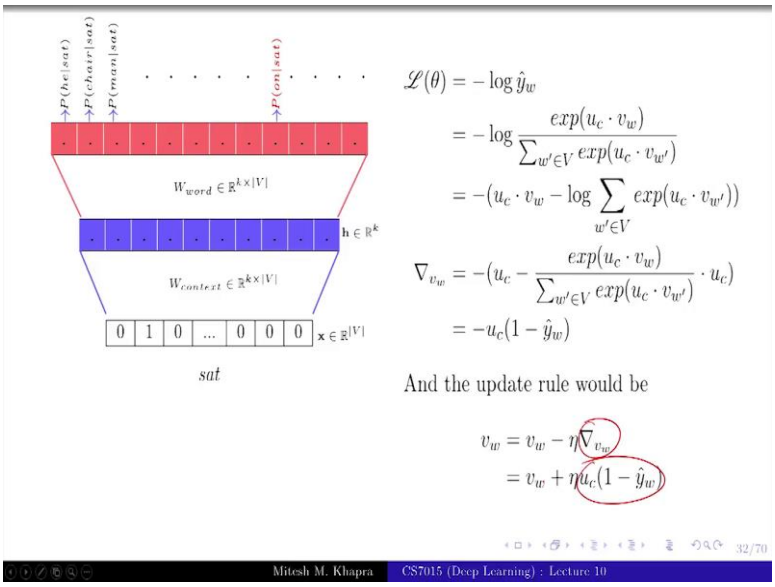


So, my loss function is this, this is actually this quantity, ok. Now I can just rewrite it as this I have just expanded the log. So, the log of a by b is log a minus b ok now I want this quantity. Because this is the parameter of the network right v w is one of the columns of W context or is it W word w word; v w is one of the columns of W word and I want to

learn I want to learn it what are the what are these column entries so; that means, I am interested in this particular gradient.

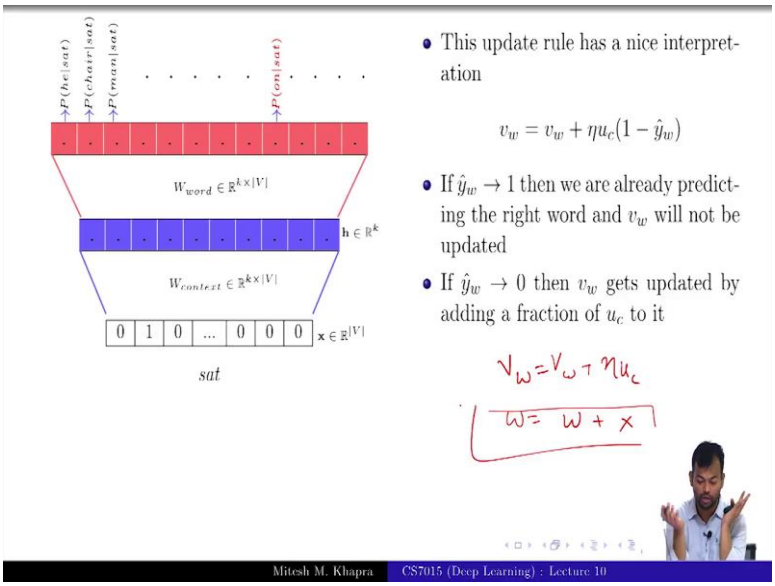
So, I will start taking this. So, what is it going to be? So, only u c will remain here, of all these summation terms only one of them would remain and then you can derive de derivative, right? So, this is what it is going to look like what is this quantity? The Softmax function. So, this is what I get.

(Refer Slide Time: 20:42)



So, now my gradient update rule is going to look like; everyone is fine with this I have derived this formula and I have just substituted that here, and this negative and this negative, ok. So, now let us look at this update rule.

(Refer Slide Time: 20:57)

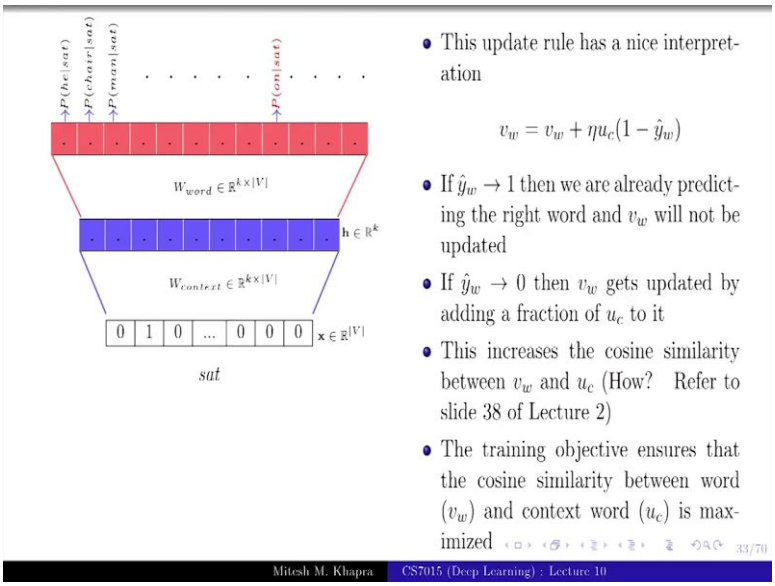


So, this update rule has a very nice interpretation which allows us to understand, what does the continuous bag of words model actually learn. Now suppose y hat w tends to 1 what would that mean? Your prediction is very correct right you are almost predicting it has probability as 1ok what would happen to the update in that case? There will be no updated if it is one there will be no update if it is close to 1 there is going to be very minimalistic update; that means, you have already learned the v w well enough, ok.

On the other hand if I am very bad, if y hat w is close to 0 what would happen? Just tell me the case when y hat w is actually 0 what is the update rule? Have you seen something similar ever before? Have you seen something similar before where did you see this update rule? Perceptron what happened when you did this? W and x came closer to each other the angle between them actually decreased. So, the same thing is happening here.

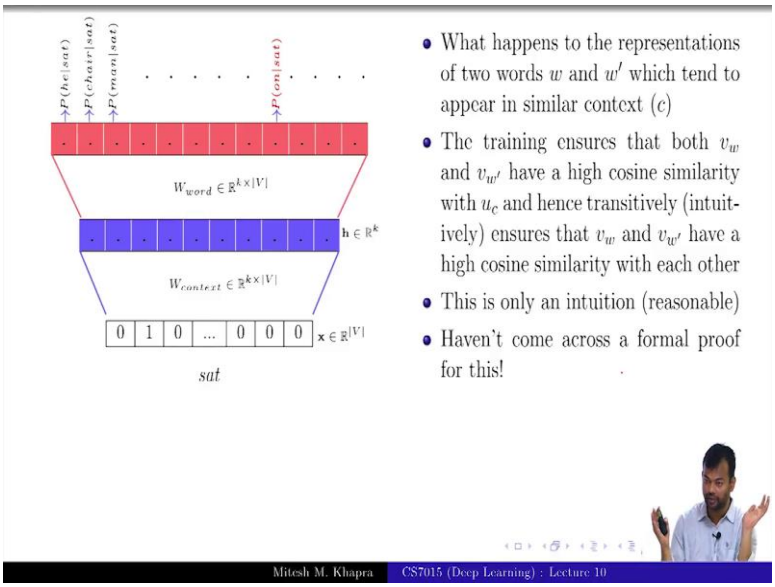
So, what you are trying to do is, you are trying to make your word representation closer to the context representation is that clear how many if you get this? It straight away follows from the update rule right because you are adding a fraction of your context vector to your word vector and we know that when we add 2 vectors they come close to each other the cosine between them decreases, that is what we proved in the word to it lecture in the perceptron lecture, right?

(Refer Slide Time: 22:30)



So, you can go back and refer to this slide on lecture 2. Now, so, the training objective is essentially ensuring that the cosine similarity between v w and context word is maximized; between the word and the context word is maximized.

(Refer Slide Time: 22:47)



Now, what is the result of this? Now I want you to think go back with a starting example, where we said that we want to learn representations such that cat and dog are close to each other, but cat and truck are not close to each other.

I want you to think whatever you see on this slide, the conclusions that you drew from this slide, how do they help you to relate back to that initial goal, ok. So, now, let us let me give you the intuition, right? So, what happens to the representations of two words w and w prime which tend to appear in the same context c? So, say dog eats cat eats, right? So, dog and cat are 2 words, which appear with the same context eats. So, what will happen to the representation of dog? It will come close to eats; what will happen to the representation of cat? Come close to eats. Not only that dog will also go close to pet animal sleeps right and so on and cat will also go close to these. So, transitively what will happen? Dog is going close to a certain point or certain sets of points; cat is also coming close to the same set of points. So, transitively dog and cat will come close to each other you get this intuition.

Anyone sees a problem with this? No. So, known objective and I said that dog comes close to eats is that what he wanted I mean, why should dog be close to eats; that means, if I find the nearest neighbors of dog, I will get words like eats, sleeps, barks and so on is that what I wanted? So, that is exactly what is happening? And based on that I convinced you that dog and cat will come close to each other, but there is a subtle gap here I want you to close that gap how many matrices do we have? two that is enough hint; we are going to either take columns of this matrix as the representations or the columns of this matrix as the representation not mixed.

So, now can you tell me? So, dog here will come close to eats, sleeps, barks, here word will come close to context word right? Cat here will come close to eat, sleeps, and so on, right? So, transitively dog and cat will come close to here and this is the representation that you care about not representations across these two matrices ah. So, what I said is that the training rule ensures that the words representation comes close to the context words representation ok that is what we saw with the training update rule.

So; that means, dog will come close to any kind of context word that it appears with. So, dog I would expect it to appear with context words like eats, pet animals, dog, barks drinks and so on, right? So, dog is coming closer to these words, and I expect cat also to come up here with these words and of course, I do not expect truck to appear with these words, right? So, then cat will also come close to these set of words, dog will also come close to these set of words. So, transitively dog and cat will come close to each other

right all of them are coming close to each other, ok which is fine which was my original goal.

But my original goal was not that dog and eats should come close to each other, because eats and dog are neither synonyms when they do not have any semantics I mean they have a semantic relation, but that is not what I wanted. I wanted similar words to come close to each other, but now I have the side effect that dog is coming close to eats, but that is bad was how can I live with that.

So, the I mean the key thing that you should notice is that, you have one matrix of words; the other matrix is of context words. So, the representation of dog in the word matrix is coming close to the representation of eats, sleeps etc in the context representation on the context matrix. The representation of cat is also coming close to these words in the context representation and transitively because of this dog and cat in the word matrix are coming close to each other and this is the matrix that we care about.

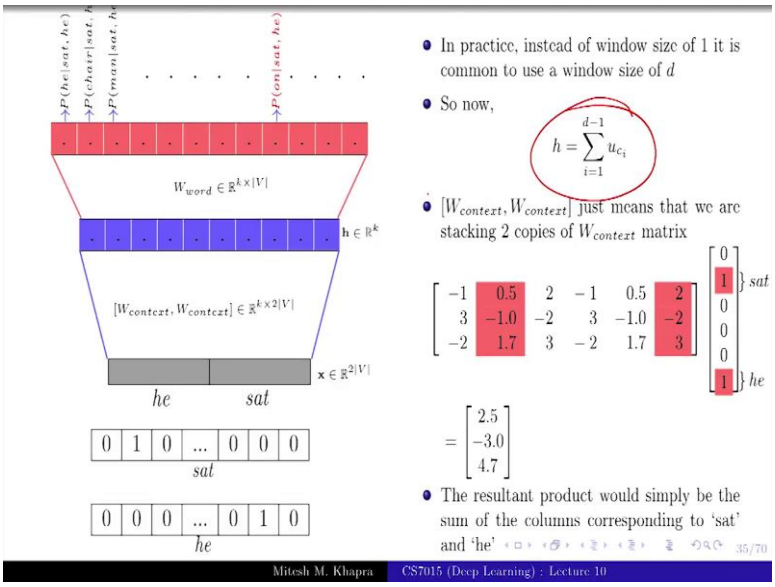
In this matrix dog and eats, dog and sleeps are not close to each other right is that fine everyone gets this now, ok. So, this is only an intuition and this becomes very tricky when I will blow this up; what do I mean by blow this up? Right now what am I trying to do what is the size of n? two right I am taking one word and outputting the other word, hence you get all these neat interpretations that you are moving close to that vector and so on. The moment I add more words to n these interpretations become more and more hard right, but this again I mean this is good to understand that this is what happens at least in the best case. So, this is only an intuition which is reasonable in my opinion, I have not come across a formal proof, which says that this is what actually happens and that is one criticism for word2vec. It works very well, but there is no formal proof which tells you why exactly it works.

As opposed to SVD right there we know there is a principle behind it, here that is not very clear right, but it works very well based on this intuition, ok? So, everyone gets the whole set up how we started with a classification problem of predicting the n th word given the n minus 1 words, which had nothing to do with word representations that is a simple language modeling problem which has existed forever. We smartly modeled it or someone smartly modeled it using a neural network such that the, parameters of the neural network end up giving you the word representations. And this network is end to

end trainable using an objective function the training data comes for free, for popular languages you have like tons of training data the entire Wikipedia entire web whatever you can scrape, that is why with more and more training data you can learn even better and better representations. So, for popular languages the representations are really good.

And then we saw an intuitive explanation for why this works because of this movement of things closer to each other and the key thing to notice there are 2 different representation matrices one for the words one for the context. And this is not surprising the same thing happened for SVD also, u sigma was W word and v was W context, right? So, it is all in the same spirit, right?

(Refer Slide Time: 28:38)



Now, in practice instead of window size of 1 it is common to use a window size of d; either d could be 4 or 7 I have I have even say and seen 11 actually, but not beyond that, ok. Now let us see what happens if you have 2 and here itself it should become clear that, now those interpretations are not very neat. So, what I will use suppose I want to take a context of 2 words, then I have he sat and now I want to predict the next word.

So, what is my input now? He and sat, right? So, I will take the one hot representations of he and sat, I will just concatenate them sorry I just concatenate them is that fine and my input now belongs to R raise to 2 V, in general it will belong to R raise to d V ok and now what is the next step? Do you see something funny here? I have just created 2 copies of this ok I am telling you an inefficient way of doing this, later on it will be a

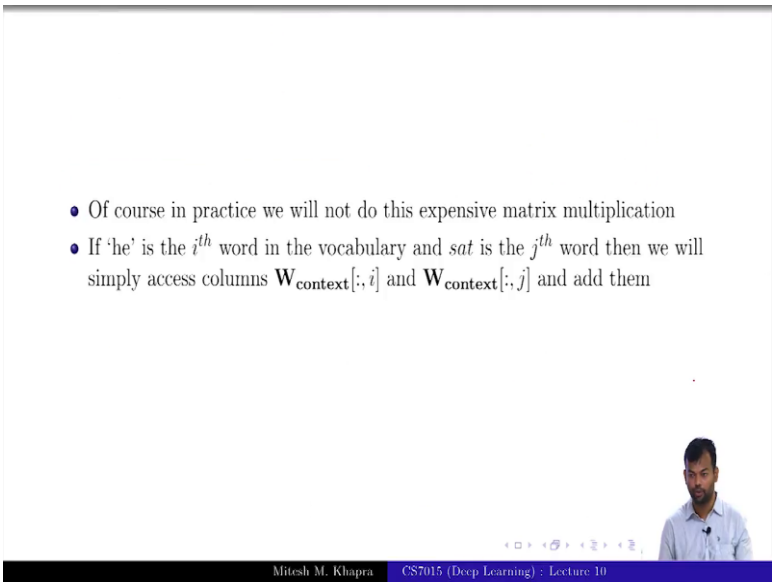
very simple thing to do a very efficient way of doing this right, but first just to get the Math around I will just do inefficient way of doing it.

Why have I staged it twice 2 words, right? So, now, my h is actually going to be the sum of all the columns of w which correspond to my input words is that fine? I have to earlier I had just one word as the input. So, my h was just equal to that column of W; now my h is going to be equal to the sum of all the columns of w corresponding to the words that I have and I will tell you why. So, I have taken w contexts comma W context which is just the W context matrix staged twice back to back. So, this was my W matrix, this is my 2 hot vector because I have 2 inputs now, right? So, my vocabulary size is 3. So, the first one hot vector followed by the next one hot vector and now I am going to repeat W W, now what is the product of this? Is the sum of the 2 columns that you see highlighted right and exactly that is what I have written here.

So, if I do it this way then I can just do this very expensive matrix multiplication and to do a something very trivial which is just taking the sum of 2 columns, right? But at least you get the operation and I will just on this next slide or something I will tell you an obvious simple way of doing that. So, I just get the sum of the 2 columns. So, that is the input to my network. If I had k words as input if I had my window size 4 what would it be? I would have these 4 copies of W context I will have these 4 one hot vectors and it will just give me the sum of those 4 columns ok that is going to be the input ok and the rest of the story remains the same, right? Once you have this h the rest of it from there remains the same and this is the formula for h in general.

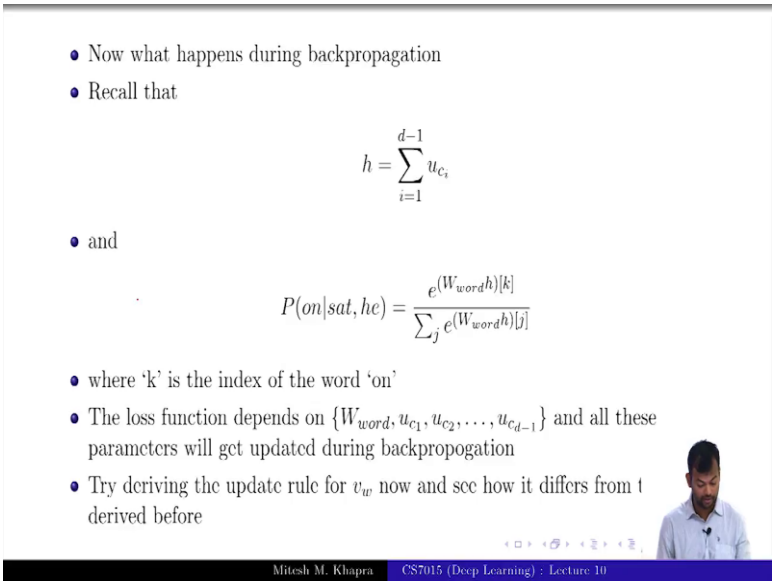
In the special case it was just the i'th column, in the general case is the sum of all the columns that are there in your input.

(Refer Slide Time: 31:35)



Now, in practice of course, this is a very mate expensive matrix multiplication, it is stupid to do it that way; what you will do is you will just slice of those columns from W context right and then just add them up. So, you do not really need to do that stupid mate matrix multiplication because you know that the matrix multiplication is essentially just selecting these columns and adding them. So, just select those columns and add them up. So, you do not do that bad matrix multiplication operation, is that fine?

(Refer Slide Time: 32:01)



Now, what happens during back propagation in this case in the generic case? The ordering does not matter is what you have seen yes it does not matter yeah there is some assumption of the model. So, it is that is why the name of bag of words, you are not relying on the sequence. So, this comes from NLP that if you rely the sequence you call it sequence, if you just going to take the words in the sequence, you just call it a bag of words. Because once you put them in a bag there is no ordering there right that is why the word name bag of words.

So, and again P on given sat is given by this Softmax formula, ok now tell me during back propagation and if you give me a right answer to this I really feel happy that you have understood everything right from the beginning of the course so no pressure. So, which are the parameters which are going to get updated during back propagation, which are the 2 large matrices W word and W context? So obviously, the answer is not W word and W context otherwise I would not have asked you. The answer is some dash of these two some subset of these two, which subset let us start with W context which is the input do we are we going to update the entire W context, did it participate the entire W context participate in the computation, only those columns corresponding to the words. So, only those parameters will get updated.

So, how many columns will get updated? D columns, right? W word till all the columns of W word participate in the computation how many of you say yes, how many if you say no? The others do not care; can you just focus on this circle did all the columns of W word participate in the computation? You see the summation at the bottom, it is over all the columns of W word all of them participated. So, the parameters which will get updated are W word and all the columns of the input words and same back propagation will work again is that fine.

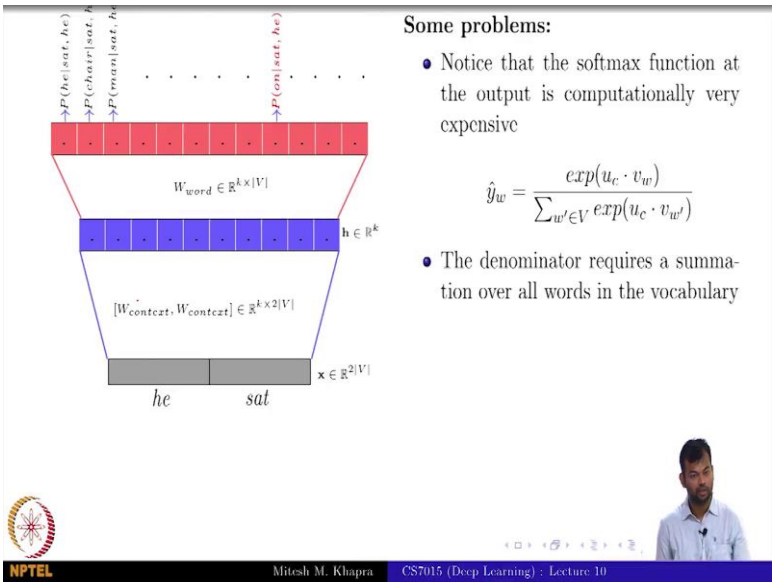
So, remember that and this is I cannot emphasize it enough; whatever I have explained is only for an intuitive explanation, you will never ever do this matrix multiplication right and that is why what you are going to do is you are just going to select those columns add them up and feed them. And the network will take care or rather you will take care that you update those parameters only and you do not update the entire W context matrix because anyways there is no gradients flowing to the other components. So, remember that in the practical implementation of W of word 2 vec do not search for this matrix

multiplication at the input, or if you are writing the code on your own which is highly unlikely do not do it that way.

So, if you whatever code that you look at did not have this complex matrix multiplication typically. They will just pick up the columns and add them and feed them right and I think the tensor flow way of doing is you have this word embeddings matrix and you can slice columns from there and so. So, everyone understands this so far. Now what are these problems with this, why is this not as simple in some sense as the MNIST data set. Again focus on the circle, this Softmax computation is a very expensive operation right you have a v cross k sized matrix somewhere there, and unlike at the input here you will have to do this matrix multiplication, right?

So, we have a v cross k matrix multiplied by a k cross 1 vector, and there is no simplification of this you have to do this multiplication what are the sizes of v that we saw in practice? 50 k, 100 k and if you had Googled 13 million or something, right? So, this is not feasible we cannot do this expensive matrix multiplication.

(Refer Slide Time: 35:56)

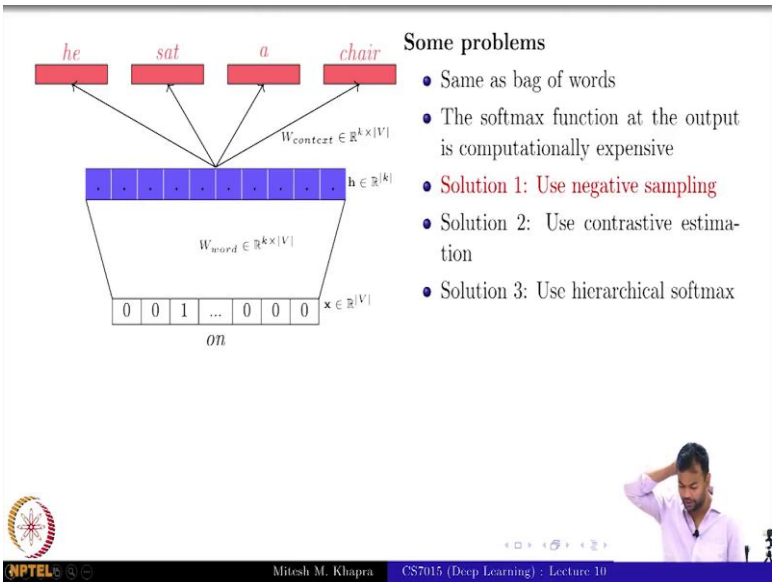


So, although all of this works very fine we need to think of ways to simplify this Softmax computation, where the denominator requires the summation over all the words in the vocabulary. So, you have to do that many matrix multiplications.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

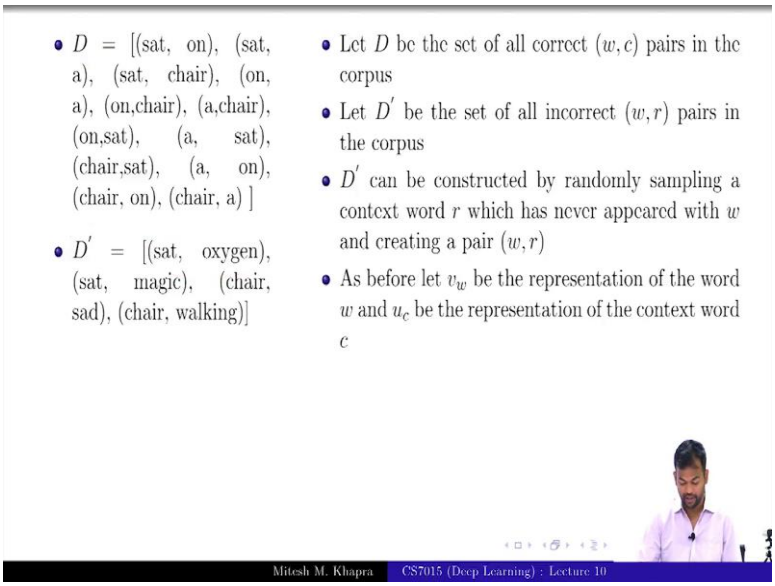
Lecture – 10
Skip-Gram Model (Contd.)

(Refer Slide Time: 00:11)



So, what I will do is I will quickly, go over what we were doing yesterday. And then by the time people come in we can start with the new stuff, right? So, we were looking at. So, that is so this needs to be corrected someone who pointed out yesterday, same as bag of words. It should be same problems as the bag of words model. So, we are trying to fix this problem where we have this large Softmax computation which is very inefficient, and you wanted ways of getting rid of that. So, the first thing that we were looking at is using negative sampling.

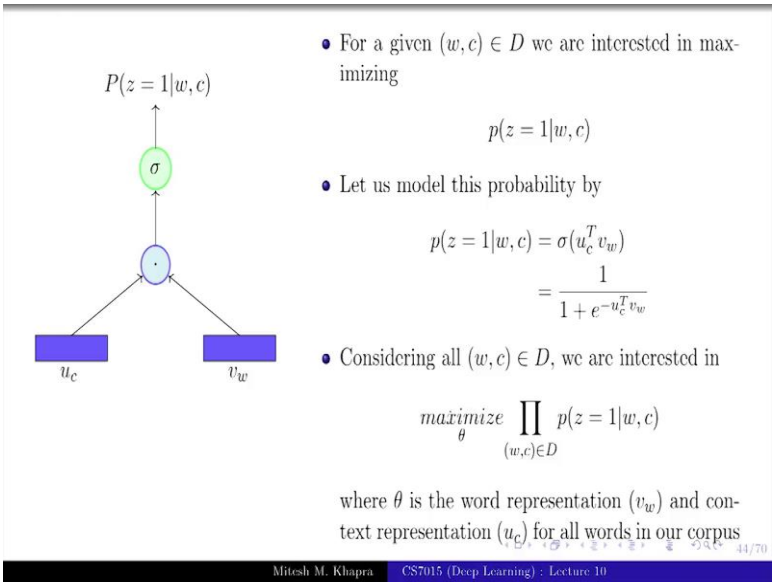
(Refer Slide Time: 00:47)



And here the key idea was to con construct this D and D prime, where D prime was the random corpus and D was a true corpus.

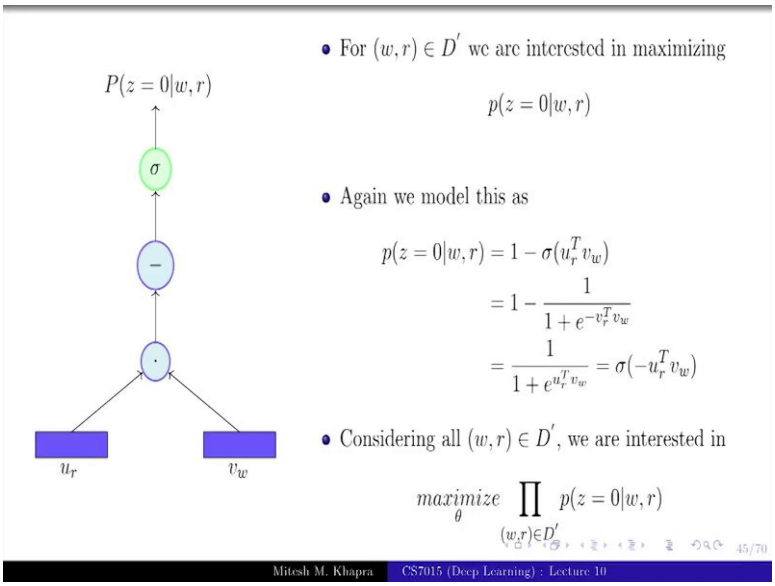
And how do you create this random corpus is something that, was left at the end and which I need to go over today.

(Refer Slide Time: 01:05)



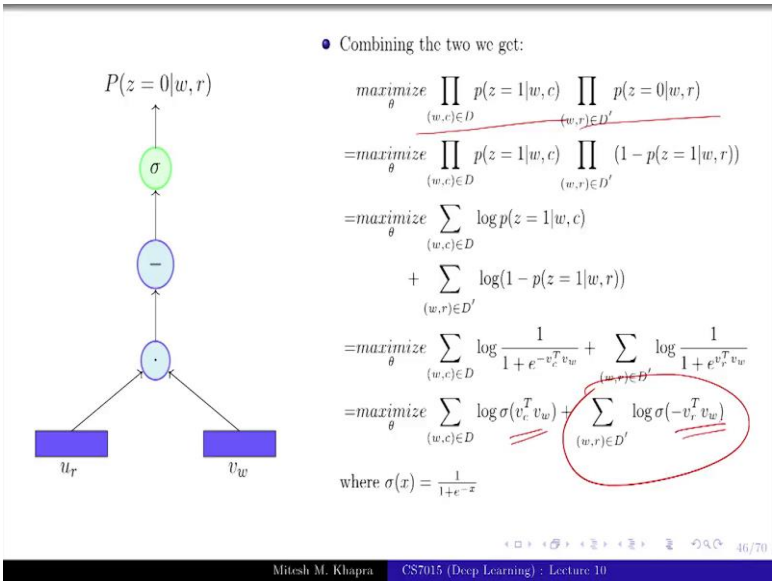
So, I will go over that and then we realize that this actually could be modeled using such a network, where you take the dot product between the word representations.

(Refer Slide Time: 01:23)



And try to maximize this to dot product for all the correct pairs by setting up your loss function accordingly.

(Refer Slide Time: 01:26)



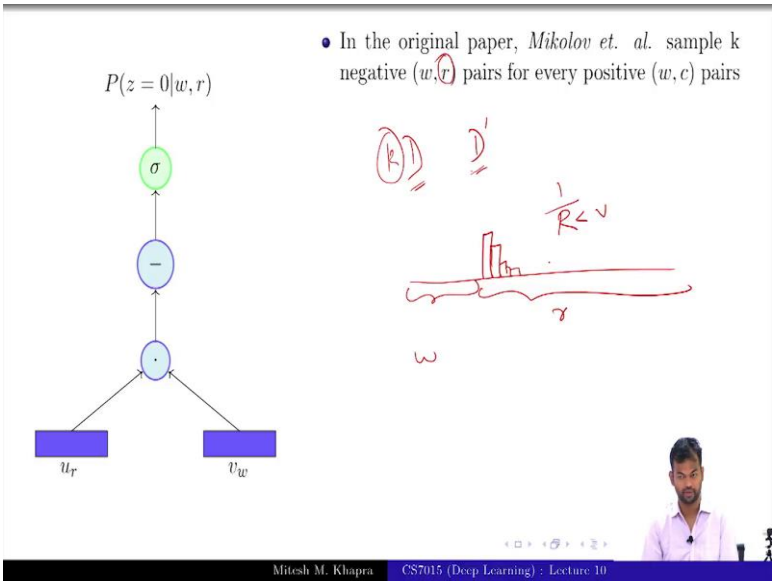
And try to maximize or rather minimize this dot product, minimize this dot product for all the incorrect pairs by again setting the objective function appropriately.

So, we had this objective function where we want to maximize the probability that the pair is correct for the correct pairs and maximize the probability, that the pair is incorrect for the incorrect pairs, and both these probabilities we had modeled using a sigmoid

function, and inside the sigmoid function we had the dot product between the corresponding representations.

So, the net effect is you either maximize the dot product of the correct pairs, or minimize the dot product of the, or rather and in minimize the dot product of the incorrect pairs fine.

(Refer Slide Time: 02:01)



And then so now today the part which was remaining about the comparison between D and D prime; so, what I was saying last time is that D prime is actually k times D; that means, in sample more negative examples than positive examples. So, if you think about it actually the number of negative examples in the language is much, much more than a number of positive examples. Let us say if you have 50 k words in your vocabulary most of them do not appear together, right? So, that number is actually very, very large as compared to the number of words which can occur together.

So, how do you account for this natural imbalance? So, they said that if you keep it same, then we are saying that the size of D prime and D is going to be same; that means, the words which appear together and not to appear together we are keeping those 2 corpora as the same. So, that does not sound reasonable; so, they decided that we will keep it k times, ok? Now this k was a hyper parameter which was tuned based on the data that they had. And can you guess, how they would have tuned it, no what do you

tune your parameters on, what did how did you tune your parameters for the back propagation of the word, no using, what?

Student: validation set.

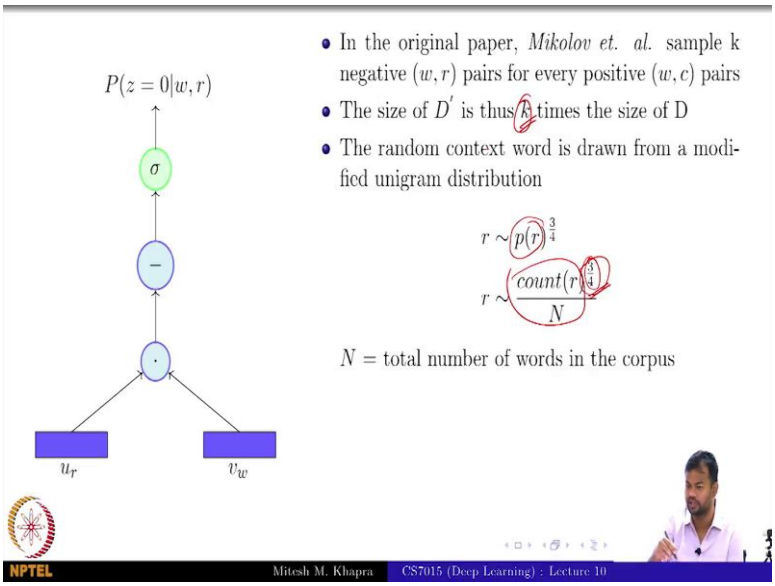
A validation set is it too early in the morning, it fine validation set. So, they might have had some validation set. And if you look at the original word to word code which someone had posted yesterday, which allows you to compute the distance matrix right. So, you could what you could do is you could learn these representations, take a few pairs of words. And take a few pairs of good words right say cat and dog or cat and feline and so on. And also bad words like cat and truck bad combinations rather. And see if the distance between cat and truck is much, higher than the distance between cat and feline or cat and dog.

So, you select that k which gives you the best performance on your validation set and the validation set here would essentially be to find if you get good representations for word pairs that you care about and for word pairs that you do not care about, ok? Now the other thing was how do you create this R? So, you have v words in the vocabulary you are looking at one of those w.

You know that some of those have appeared with w in some context, but there is this large set which has not appeared with w in any context right? So, you are going to draw R from this set and the simplest thing to do would be to just draw the uniform distribution; that means, all words and let us call this suppose there are capital R words here all of these words could be drawn from using the probability 1 by R. Where R is less than v.

Is that fine? That is one way of doing it just randomly pick any word from the remaining words and put it a pair it with w, but you would also want to account for the individual frequencies of those words, right? If the word is actually very frequent, pair it up more with w. If it is not frequent do not pair it up enough. Does that make sense? So, I could actually use the frequencies of each of these words. And sample according to that frequency right? Instead of using a unigram distribution.

(Refer Slide Time: 05:15)



So, they did something similar, but they had this hyper parameter again. So, basically I was sampling using the probability of r, which is equal to count of r divided by the number of times number of all the words in the corpus. That is actually the frequency of r divided by the total number of words in the corpus. So, instead of just taking that they had this wearied factor of 3 by 4; do you we realize that if you take this 3 by 4 you get the best performance.

So, let me just make a few comments on that. So, the original code of or rather the original skip gram or the bag of words model, actually worked very well and it kind of hard a lot of seminal effect or a lot of revolutionary effect on the field of NLP right. So now, everyone started talking about word vectors, and how you can use this meaningful representations of words, as features for various down steep NLP thus right.

So, at the end in NLP what you are doing is you are collecting of a bunch of words a document or a sentence or something and trying to do some processing on that. Now earlier used to construct features out of these sentences using some handcrafted features, but now someone said that there is this automatic way of constructing word features right, which is using this method. So, people really bought onto that idea and a lot of work started happening. And then later on at the end of the course we will see something that what it eventually led to.

But later on when people started analyzing this more carefully right, they realized that the original word2Vec implementation, had a lot of these heuristics or lot of these parameters, which need to be really tuned to the core for it to be able to compete with SVD right. So, that is what we look at the end. So, SVD was already one way of computing word representations ah, which while popular was not so popular it was used for various reasons, but it was not like every NPL application is using SVD representations right, but now it is almost like every NPL application is using word representations.

So, later on we will see that some of these things like 3 by 4 or k the value of k, the value of learning rate and some other hyper parameters. If you really tuned them very, very well it is only then that as this word2Vec algorithm can beat the world representations learned by SVD, or rather the other thing that if you introduce some parameters in SVD and tune them, because remember for SVD there was no tuning right, we just got a solution. We just had the closed form solution which is the Eigen vectors.

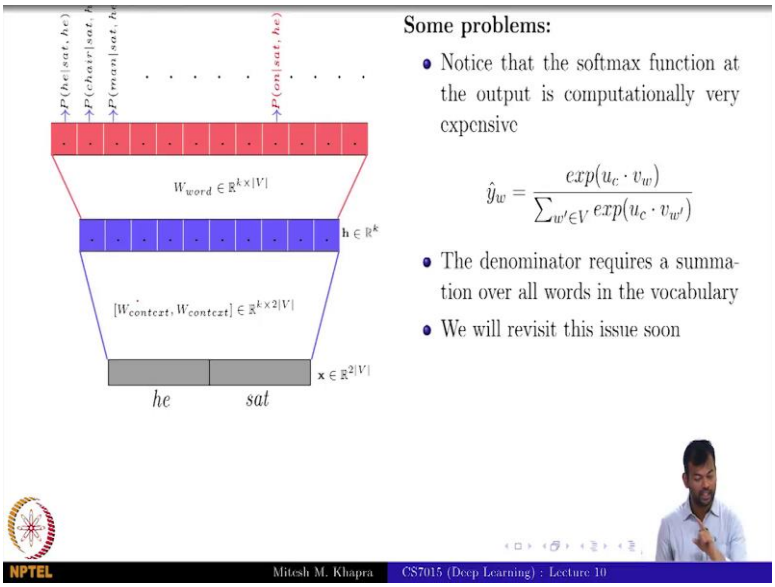
But you could do some things for creating the co-occurrence matrix. If you introduce some factor there which is also looks like this 3 by 4 or something like that, or if you also introduced something which looks like a k. Then you will be able to get the same kind of representations or equally powerful representations from SVD as what you get from word2Vec. So, that is why I am stressing on these hyper parameters there is some significance of those.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Module – 10.5
Lecture – 10
Skip-Gram Model

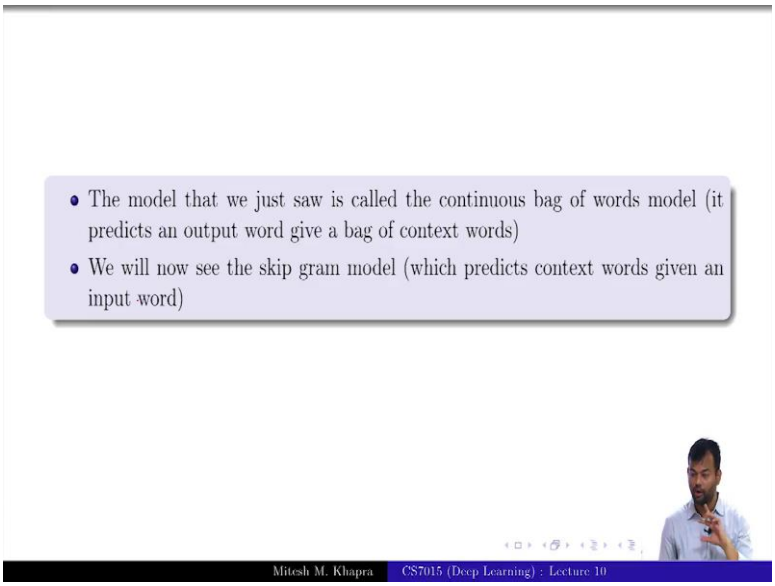
So, with that we will move on to the next model, I am still not telling you how to solve this problem; we will come to that later.

(Refer Slide Time: 00:15)



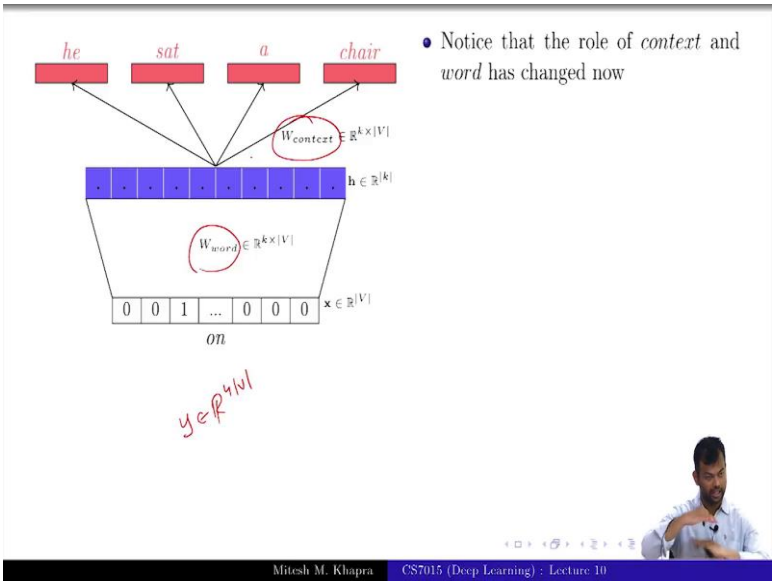
I am just going to the next model which is the Skip gram model. Ok And this is the famous word2vec which you are trying to implement.

(Refer Slide Time: 00:23)



The model that we just saw was known as a continuous bag of words. It predicts the output given the context; skip gram model does the reverse of that.

(Refer Slide Time: 00:34)

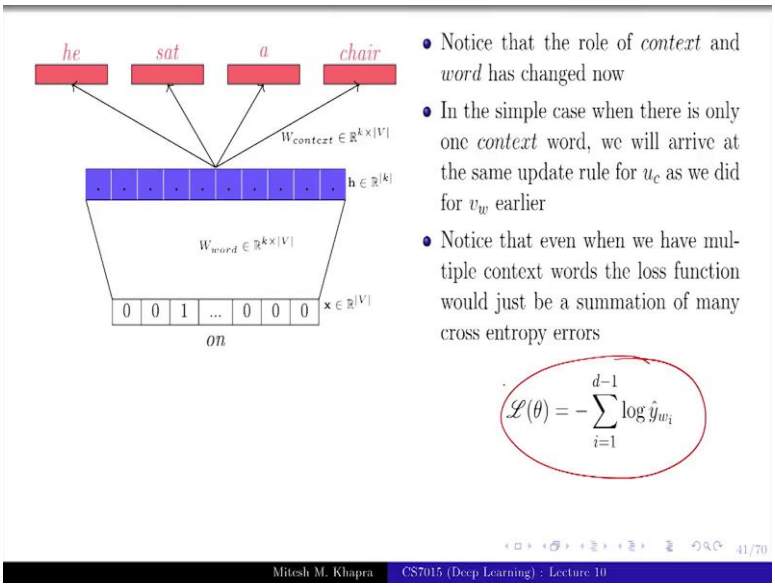


You are given a word; you want to predict all the context words. So now, I am given the word on, I am trying to predict the words which appear on the left and right side of it is that fine. So, how many prediction problems am I solving? How many predictions am I giving you? 4 in this case, right? So, you see that this is a case where your y actually belongs to R 4 right? Of course it is not R 4 it is 4 into V, and because you are predicting

the entire distribution, but what I meant is that you want these 4 different outputs you just do not want one single output.

Apart from that does everything else remain same? You have an input word you compute a hidden representation from that hidden representation you try to predict the outputs. You get a probability distribution, what is your loss function? It is a dash of cross entropies sum of cross entropies, how many cross entropies do you have? 4 in this case and also notice that I have, I hope I have yeah, I have changed W word and W context they are flipped now is that fine. The role of context and the word has changed.

(Refer Slide Time: 01:46)

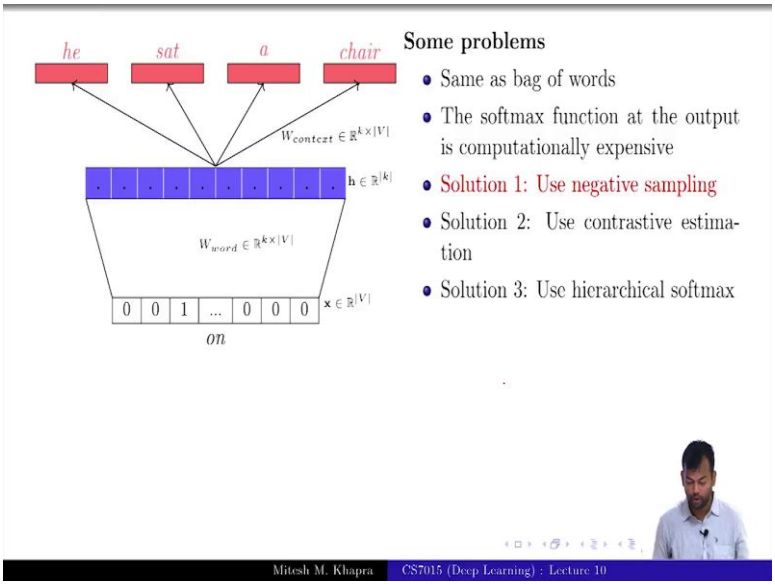


In the simple case when you are trying to take one word as the input, and only predict one word around it. It just becomes the same as the first case that we saw in the continuous bag of words, there is no difference there because there also you take one word and predicts the other word.

So, the entire Math's remains the same how many of you get that, and even when we have multiple context words, our loss function is just going to be the sum of the cross entropies for all those D predictions that I need to make or D minus 1 predictions that I need to make. And then once I see a loss function which is a sum of some things I am not worried, because I know how to deal with each of these components and gradients are additive. So, if you have the gradient of some, it is just the sum of the gradients. So, as I

long as I know how to deal with one of these I can deal with the sum. So, that is why I do not really worry all of you are at that level.

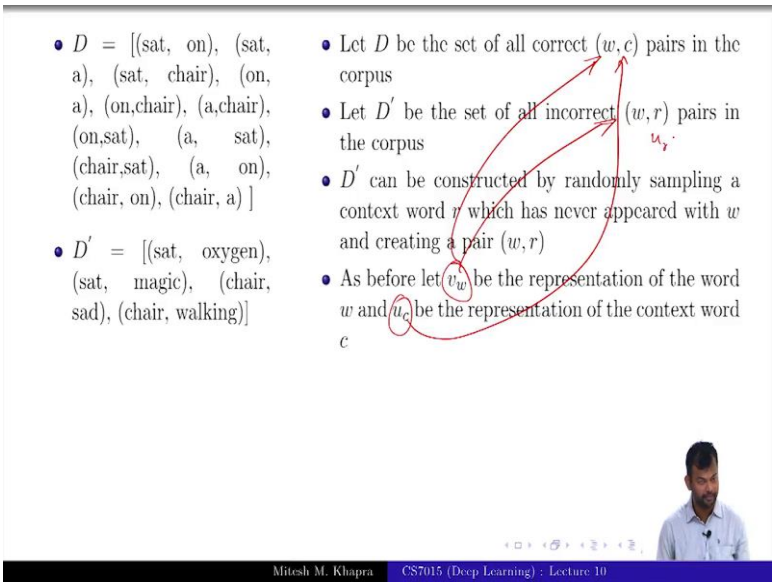
(Refer Slide Time: 02:40)



Where you do not worry with the sum as a loss function, what are the problems with this already written there same as the bag of words right.

Now, we are doing these 4 expensive computations at the end. So, the Softmax computation is expensive, there are 3 different solutions. And there are 3 different ways that we can deal with it one is something known as use negative sampling, the other is to use contrast of estimation and the third is to use a hierarchical Softmax. So, we are going to see all of these and I will shamelessly continue for a few more minutes. So, first we will see use negative sampling because that is very easy.

(Refer Slide Time: 03:13)

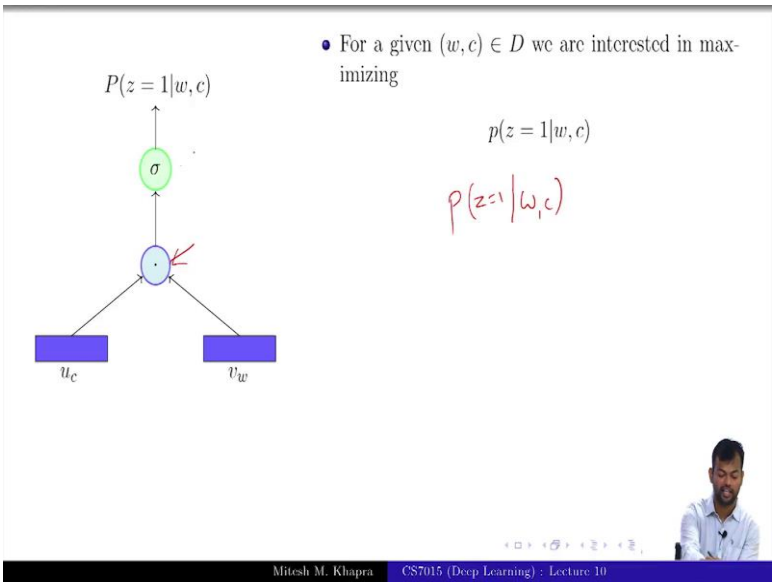


So, let D be the sat set of all correct w comma c pairs in the corpus, what do we mean by that? All words which actually appeared in the word comma context pair. So, you can look at the vector, which I have constructed. So, sat on sat or sat chair you can imagine that all of these appeared in the context of each other. So, this is my correct corpus as from what I got from my data.

Now, let D prime with a the set of all incorrect w comma r pairs in the corpus, and r here stands for random, some how am I going to construct this corpus. So, I take a word I know all the words which appeared with it and I know all these other words, which have not appeared with it. So, I will randomly sample a word from there and put it as r, is that fine. So, I can compute D prime again D prime comes for free. D was always for free now D prime is also; obviously, for free.

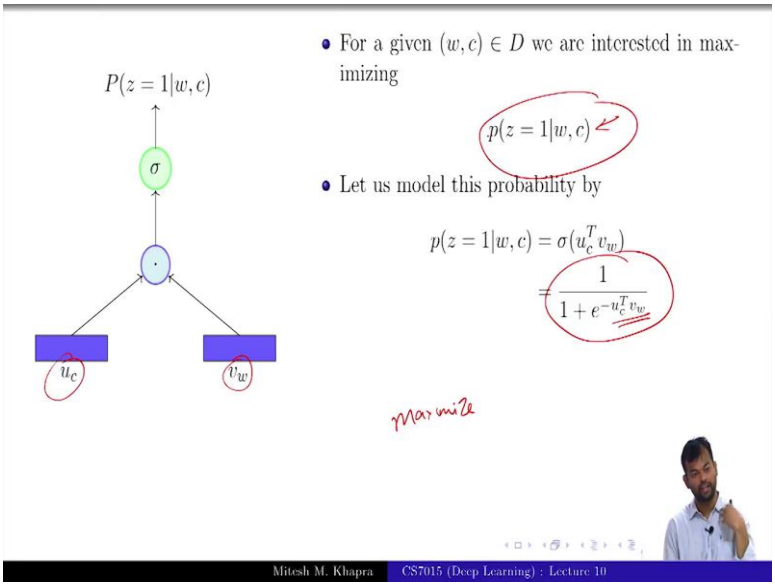
So, I have w comma c and w comma r and I have these corpora D and D prime. And as before let v w be the representation of the word, and u c be the representation of the context word, ok? So, v w will try to these 2 and you see will try to this and u r something else that we will use for this hopefully, is that fine? Okay.

(Refer Slide Time: 04:31)



Now, for a given w comma c which belongs to D, which is the true corpus, what are we interested in maximizing. So, let us think of z is a random variable weather which tells us whether this is a true pair or not. So, given w comma c, I want to maximize that p of z is equal to one, ok? Now, this is what I want to maximize. Now it depends on me how do I model this probability. So, can you guess how am I going to model this? The answer is there in the figure; can you tell me what is the model that I have chosen? Can you tell me what is the formula for z equal to 1 given w comma c that I have chosen? This stands for dot product this stands for the sigmoid function.

(Refer Slide Time: 05:16)

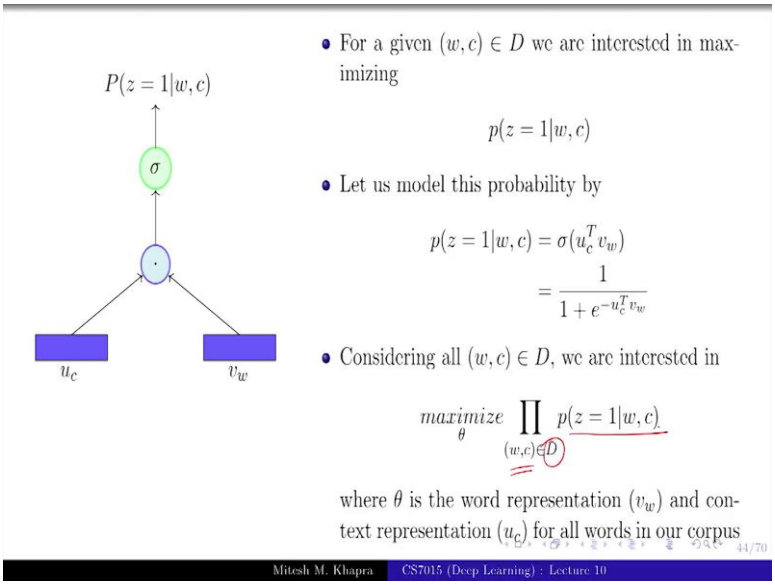


I know this is some uc representation, this is some vw representation note that these representations are not learned yet I need to learn them using the training objective that I set.

But at the beginning they are initialized to some random values. And the way I am going to model probability of z equal to one given wc is that I am going to say that, it is just the sigmoid function of the dot product between them how many of you get this are you comfortable with this. This is the modeling choice which I have made or rather the authors of skip gram, right? Now how am I going to now what do I want to do for all w comma c belonging to D. I want to maximize this probability is that fine, for all the w comma c pairs which belong to my true corpus which is the D corpus; I want this probability to be high, how many such pairs do I have many many right? So, let us call them as I have n such w comma c pairs.

So, can you tell me what my loss function is going to look like maximize this for the first pair and for the second pair and for the third pair all the way up to the end pairs.

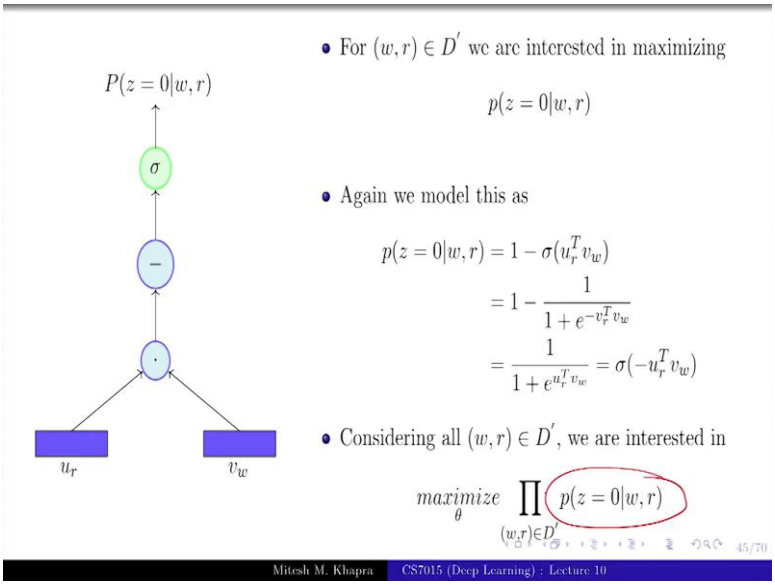
(Refer Slide Time: 06:25)



So, what is it going to look like for every w comma c which belongs to my correct corpus, I want to maximize that probability of z equal to 1 given that, w comma c pair right. And since it is an and I will have this product how many of you are comfortable with this.

So, this as such and this is something that you do regularly you should have done this in machine learning or pattern recognition or somewhere right. That you want to basically maximize the log likelihood of the data, which is saying that you want to maximize the probability of every training instance, which is saying that you want to maximize the and of all these probabilities right be you take the and of all of them is that fine.

(Refer Slide Time: 07:02)

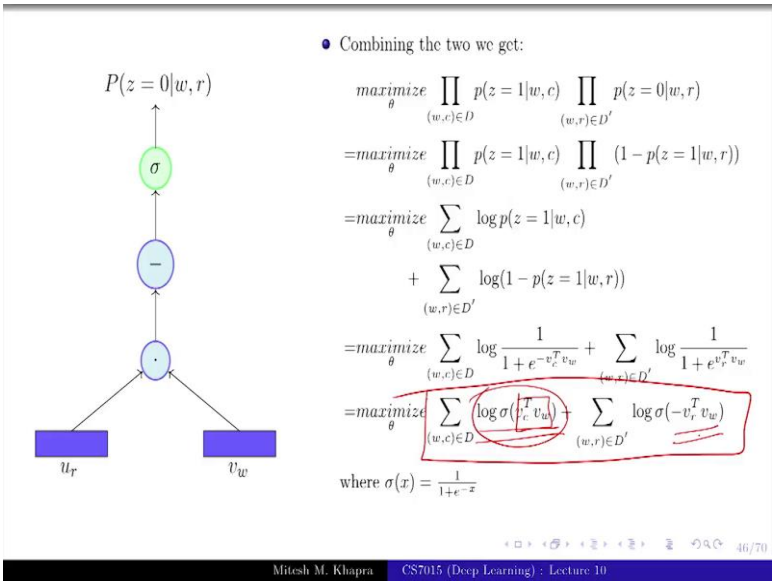


Now, for the other case wr belonging to D prime, what is it that I want to maximize? This probability right because I know this is an incorrect pair. So, I want my random variable to output 0. Ok?

Now, what is this going to be the probability 1 minus the probability, that it was correct. And that actually if you just simplify a bit it turns out to be this. Now for all the elements which belong to D prime, what is the objective function that I have? I want to maximize this for the first w comma r random pair for the second w comma are random pair and so on for all the random pairs in my corpus. So, it is just going to be a product of all these probabilities, is that fine? So now, what is my total objective function?

For every pair in D maximize that, for every pair in D and for every pair in D prime maximize the 0 probability. So, what is the total going to be is this fine.

(Refer Slide Time: 08:01)

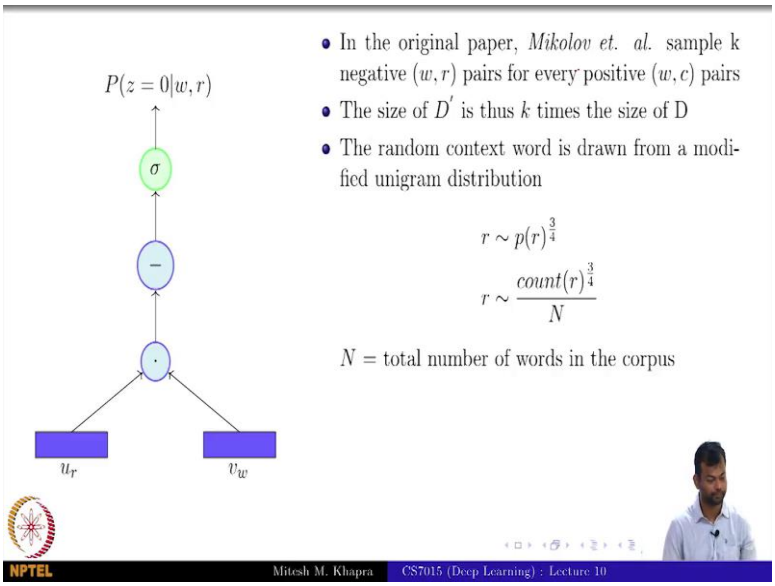


How many of you agree with this? So, for everything belonging to D I had this and rule for everything belonging D prime, I had another and rule and I am interested in both the acts right maximize for D and maximize for D prime of course, different quantities for D and D prime ok fine. So, you get this once you basically take the log and so on. So, this is a simple set of math operations that I do you will end up with this neat formula ok?

That for all the w comma c pairs belonging to D, you want to maximize this quantity which means; you want to maximize what? you want to maximize the when will this quantity be maximized. When the dot product between the 2 is high; that means, again what are you doing? We are trying to bring the context vectors close to the word vectors again transitively what will happen the words, which appear in the same context will go close to each other. What is the additional thing that you are ensuring here? The words which do not appear in the same context you are trying to push them apart, why? Because of second loss function.

You see the difference between the 2 now. In the first case you are only opt I mean you are obsessed with bringing things close together, here you are also focusing on the case that where you do not want certain things to be close together because they never appear in the same context is that fine. So, you see that this is a more powerful loss function in the earlier one. So, that is what the skip gram model does.

(Refer Slide Time: 09:27)



And in the original paper Mikolov et al sample k random pairs for every positive pair, right?

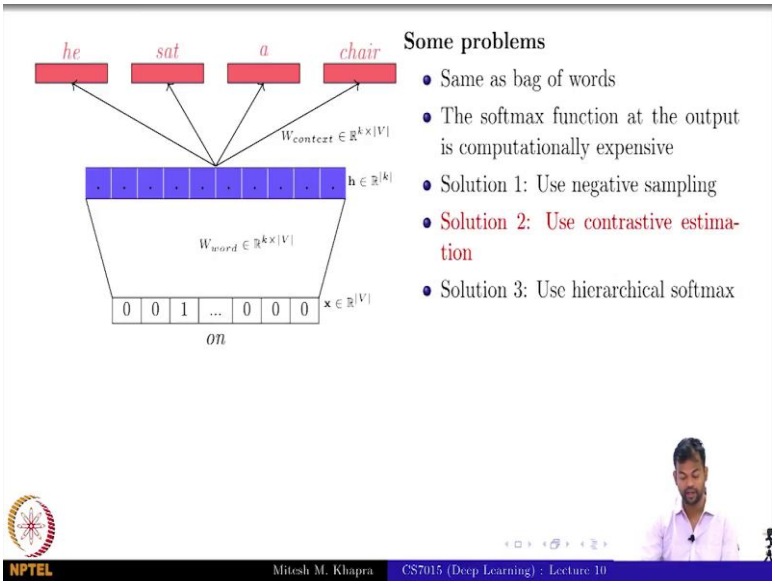
So; that means, if your size of D was n what was the size of D prime b k into n. So, that they had that many positive examples and k times that the number of negative examples. And this was a hyper parameter which was tuned and they used a value of k such that it gave them the best results. Also remember that we have this problem of constructing w comma r. Now I said that consider all the words which do not appear with your word, and sample from there and put something there. So, they used a slightly that means, how do I sample that? One is I assign all the words a uniform distribution that every word is equally likely, what is a better way of doing that? Okay I think I just finished this next time.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Module – 10.6
Lecture – 10
Contrastive estimation

So, we will move on to the next way of dealing with the expensive Softmax. So, remember that; so, this is known as Contrastive Estimation.

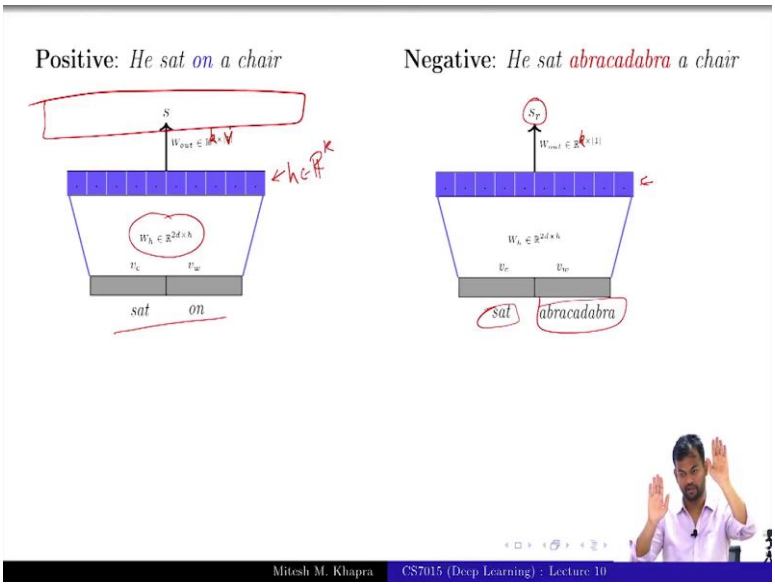
(Refer Slide Time: 00:24)



So, remember that this is where we are in the story that, we saw the bag of words model we saw the skip gram model. And we saw that both of them have this expensive Softmax computation at the end, and that is the problem we are trying to deal with. So, we saw one way of dealing with which was negative sampling. So, you I hope you saw that there was no expensive computation there.

The only computation there was the dot product between the 2 words, which appear together or which do not appear together. Now let us see what happens in contrastive estimation.

(Refer Slide Time: 00:51)



So, here again you use a same idea. So, you have a positive sentence or a positive example he sat on a chair. You create a negative sentence which you replace the word by some random word. Now you construct a feed forward network like this which takes these 2 one hot representations. Basically uses your word context matrix, to give you the summation of these two representations right. That is exactly what we have done in the skip gram model. Now you have this hidden representation which is the sum of the 2 word representations.

Now from here on instead of doing this Softmax computation which we had earlier; we just predict a single score ok. We just predict the score for this word pair being of correct word pair. we do the same thing with the random pair. So, we take sat we take abracadabra and the add up there word representations, you get this hidden representations and you get a score Sr fine. So, what is the output computation right now? What is the; is it a matrix operation? Is it a scalar operation? Is it a vector operation? What is this h is equal to? We need to change this to k on the slide please note. So, what is this product w into h, just a dot product between 2 vector.

W is just k cross 1, that means, it is a vector. So, as compared to k cross V earlier, we just have k cross 1, you get that? How many of you get this we have a very simple computation at the end, ok. but now how we set up by loss function? Earlier I could set up the loss function as maximizing the log like it of the correct word, but now I just

predicting 2 scores. So, what is the loss function? What should I try to intuitively do? And today we are going to see a new loss function which we have not seen earlier. So, try to think about this, what would you do? Forget about the math forget about the machine learning all that, what would you actually want? What is your wish list? That should be easy to characterize

Score S score Sr do you want this or this first one right, you want S to be greater than Sr. Can you think of making an objective function out of this you want to maximize.

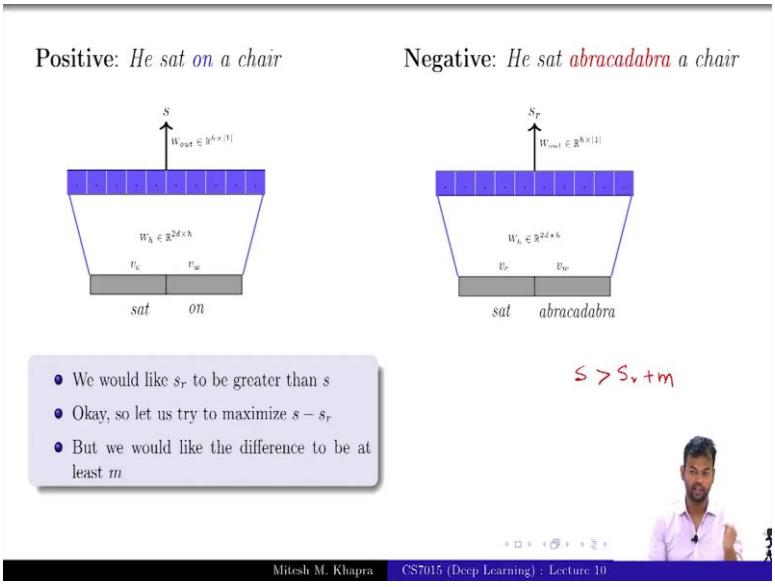
Student: (Refer Time: 03:18).

S minus Sr. Fine that is a good starting point. So, would you be happy with this? What would you want? This or this, both cases S is greater than Sr right, what would you want?

Student: A big margin.

A big margin fine

(Refer Slide Time: 03:39)



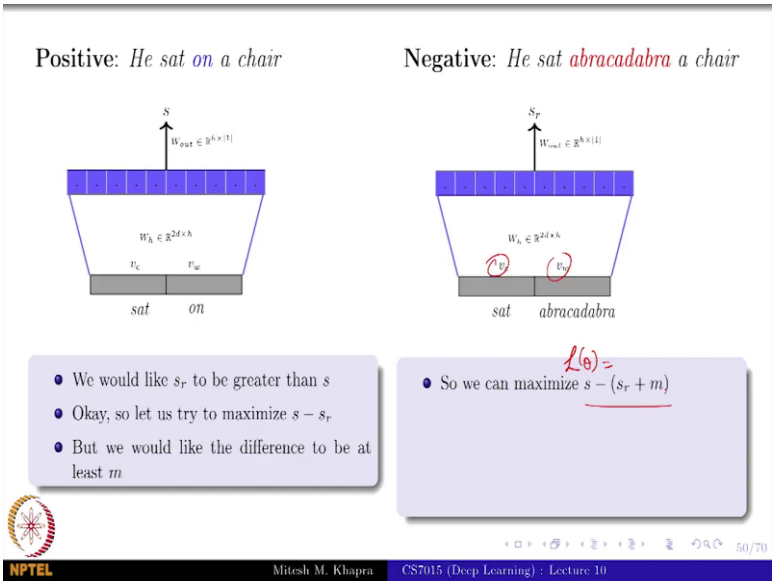
So, we would like Sr to be greater than S and not just. So, we could try to maximize S minus Sr,ok but we would also like this difference to be a certain margin; that means, I would want S to be greater than Sr, by at least a margin of m; and that m is something I will decide. So, I could say that it should be at least 10 points greater than Sr or 1 point

greater than Sr, depending on the scores that I have. So, all my scores are between 0 to 1 then probably a margin of 0.3 or 0.4 is ok sounds reasonable right; that means, S could be 0.6 and Sr could be 0.2 does that make sense?

So, what I am saying is, what I am trying to say is that; this is my Sr, I want S to be greater than Sr. I am not just happy with that I am saying that even if I add a margin to Sr even then this condition should hold right?

And that is the same as saying that S Sr and there should be at least a margin of m between that. That is the difference that I accept. I am not if you tell me that S is 0.99 and Sr is 0.98 where then you are not really distinguishing much. I want at least S to be 0.9 and Sr to be at least less than 0.5 or somewhere.

(Refer Slide Time: 04:55)

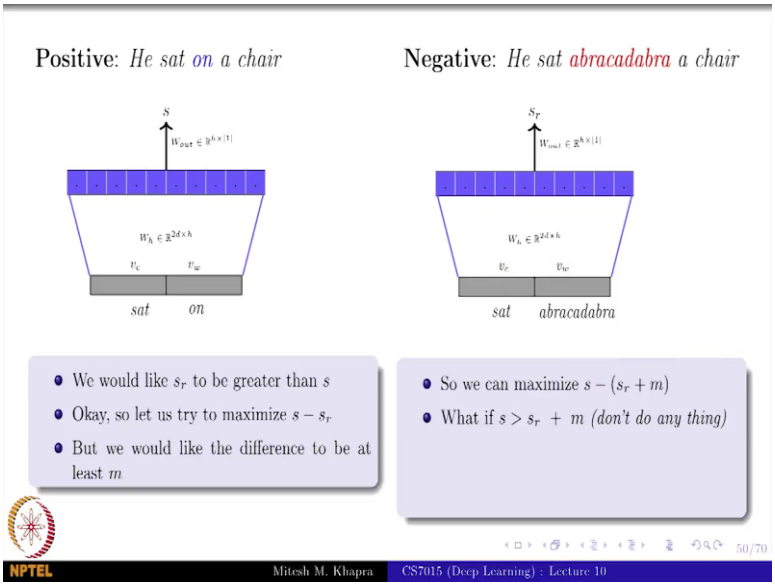


So, there should at least some gap between that and that gap is m. So, instead of maximizing S minus Sr, I am going to maximize S minus Sr plus m, is that fine? Ok. Now suppose you are at some point of training, I will have some need some parameter configuration; that means, you have learned some values for Vc and Vw. And you do this forward propagation compute S and Sr. And we actually find that this condition holds right. So, right now my loss function is this at some point you are doing this, and you observe that this condition holds; that means, S is actually greater than Sr plus m. In that case, what do you want a loss to be? How many of you get the question?

I want that S and Sr should be separated from a margin of m, in the favor of S. I am doing my training I am at certain configuration for Uc S and Vw S and so on. I pass it through the feed forward network and I get S and Sr. And I observe that this condition already holds.

Is my network doing anything wrong at this point? It is doing it is job properly? What should be the loss that I back propagate? 0. Again gets that there is nothing to correct here, I do not need to back propagate any loss.

(Refer Slide Time: 06:14)

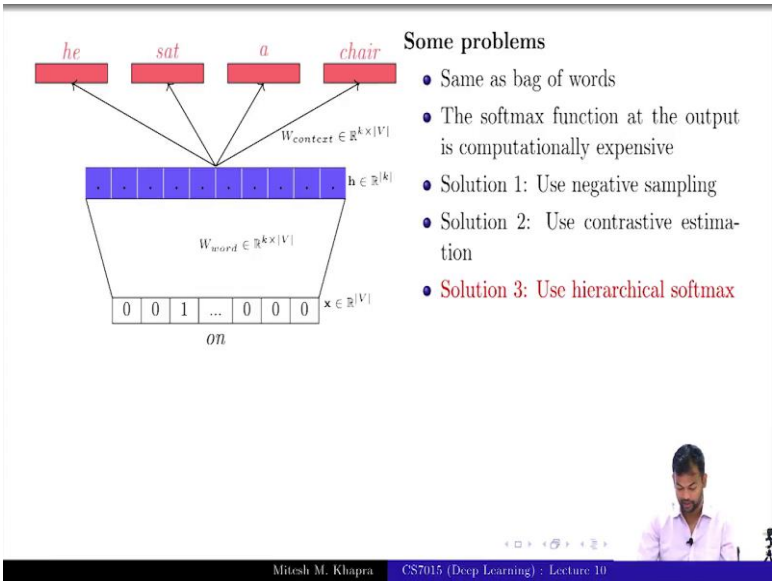


So, then can you give me the full objective function? Maximize this, but at this condition already holds then do not do anything is that fine? So, that is about this so and again observe that we have gotten rid of the expensive Softmax computation.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

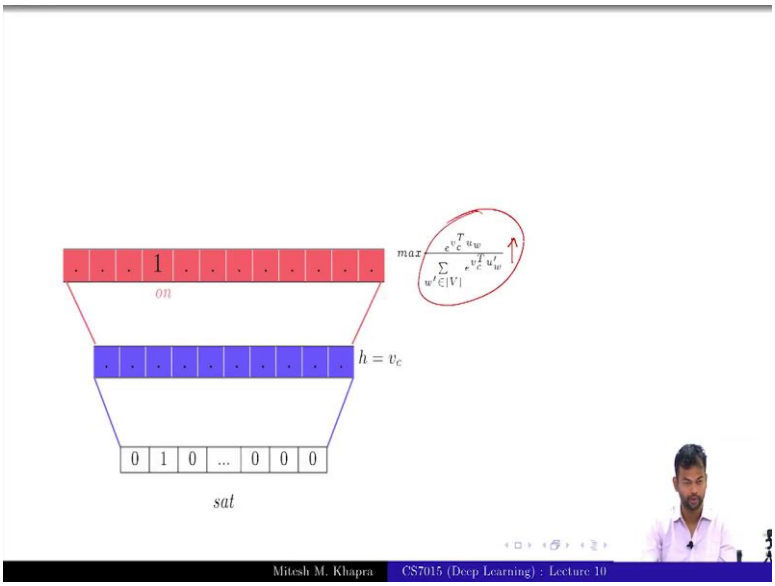
Module – 10.7
Lecture – 10
Hierarchical Softmax

(Refer Slide Time: 00:16)



The next one is a bit tricky so, the third solution is to use something known as Hierarchical Softmax. This is a bit counterintuitive in the sense it is a very smart trick, but it is not something which is very obvious. So, just pay a bit attention on this it is a neat way of handling this large vocabulary thing. And this I think used in various NLP applications, where speed is important not often, but wherever speed is important.

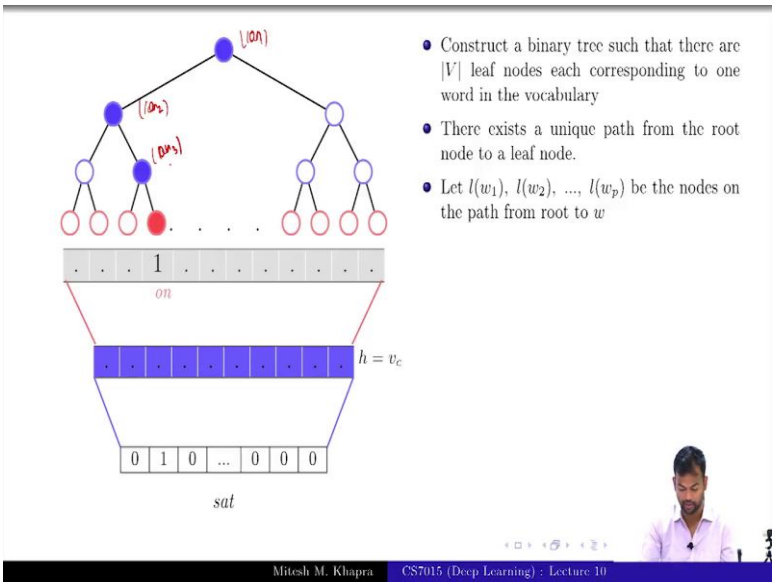
(Refer Slide Time: 00:35)



So, this is what our original network was. This was the either you take it as a skip gram model or you take it as a continuous bag of words model, right? Let us take it as a continuous bag of words model.

You had a word as the input, and then you had this large prediction, and you had this Softmax computation which gives you the probability, and you are trying to maximize this probability for the correct word, right where V w is the correct word?

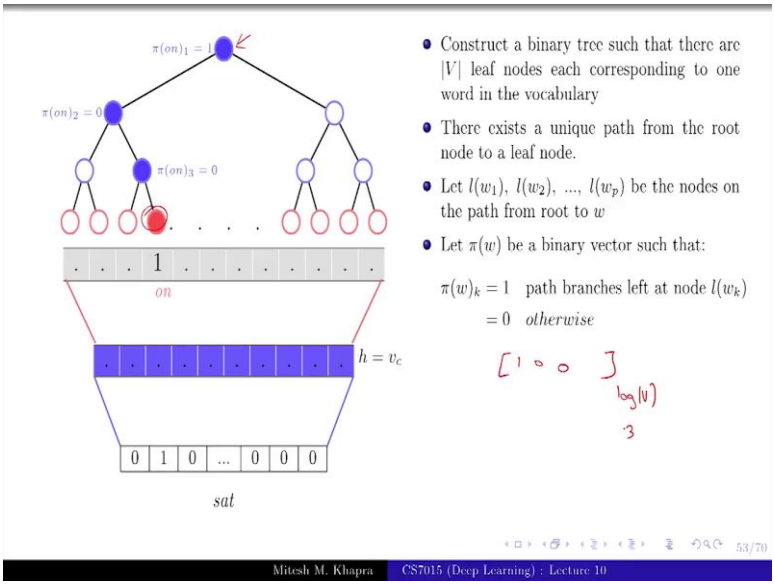
(Refer Slide Time: 01:04)



Now, instead of this the hierarchical Softmax says that you construct a binary tree such that your tree has how many nodes? V nodes, it has one node corresponding to every word, ok. And there exist a unique path from the root node to every leaf node. Every leaf node corresponds to a word and there is a unique path from the root node to leaf node. Of course, there will be overlapping things for example, for this word the path is these nodes, and for this word also the path is like there is some overlap in the path.

But for every word there is a unique path, how many if you get that setup. Now let lw 1 lw 2 up to lw p be the nodes on this path. So, I am calling this as lw1, lw2, lw3 sorry, sorry, sorry, sorry yeah actually it is. So, actually this is l on 1, l on 2, l on 3; that means the third node on the path of on, the second node on the path of on and so on, right that is how it is going to be and let pi w be a binary vector

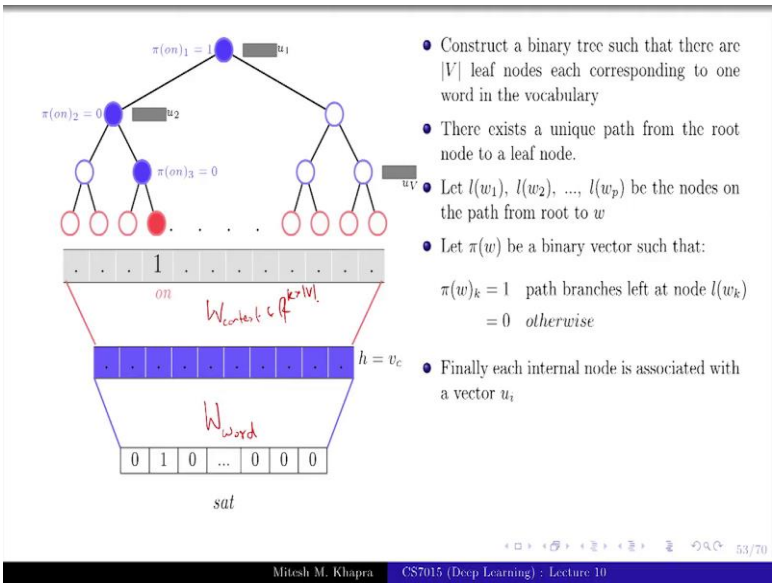
(Refer Slide Time: 02:15)



. So, what is the size of pi w actually binary tree log of v, right? So, the size of pi w vector is going to be log of v. So, if there are 8 leaf nodes, you will have 3 nodes as the size of the vector. So, for each of these things, this vector takes on a value 1 so, here the value would be one, because the path branches to the left if the path branches to right, then the value is going to be 0, right? So, for every node or every word I have this way of uniquely defining it is path, I can say that the path is 1 0 0, is that fine? For the word on the path is 1 0 0, if I consider some other word the path would be different, is that fine?

And of course, I have assumed there are only 8 words here, right that is why this holds if there are either otherwise I would have a vector whose size is log v, right now my V is 8 so, it is just 3.

(Refer Slide Time: 03:25)

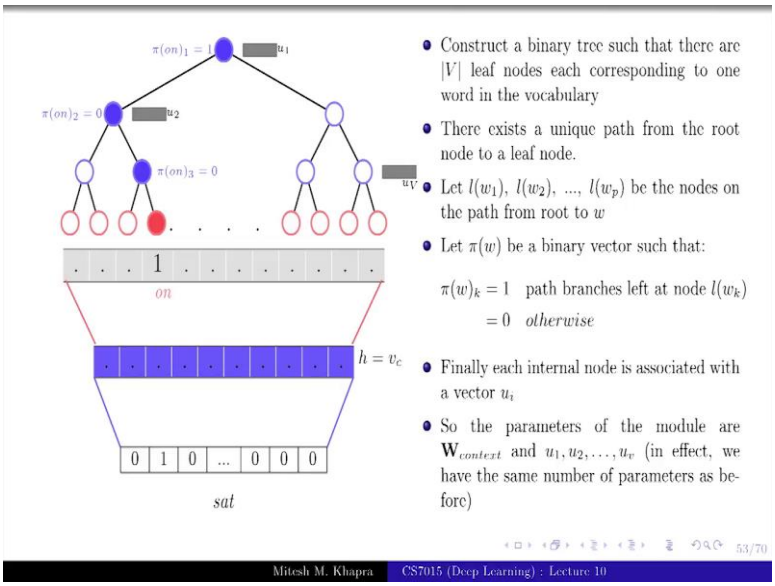


Finally, each of these internal nodes is associated with a vector, ok So, I have u 1 u 2 u 3. So, how many of these would I have? If there are V nodes at the leaf, how many non-leaf nodes do you have in the binary tree, V. you all know this, right?

So, if you have V nodes at the leaf then you will have V nodes internally. So, for each internal node, I have a vector associated with it. So, how many vectors do I have in all? U V and my input side is still the same, right? I have this w word or w context depending on whether it is a skip gram or by or continuous bag of words model.

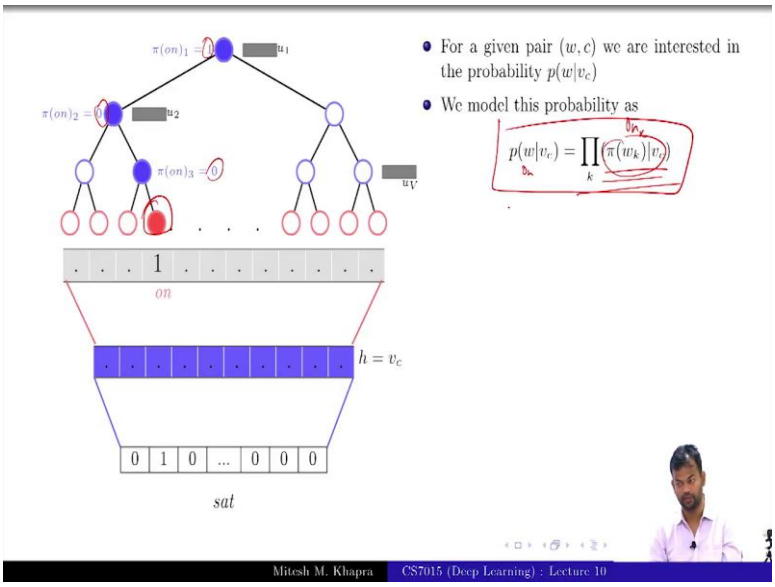
So, how many parameters does this model have? Is it same as the bag of words model or less than the bag of words model or more than the bag of words model? This is how you will think, you will see how many input parameters do the pool 2 models have, how many output parameters to the 2 models are input parameters, same output parameters, how many vectors do you have? U 1 to uv each of size k, same as the original model, right it is just as an original model I had put everything inside as w context which was k cross V, right? So, it is the same number of parameters.

(Refer Slide Time: 04:57)



So, the total number of parameters in the network is the same.

(Refer Slide Time: 05:01)



Now, for a given pair w comma c, which is the correct pair, we are interested in the probability p of w given vc, nothing great about this it is the same as I have been saying always that, we want the pa probability of w given c, what we are going to model as w given vc, because vc is the representation of c. And we model this probability now as the following thing, why does this make sense? You just assume this is on, and these are on k's, right? So, on 1 on 2 on 3, why does this make sense?

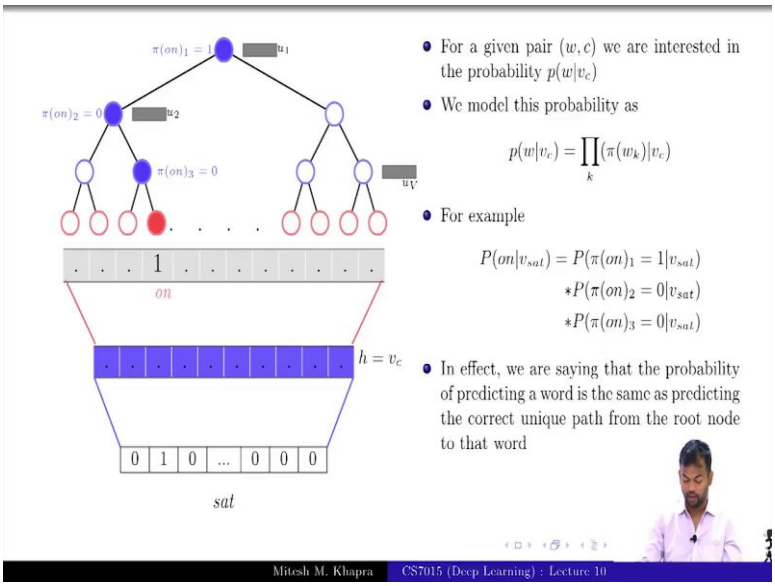
I will get the word on at the output only if the first element on the path was pi on 1 and the second element on the path was pi on 2 up to the k'th element on the path was pi on k. How many forget that? Please raise your hands, ok right? So, that is how we are modeling it. Is it? But what about pi on 1, pi on 2, pi on k, how do you model that? At least this form is clear to everyone, right if it is not let me know. Because then you not understand the rest of the stuff, yeah, ok.

So now see that modeling part is always in your hands, right? You know that you want you are interested in a certain probability; it depends on you how to model it. So now, what you have done is you have con constructed a binary tree. Now I am interested in p of on given some word vc, right or some word vector vc. Now I can say that, but the way I am thinking about this is that, I get the word on only if the first if I started from the root node, the first vector took on the value 1 or the first branch took on the value 1, the second branch took on the value 0, and the third branch took on the value 0. So, that is exactly what I am saying here.

It is a probability that the first turn that I took was a left turn, then a right turn then a right turn, yeah the path is you have constructed the binary tree, and the path is fixed now for all the words. How to construct the binary tree is a separate thing, but the binary tree has been constructed and every word has a unique path associated with that. So, that word will occur only if that path is executed, right? So, I am just trying to find the probability of that path being executed.

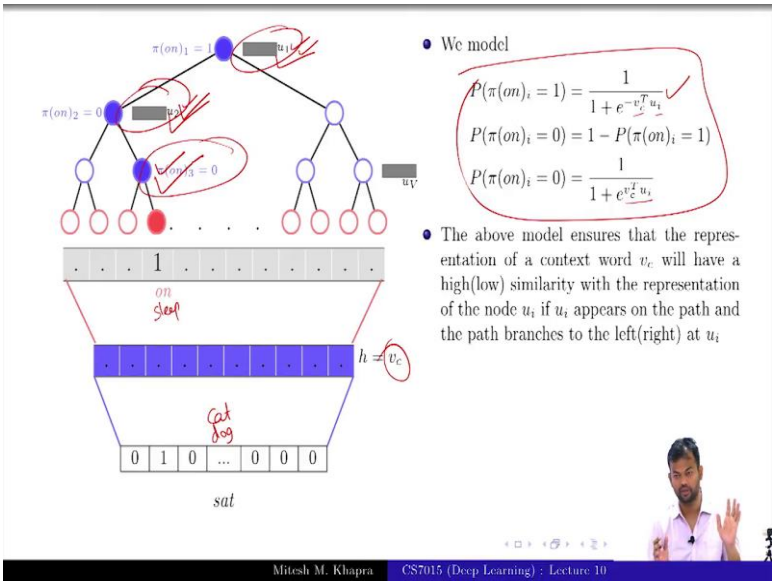
Now, I need to tell you what does each; so, how many terms are there in this product? K terms, right how do I estimate each of these k terms is what I need to tell you, ok. Can you think of it? How would I model each of these probabilities, remember that every node has a vector associated with it? How many if you can think of an answer? I hope I are you saying what I think you are saying.

(Refer Slide Time: 07:48)



So, this is what I will do. So, as I said for the on example this is what you want, this is the path that you want to be executed.

(Refer Slide Time: 07:59)



And I am going to model it as this.

So, getting a left turn, I model it using this that dot product between the original word vector which was the input word vector which was vc, and the node representation of the node associated with that particular node. Does this make sense? So, I will tell you what

we are trying to do. So, this path was clear that the probability is going to be a product of these probabilities.

Now I want how do I get each of these probabilities. So, that is again in my hand, right, I am going to say that, I am going to train my parameters vc and ui where ui is the parameter corresponding to every node. I am going to train it in a such a way that whenever I want this to take on the value 1, this should be close to 1, ok. Because I will set up my loss function accordingly, we will see the loss function.

But I am saying that whenever I want the probability to be equal to 1, I am going to use this to computed. And alternately when I want the probability to be 0, I am going to take 1 minus that which is just this, is that fine? Okay, let us go ahead a bit and then we will come back if you are still lost.

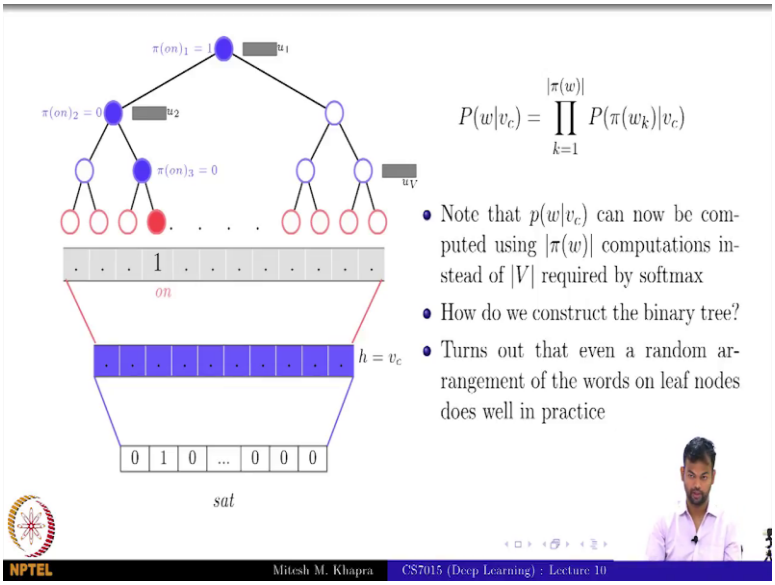
So, what does this actually ensure? This ensures that the representation of a context word vc will have a very high similarity with the node ui if the path takes a left turn there, and it will have a very low similarity with the node ui if the path takes a right turn their. How many if you get this part? Based on if you assume that this is how we are going to model it, when is this going to be high? When the dot product between vc and ui is high. When is this going to be low?

When the dot product between these 2 is low, right there is a negative, yeah. So, we, ok sorry I or rather when is this going to be low, right? So, you get that, so, it is coming so, the word representation which is vc which is this guy would come to the come close to all these representations, or move away from them depending on whether you want to take a left turn there or a right turn there?

Now, what would happen to words which appear in similar context? The same thing that we have been discussing so far, right they will come close to the node representations which are along the path, right is that fine? So, this is the context representation, right? This is actually you are representing every context word by these 3 representations. Now if a word appears in the same context, it is representation is going to either come close or move away from these representations,, right? So, words appear in the same context, if you have cat and you had sleep here, then cat has to come close to this it has to move away from this, and it has to move away from this. Is that clear? That is how we have set up the probabilities.

Now, instead if I had dog and again you had the context word as sleep. Now the representation of dog also has to go close to this. It has to move away from this, and it has to move away from this. So, in effect again the same thing is happening that the representation of cat and dog are moving in the same directions. So, they will eventually come close to each other, how many if you get this intuition?

(Refer Slide Time: 11:25)



And how many computations do you need now to compute the probability of this. So, earlier you acquired that complex Softmax computation, how many computations do you need? Now you definitely need these many computations. And each of these computations requires a sigmoid over or dot product, right? So, that is much much lesser than so, you just need these many dot products as compared to your expensive Softmax computation earlier.

So, you see how you get the savings using the hierarchical Softmax. So, this is as I said this is not very intuitive it is like a really smart trick, and it takes time to get your head around it, but I am sure if you go back and look at the slides you will get it, right? If it is if you have just got 50 percent of the idea here that is typically how it happens every time, but and I probably not figured out a better way of teaching this, but once you go back I am pretty sure that you will get to understand what is happening.

So now the question is how do we construct a binary tree? Anyone has any thoughts on that? Do we need to ensure certain things while constructing the binary tree? Okay, I will

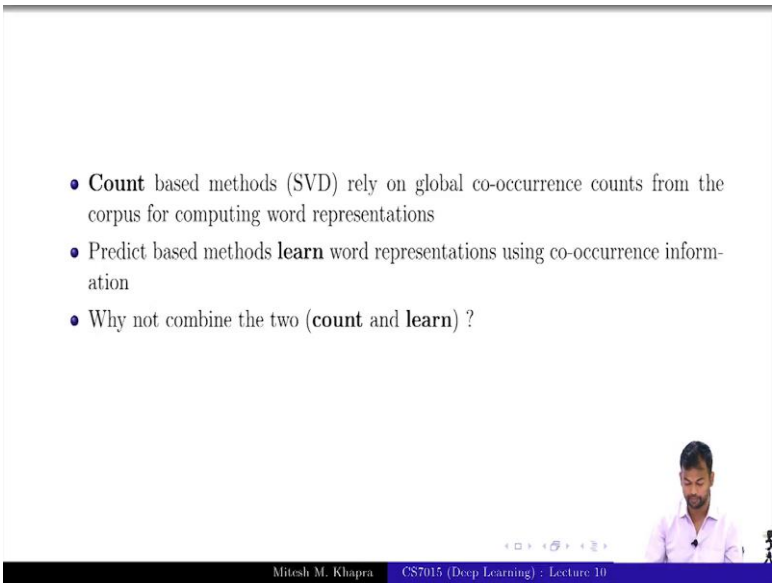
ask this as a quiz question, just note that. There is some subtlety here ah, in practice this is what is done. You just randomly arrange the nodes on the leaf nodes, and then you just construct a binary tree from there, right? So, you have distributed all your leaf nodes randomly, and on top of that you have constructed a binary tree. My question is there a problem in doing that which I will ask you on.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Module – 10.8
Lecture – 10
GloVe representations

So, now, from here we will move on to yet another way of learning word representations, which is known as the GloVe representations.

(Refer Slide Time: 00:20)

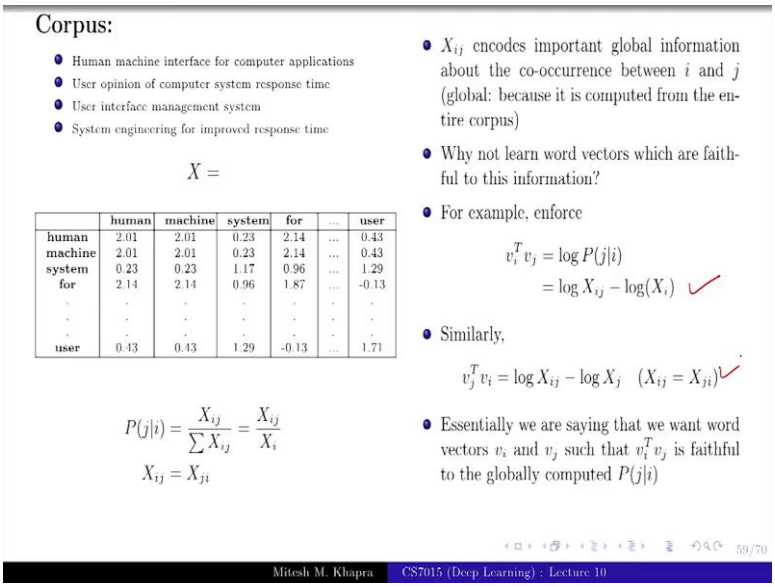


So, the count based methods rely on global co occurrence counts from the corpus for computing word representations that is what we saw in SVD. They look at these co occurrence counts and from there they build the word representations.

The predict based models, set up a learning problem where you have this feed forward net network and it tries to predict certain things from the given words and then you learn the parameters of that network, and you set up the task in such a way that the parameters actually correspond to word representations. So, this was the difference between count and predict based methods. Now what is the obvious next thing to do? Like hear the answer from a few of you, but I want to hear it from everyone.

What is the obvious next thing to do, you have count based methods you have predict based methods combine the two, right? So, come up with some kind of a hybrid. So, that is exactly what GloVe does which is known as global vectors.

(Refer Slide Time: 01:10)



So, I will go back to the co occurrence matrix. So, remember X ij, encodes the important global information about the word i and j and whether you replace it by pmi or ppmi or just keep the counts, it just gives you some information about how many times these two words actually appeared together.

So, X ij encodes this global information and I call it global, because it is computed from the entire corpus fine. Why not learn word vectors which are faithful to this information. So, what do I mean by that? Suppose v i is the representation of the i'th word and v j as a representative the j'th word, which I want to learn I do not have these representations I wanted to learn. Now, this gives me the dot product between them, which gives me the similarity between them. Why not I set up my task in such a way that this similarity is actually proportional to this probability?

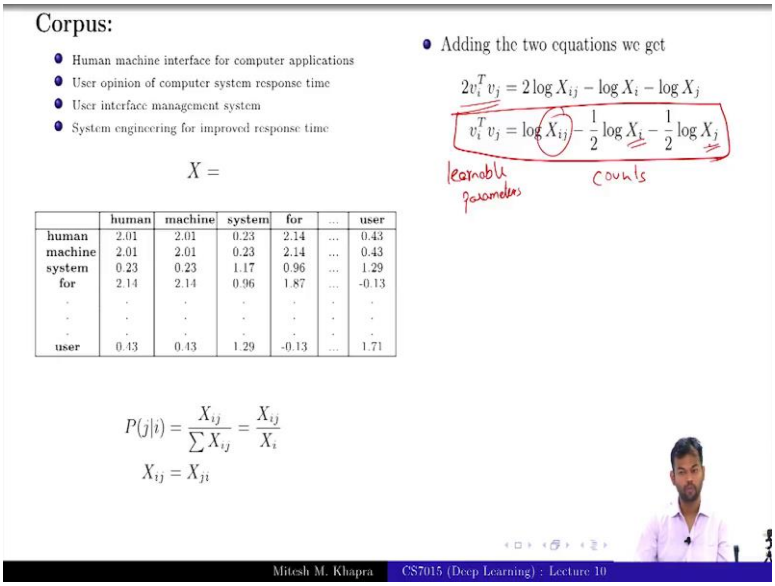
So, what does a similarly tell us, how well these two go together, what does p of j given I tell us, how likely j is given i right. So, does that make sense to have this analogy that, the dot product tells me the similarity the other notion of similarity is that how likely j is to appear in context of i which is given by p of j given i. So, why not set up my task such

that whatever vector as I learn are actually faithful to this global similarity that I have computed from the entire corpus how many if you get this intuition?

How many if you see the difference between this and the predict based models? In the prediction based models you are operating at one word pair at a time; here you are looking at these global counts, ok? We are trying to directly learn vectors which are faithful to your global similarity as given by your co occurrence counts. You get the merger between the two methods? You should not get it yet because we still have to do something or at least you get the intuition. Now what is p of j given i? It is actually this, ok?

So, I can write it as this ok and similarly I can write the other guy v j transpose v i and that is going to be different because that is going to have p i given j instead of p j given i. So, I will have log X ij is fine, but instead of X i I will have X j here.

(Refer Slide Time: 03:34)



	human	machine	system	for	...	user
human	2.01	2.01	0.23	2.14	...	0.43
machine	2.01	2.01	0.23	2.14	...	0.43
system	0.23	0.23	1.17	0.96	...	1.29
for	2.14	2.14	0.96	1.87	...	-0.13
.	.	.	.	.	.	.
.	.	.	.	.	.	.
.	.	.	.	.	.	.
user	0.43	0.43	1.29	-0.13	...	1.71

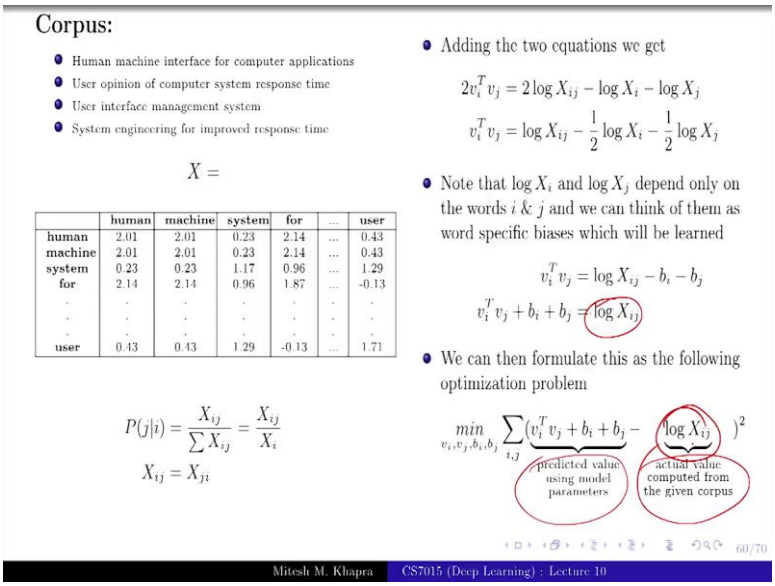
Now, if I add these two equations. So, I am going to add this equation and this equation. So, the left hand side I just get 2 times v i transpose v j, because v i transpose v j is the same as v j transpose v i and on the right hand side I get certain quantities. So, this is what I would actually want my word vectors to look at look like. I would want my word vectors to be such that, when I take their dot product they give me the quantity on the right hand side and this quantity has come based on counts learned from the corpus.

So, I have counts on the right hand side, and I have learnable parameters on the left hand side. So, you see how we are merging these two, but how do you learn this problem. Now it is ok to say, what I like what I have said now is that this is what I desire. I desire that my word vectors should be learned in such a way that they are faithful to the global counts through the following equation. This is what I desire. Desiring something is one thing, but now how do I set this up as a learning problem.

So, when I ask you what is the learning problem what do you need to think about? Objective function good that is a good start. So, what is the objective function for this, what are the parameters of the optimization you are optimizing with respect to what? The v i is and the v j's right all the word representations how many of those do you have? V each of size k. So, those are your parameters for optimization, now what is the loss function? If I give you the loss function it will look very very obvious, but I do not want to do that.

So, just continue thinking about that, while I will make some more simplifications to what we have here now, what is this count? This is the co occurrence count how many times these two occur together, what is this count? The number of times the word i appear. So, this depends only on i what is this count? The number of times the word g appears. So, to make the model more flexible; that means, give it some more freedom what I am going to do is instead of log x i and log x j I am going to introduce parameters b i and b j. ok

(Refer Slide Time: 05:32)



I am saying that these parameters can also be learned. So, effectively using all these three I should be able to get this, this is what I desire now set up the loss function. Using these two things come on that should not be so hard, what is this? This is what you are trying to predict; what is this? This is what you know is true because you have computed from the corpus; now can you come say the loss function the difference between these two right. So, you could have this as the loss function, this is the predicted value using models parameters, this is the actual value computed from the corpus.

So, think of this that you are trying to learn the parameters in such a way, that you end up predicting this and if you predicted this, you know you have done the right thing ok and this you know already because you have computed it from the corpus. So, this is the true value and this is the predicted value. So, as in any loss function predicted minus true the whole square, does that make sense how many if you are fine with this?

(Refer Slide Time: 06:44)

So, now how will you train in this network? Gradient descent. So, I will use gradient descent and you will get these parameters.

So, there is a bit more on this which I will not cover actually. So, I will just skip this slide, you can go back and take a look at it. It is a some slight modifications to this yes.

So, again the same idea that cat will go close to all the. So, here again you will have the v i and the uc s right. So, you will have cat will come close to all the words that it co occurs with, feline will also come close to the same words. So, maybe I have not used to right notation here if you need to change it again. So, we should have v i‘s and u j’s right? So, again you have one word matrix, word representations and the other is the context representation then it is fine right that is the problem here how many if you get that right? Again we have to have these two things let us change that everywhere.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

Module – 10.9
Lecture – 10
Evaluating word representations

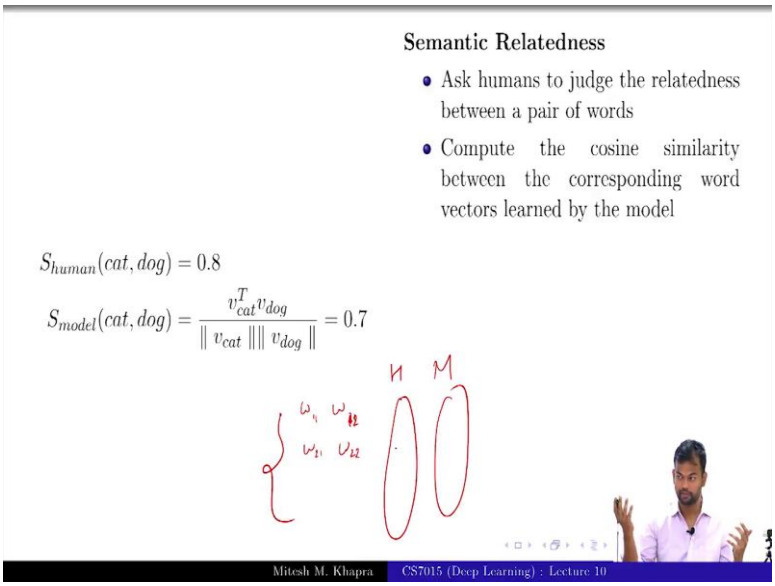
Now we come to this important part about how do you evaluate word representations.

(Refer Slide Time: 00:16)



So, there are different tasks that are set up, I hope some of you have read that paper and I can see that none of you have read that paper. So, semantic relatedness is one way of evaluating word representations.

(Refer Slide Time: 00:30)



So, ask humans to judge the relatedness between a pair of words. So, I construct some pairs of words, and I show them to a human. And ask them how related do you think they are on a scale of one 0 to 1. So, it is likely for cat and dog someone would say 0.8 or at least you would expect values greater than 0.6.

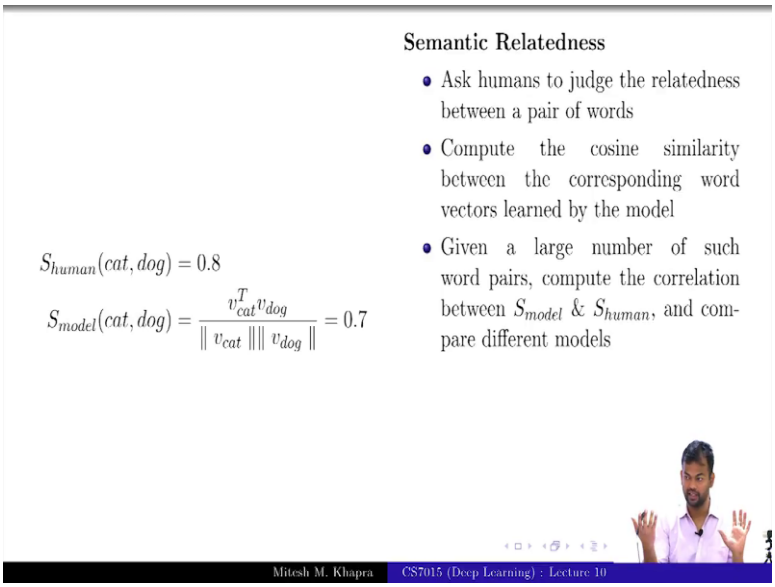
Now, you have learned the representations using your model. It could be any of the models that we have seen so far, continuous bag of words, skip gram or glove vectors. So, these are the three things right continuous bag of words, skip gram which is known as word2Vec and the glove representations. And within them you could have this hierarchical Softmax and other things and so on.

So, you could if I asked you what is the similarity between cat and dog, according to your word representations; You could just use the cosine similarity and tell me that this is the representation right. So now, I will have many search words w 1 w 2 for which I have the human judgment and I have the model judgment right. So, I will have w 1 1 w 2 one then w 2 1 sorry 1 2 and so on. I will have many such word pairs for each of these word pairs I would have the human judgments and I would have the model judgments right. How close do the humans think they are and how close do the think they are?

Now, I can compute the correlation between these 2 decisions or these 2 random variables. And I would want that for a good model this correlation should be high. So, whenever humans said that the 2 words are actually similar the models word vectors

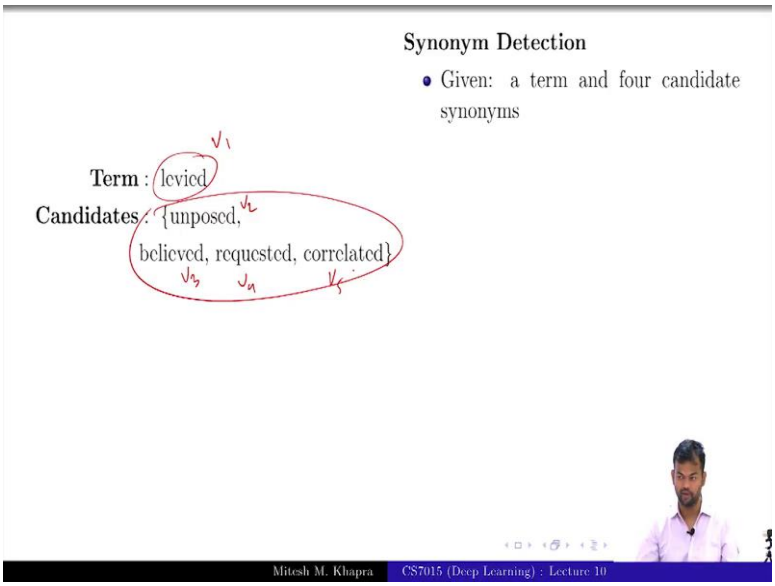
should also predict a high cosine similarity. And whenever humans said that the 2 words are not similar, the models word vector should also result in a low cosine similarity, how many forget this?

(Refer Slide Time: 02:07)



So, that is one way of evaluating how good your word representations are right. So, as I was saying earlier, how do you tune those parameters. So, you could have such a set, once you have learned some word representations and you want to see whether parameter k 1 was better than sorry, rather hyper parameter k 1 was better than hyper parameter k 2 you could just take those two word representations learned by these two different hyper parameter settings. Evaluate them on this corpus and whichever gives a higher correlation you can keep that hyper parameter; how many of you get that?

(Refer Slide Time: 02:38)



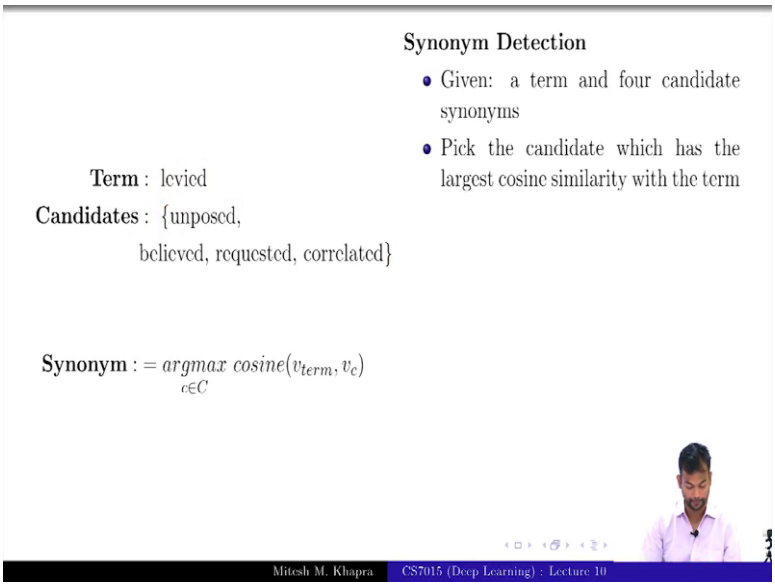
Other task is synonym detection. So, from a resource known as word net or from other dictionaries, you could get all the synonyms of a word. So, then people create a corpus where you give us in sin a word and give 4 candidates or some k candidates, out of which one of these is the correct synonym, the others are just distraction words right and distracting words. Now, what would you expect your word representations to do? You have word representations for all of these, what would you want? How would you pick up the synonym based on word representations?

Students: (Refer Time: 03:13).

The one which has the highest cosine similarity; so, again you will compute the cosine similarity, you will rank these and you will pick up the synonym right? And now again I gave you 100 such instances, I gave you a word for candidates and I gave you 100 such different word comma candidate pairs and you pick the synonym for everyone and see for 60 of them you got it right then your accuracy 60 percent. So, that tells you how good your word representations are.

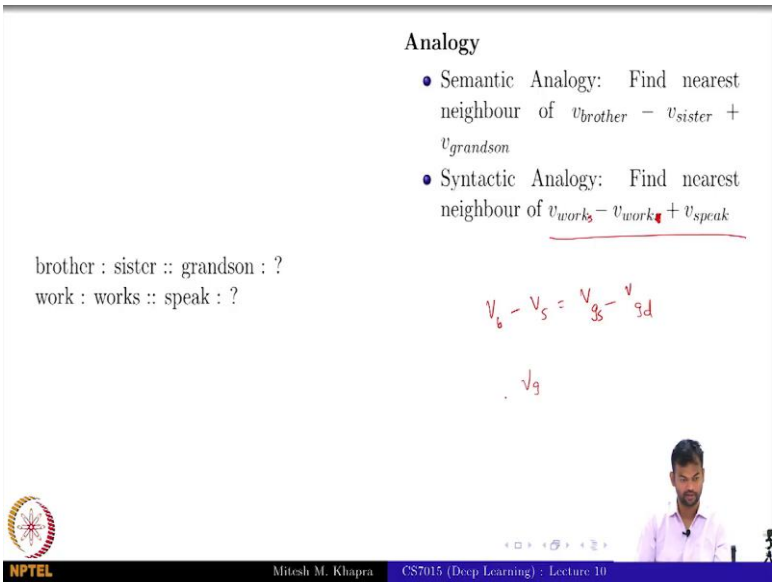
Again if you are given two different hyper parameter settings one gives you 60 percent accurate, the other gives you 70 percent accurate you will probably go with the one which gives you 70 percent accurate. They are find how you can use this.

(Refer Slide Time: 03:49)



The third is analogy task.

(Refer Slide Time: 03:50)



Where you find the nearest neighbor of this operation what should it be? Granddaughter this is this analogy teller brother is to sister as grandson is to something right. So now, the idea here is that if I mean it is like pretty weird right? So, if I take brother minus sister I get something.

Now, if I add grandson to that then I should get granddaughter. It is intuitive in a way right; I mean this is what you expect your word vectors to do right? So, that is how the

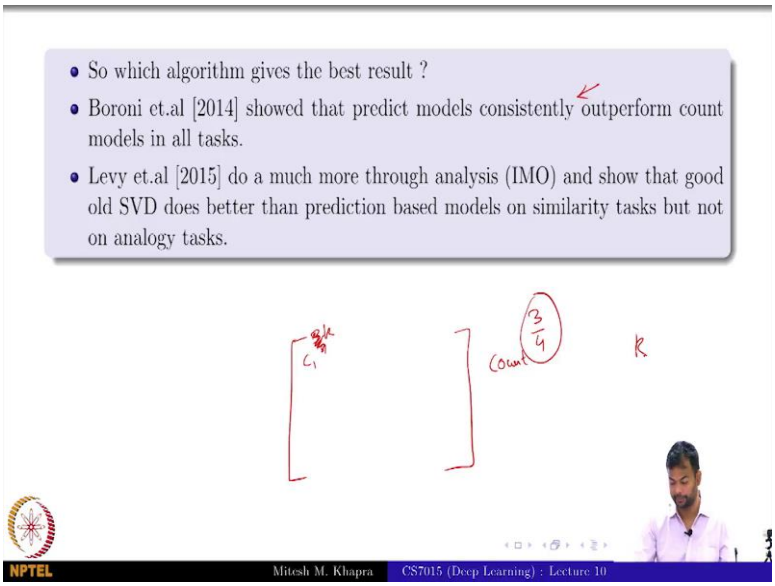
analogy task works. So, you could set up an analogy task, you could have and you could get several such an analogy tasks from online tests and so on. And you would want your word representations to exhibit this kind of a behavior right?

So, again you have these 100 analogy tasks, for each of these you know the true answer and from each of these you predict the answer from your word representations. And first see for how many of them you get it correct. Then you could also have a syntactic analogy. So, you can tell me what this would be right. In fact, here again it should be the other way round. We works minus we work, thus we speak would be we speaks right. So, that is the syntactic thing right?

So, you are getting a different form of the world. So, your word representations should also have this kind of properties, that is what you desire. So, just evaluating whether your word representations show this kind of a property or not. So, we have seen three tasks, one is semantic relatedness whether a pair of words how do humans rank it and how do the model how does the model rank it; Then the synonymous detection and the analogy tasks. In each of these you do something with the word representations in the first two you use the dot product, and this last one you use some arithmetic operation over the word representations.

So, you would want v brother minus v sister is equal to v grandson minus v granddaughter, right? So, v granddaughter, right? So, that is there is a plus minus error there.

(Refer Slide Time: 05:51)



So now which algorithm gives the best result right. So, whenever we see a bunch of algorithms same as we did with Adam and (Refer Time: 05:59) and so on we always want to answer this question, which of these gives the best result.

So, there was this study done by Boroni et. al in 2014, that show that the predict based models right. Which are either which are the predict based models actually; skip gram, continuous bag of words and even glove for that matter right because it is also a predict based model. These continuously or consistently outperform count based models that is what they said, but a year later there was a separate study done by someone. And in my opinion this was a more thorough analysis because the earlier study right, they did not really give SVD a chance to win in my opinion this is all on camera.

But the later the second set of guy right they gave SVD a chance to win. So, I will tell you one example of how they gave is really a chance to win. So, remember in word2Vec you had this weird 3 by 4, which you are using to raise the probability right. Now what they did is they said even in the co-occurrence matrix actually these counts that you have, if here you are using them by 3 by 4 in the case of word2Vec and getting better results, why not do the same thing in the co-occurrence matrix also?

At the end of the day you are raising the count to 3 by 4 right. So, whatever counts you have here based on that you will compute ppmi or pmi or whatever, but first why not try to adjust these counts. So, why not have a parameter k such that you can raise the counts

to this parameter and then do all those computation. And that is fair because the word2Vec has a parameter hyper parameter. So, why not give a similar hyper parameter to SVD.

Similarly, they did something to take care of the k negative samples which word2Vec has why not give SVD also similar chance, right? So, when they did these kind of adjustments, they found that after these modifications SVD does as well as or even better than word2Vec models for the similarity tasks, but not for the analogy task. But the analogy task was the last task right, brothers to sister's grandsons to grandmother, right?

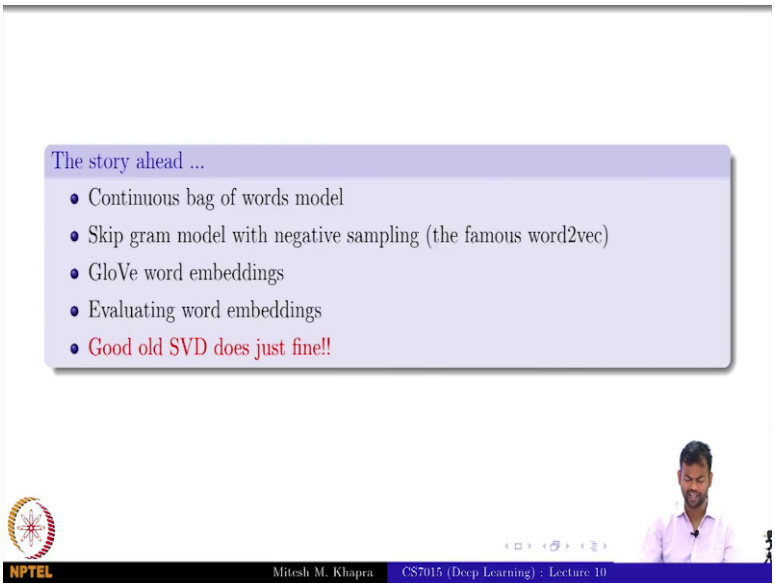
So, in most cases we care about similarity, and in very few cases we care about analogy if you are doing NLP application so; that means, in most cases SVD would just be fine. So, that is what I just said at the beginning.

Deep Learning
Prof. Mitesh M Khapra
Department of Computer Science Engineering
Indian Institute of Technology, Madras

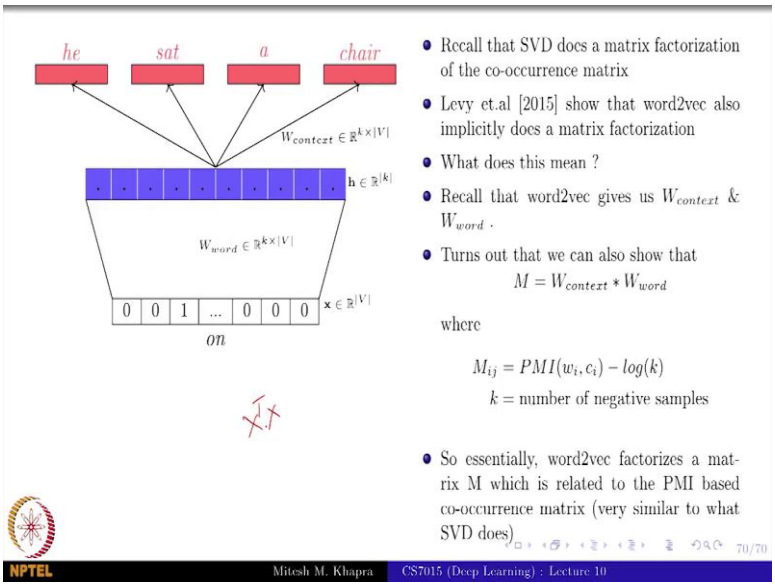
Module – 10.10
Lecture – 10
Relation between SVD and word2Vec

Now, and later on actually the same guys, they also came up with this formal relation between SVD and word2vec which is again under some assumptions.

(Refer Slide Time: 00:20)



(Refer Slide Time: 00:21)



But I am not going to do the proof here. I am just going to give you the intuition. So, recall that SVD does a matrix factorization of the co occurrence matrix levy et al showed that word2vec also does such a implicit matrix factorization. So, what does this mean? So, recall that word2vec gives us W context and W word. It gives us these 2 parameters. So, they say that there exist a matrix M such that ok this is wrong, just be the product of 2 matrices right this is the product of 2 matrices. It should be W context transpose, W word or just see which way the transpose should be.

So, it is actually a product of these 2 matrices that we have learnt ok and what is m m is actually nothing, but the PMI matrix minus this log k, where does the k come from? What was k? The negative samples that you have taken; so, they actually showed that whatever representations word2vec runs; it is actually doing a factorization of this matrix, where this matrix has a strong connection to the PMI matrix. And SVD also works with the PMI matrix.

If you take SVD matrix and do these modifications to it; that means, you take every value which is the PMI and then subtract this log k from that. And then just do an SVD of that you will essentially get back the same word representations as word2vec.

There was some certain assumptions made in the paper, but that is I mean, I do not want to go into those, but the key idea here is that, you can actually show that SVD and word2vec are actually connected. And if you think about it at an intuitive level, though

these methods are relying on the same underlying principle that words appear together. Based on that, the word representations get updated or in SVD based on there the counts get updated and you then eventually end up with certain representation.

Next the underlying principle is the same. So, there has to be a connection right it is not that they are doing fundamentally something different both of them are relying on the idea of co occurrence or the idea of distribution right? So, they have to at some level be similar in some ways, right? So, that is what they finally, showed and so, now, but still in most applications word2vec is preferred.

So, one reason for that is, that this is an iterative training procedure right as compared to SVD and I come back to your question, right? How do you do that? How do you compute the eigenvectors of X transpose X? And the answer is, there is no simple way of doing that and you have to do that expensive matrix multiplication.

And then rely on various very smart libraries for computing the eigenvectors which are still order n raise to 2 point something or something like that they not order n cube, but they are still order n raise to 2 point something, means they are still expensive. And then of course, you have this memory issue that if you have a very large vocabulary, your PMI matrix is going to be very high dimensional. And then you need to do the factorization of that high dimensional vectors,. So, that runs into these computational efficiency issues.

On the other hand, word2vec by design is an iterative algorithm because, you are going to grade gradient descent which is that every times that you are going to update some parameters of the model. You are not learning all the parameters together. You are only dealing with some parameters at every time set right. So, that is more computationally efficient. Especially, if you do the contrastive divergents or the negative sampling or the hierarchal sample; so, that is why, perhaps it is still more popular than SVD.